

Admela Jukan
Xavi Masip-Bruin
Jasenka Dizdarević
Francisco Carpio

Network of Things Engineering (NoTE) Lab

Network of Things Engineering (NoTE) Lab

Admela Jukan · Xavi Masip-Bruin ·
Jasenka Dizdarević · Francisco Carpio

Network of Things Engineering (NoTE) Lab

Admela Jukan
Technische Universität Braunschweig
Braunschweig, Germany

Xavi Masip-Bruin
Universitat Politècnica de Catalunya
Vilanova i la Geltrú, Spain

Jasenka Dizdarević
Technische Universität Braunschweig
Braunschweig, Germany

Francisco Carpio
Technische Universität Braunschweig
Braunschweig, Germany

ISBN 978-3-031-20634-4 ISBN 978-3-031-20635-1 (eBook)
<https://doi.org/10.1007/978-3-031-20635-1>

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors, and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

Preface

Welcome to the first edition of Network of Things Engineering (NoTE) Lab! This book is designed to guide the reader through a set of laboratory exercises in a real-world lab setting that combines the Internet of Things (IoT), computing systems, and networking. The importance of training students and engineers alike in these fields can hardly be overstated: from data analytics applications for connected sensors, over to edge and cloud computing, networks of "things" have become the essential enabling technology. NoTE Lab addresses the demand to educate future generations of computer engineers and scientists specifically in the architectural and system foundations. These foundations not only encompass IoT, edge, and cloud computing, but also the related networking and communication protocols.

By putting concepts of networking and computing software with devices and hardware in practice, this book gives the readers hands-on experience in software and hardware development of IoT devices in edge and cloud computing systems. With its focus on system architecture foundations, NoTE Lab complements many existing hands-on IoT courses and tutorials used already to teach students how to set up a myriad of embedded device laboratories. In addition to offering lab assignments and engineering tasks to its advanced readers, this book also offers introductory lab exercises designed for explaining sensing and actuating hardware devices to novice readers. The focus on system architecture experience, rather than on the technology design of the devices, was rather important to us when developing this book. We believe that system architecture focus advances knowledge critical to effectively extending today's networking and computing systems toward new hardware, software, and applications, as they evolve.

Target Audience

This book is designed for undergraduate and graduate course levels in computer engineering and computer science. When preparing this book, we found that scoping the learning framework was most critically important to motivating the undergraduate

students in particular. Many undergraduate students have little or no experience with typical IoT devices hardware or computer system software. We provide detailed and workable instructions about the hardware capabilities of IoT devices and the related easy-to-understand schematics on how to connect them without assumptions on specific prerequisites. We furthermore provide instructions that can help students learn how to use popular devices (such as Raspberry Pis) in combination with also popular cloud computing tools, thus creating an edge computing system. We provide instructions on a few basic computer network configurations, such as in local and wide area networks.

Graduate students can get familiar with a few communication protocols that co-exist in NoTE Lab individual lab modules, including MQTT (Message Queue Telemetry Transport), HTTP (HyperText Transfer Protocol), and AMQP (Advanced Message Queuing Protocol). The individual lab exercises are also defined with *assignments* for graduate students to stimulate and help acquire the engineering problem-solving skills in various domains. For instance, through lab module assignments, graduate students can experimentally study the performance of communication protocols in terms of delay. The lab assignments are optional for undergraduate students.

All students are expected to use both low- and high-level programming languages. For example, C++/Arduino is used on IoT devices and Python on Raspberry Pi. Overall, it is advantageous but not mandatory to acquire basic programming skills. For undergraduate students, this book provides all programming scripts and tools required in individual labs and tasks. This is also helpful to course instructors and tutors, so they do not have to dwell on system administration details. For graduate students, we define assignments that often require changing the existing scripts and configurations, thus stimulating their interest in developing their programming skills further. Finally, readers are introduced to a few open-source tools that are widely used in cloud computing today.

What NoTE Lab Does Not Offer

The knowledge gained through our system architecture approach comes at a price, i.e., NoTE Lab does not teach details of underlying technologies, protocols, or systems. For instance, this book does not cover the basics of Internet addressing and routing. Also, the programming techniques do not receive any attention in this book; it is with no doubt possible to better code all the instructions we herewith provide, but this is not our scope here. The book does not provide any solid foundation in cloud computing tools either, which we use in our labs as examples only. The same applies to various operating systems and computer architectures. Interested readers are however provided a few stimulating assignments in each lab module, as well as pointers to relevant sources of information.

Choosing Hardware and Software Solutions

NoTE Lab implements a few popular hardware solutions along with free and open-source software implementations.

In the IoT context, we choose Arduino-based boards which support a variety of low-cost sensors and actuators. When programming for this type of board, we use Arduino sketches to run as main programs using external libraries, and we also make use of Arduino's Object Oriented Programming capabilities by writing our own implementation of libraries to facilitate the modularity between different parts of the book. As a development environment, we use the official Arduino IDE, but other compatible IDEs or plain text editors may be used as well.

In the edge computing context, NoTE Lab implements off-the-shelf single board computers, with their corresponding operating systems that need to be preinstalled. Raspberry Pis are the most popular solution, due to their low-cost, compact design, and, most amazingly, their powerful computing features and interfaces. In NoTE Lab, Raspberry Pis are used as examples of edge computing resources to process sensor data.

NoTE Lab does not use commercial cloud computing platforms to allocate virtual machines, since current platforms typically charge for their computing services. Instead, we propose that local servers are deployed with known and widely used cloud computing open-source tools, for example, Kubernetes. In these ways, students can learn the basics of monitoring and managing containers in cloud computing, which is our main goal. The lab instructions we use on local servers are the same as if commercial cloud platforms were used.

Devices and Configuration

We introduce three types of devices conceptually, i.e., *IoT devices*, *Edge constrained devices*, and *Cloud/Edge servers*. This classification is specific to NoTE Lab and is made to distinguish between the device's communication, storage, and processing capabilities; see Fig. 1a. The resource constrained *IoT devices* assume no storage, low processing capabilities, and low communication capabilities and are mainly focused on sensing and actuating. For instance, NoTE Lab implements different sets of sensors and actuators using Arduino Uno WiFi Rev2 boards. *Edge constrained devices* assume low-to-medium storage and higher processing and communication capabilities than IoT, such as in Raspberry Pis. Finally, *Cloud /edge servers* offer the highest storage and processing capabilities in the system, which can be implemented using virtual machines in the cloud, dedicated servers, or standalone desktop computers.

From the networking and protocol configuration perspective, see Fig. 1b, we can see that IoT and edge constrained devices are placed into a local area network, while the communication to an edge/cloud server is implemented through a wide

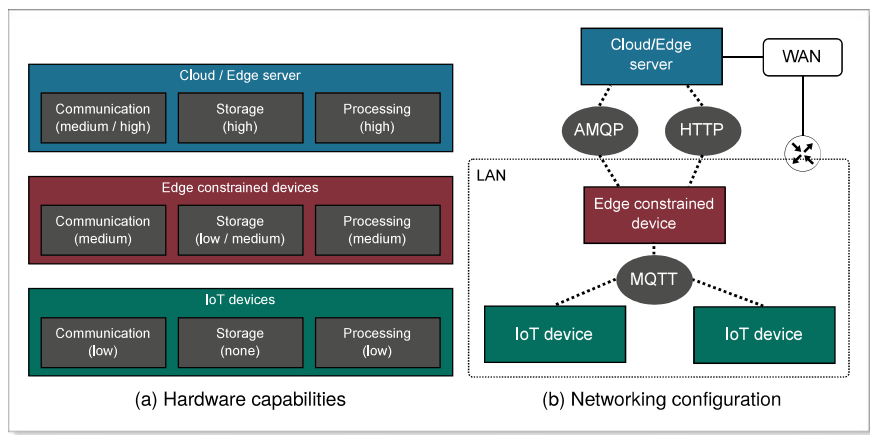


Fig. 1 Devices and configurations

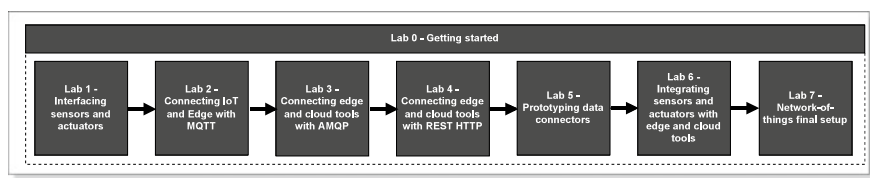


Fig. 2 NoTE Lab modules

area network. In NoTE Lab, we choose MQTT, HTTP, and AMQP as communication protocols between various categories of devices; IoT and edge constrained devices implement MQTT, while HTTP and AMQP protocols are implemented in the cloud/edge server. Choosing these protocols for implementation was one of the key decisions in NoTE Lab. It is well known that no single protocol is suitable for every part of the system. We hence choose different protocols to co-exist in various devices, i.e., MQTT, HTTP, and AMQP, just like in a fully engineered, real-world IoT/edge/cloud computing system.

Overview of the Lab Modules

NoTE Lab offers seven lab modules, including one introductory lab and six main labs, as illustrated in Fig. 2, i.e.,

- *Lab 0—Getting started*—This is an introductory lab module, where configuration instructions for all devices used in NoTE Lab as well as introductory information relevant to various software tools and libraries are given in form of a tutorial.
- *Lab 1—Interfacing sensors and actuators*—This lab module familiarizes readers with Arduino configuration using Arduino IDE, as well as explains methods to program different functionalities for different types of sensors.
- *Lab 2—Connecting IoT and Edge with MQTT*—This lab module establishes communication between the Arduinos while exchanging sensor data. Arduinos are programmed to run MQTT clients, while Raspberry Pis implement MQTT broker.
- *Lab 3—Connecting edge and cloud tools with AMQP*—The main goal in this lab is to deploy and test AMQP protocol. The concept of *cloud/edge server* is introduced and configured to run an AMQP broker, while an AMQP client is implemented on Raspberry Pi.
- *Lab 4—Connecting edge and cloud tools with REST HTTP*—This lab implements and tests HTTP server on the server machine and an HTTP client on Raspberry Pi. In addition, it introduces a known cloud computing tool Elasticsearch.
- *Lab 5—Prototyping data connectors*—In this lab, we implement data connectors between various clients, brokers, and servers, while deploying MQTT, AMQP, and HTTP protocols. It also introduces a cloud computing tool Logstash for data pipelining.
- *Lab 6—Integrating sensors and actuators with edge and cloud tools*—This lab integrates the software prototyped as well as the cloud computing tools with real-world sensors on the one hand, and with actuators on the other.
- *Lab 7—Network-of-Things final setup*—This lab module implements an end-to-end multi-protocol connectivity among all devices in the system, thus combining all previous labs in an integrated IoT/Edge/Cloud system.

Structuring Individual Labs

All individual labs are structured to include Prerequisites, Tasks, and Assignments, i.e.,

- *Prerequisites*: Before starting an individual lab, students are introduced to its prerequisites, related concepts, and the lab hardware and software setup.
- *Tasks*: Each lab includes a number of practical experiments and tasks which need to be completed.
- *Assignments*: A set of assignments is included per each task, to be completed and submitted to the instructor in form of a lab report.

Table 1 Grading examples for lab reports

Assignment	Percentage
Lab 0 assignments	5%
Lab 1 assignments	10%
Lab 2 assignments	10%
Lab 3 assignments	10%
Lab 4 assignments	10%
Lab 5 assignments	15%
Lab 6 assignments	20%
Lab 7 assignments	20%

Grading

The instructors can distinguish between undergraduate and graduate grading. Undergraduate students may be asked to successfully complete each lab module, along with prerequisites and tasks to pass the course, i.e., without assignments. For graduate students, the instructor may define that in order to successfully complete each lab, the assignments need to be finished with a score of 60% or better to pass the course. This requires an evaluation of the lab reports. An example of a grading system for lab assignments is given below (Table 1).

Free and Open Source

NoTE Lab implements only Free and Open-Source software, including operating systems and related tools, and whenever possible also open-source hardware. All code implemented for the realization of this book is also free and open source, available at

- Github: <https://github.com/tubskns/notelab-code>
- Gitlab: <https://gitlab.com/tubskns/notelab-code>
- Sourcehut: <https://git.sr.ht/tubskns/notelab-code>

Acknowledgements

- Circuits in this book, including sensors and boards, are created using <https://github.com/fritzingFritzing>.
- Figures for this book were created using <https://github.com/jgraph/drawiodraw.io>.
- We are thankful to the free and open-source community, which makes experimental and educational labs like NoTE Lab possible.

- We are thankful to our undergraduate and graduate students who tested the lab modules and provided feedback.
- We thank Professor Joerg Liebeherr (University of Toronto), who tested the early version of NoTE Lab and offered encouragement and help.
- We thank Manu, a 14-year-old secondary school student from Spain, who visited us and successfully tested labs 0, 1, and 2.

Braunschweig, Germany
Vilanova i la Geltrú, Spain
Braunschweig, Germany
Braunschweig, Germany

Admela Jukan
Xavi Masip-Bruin
Jasenka Dizdarević
Francisco Carpio

About this Book

This book provides a hands-on experience in software and hardware engineering of IoT devices in edge and cloud computing systems, by putting in practice state-of-the-art concepts of hardware devices, networking and computing software. It proposes a Network of Things Engineering (NoTE) Lab, with seven hands-on lab modules covering topics ranging from “Interfacing sensors and actuators” and “Connecting IoT and Edge with MQTT” to “Data pipelining in cloud computing”. All tools and software used in the NoTE Lab are free and open source, and available to the readers. Specifically, Arduino-based boards that support a variety of low-cost sensors and actuators are used in IoT context. In edge computing, NoTE Lab implements off-the-shelf single board computers, Raspberry Pis with corresponding software and hardware. For cloud, well-known and widely used cloud computing open-source tools (e.g., Kubernetes) are deployed, where readers can learn the basics of monitoring and managing containers in cloud computing. Three communication protocols are used in the end-to-end setup, including MQTT, AMQP and HTTP. This lab book is a “must experiment with” for anybody in academia and industry participating in the fascinating IoT-edge-cloud continuum development.

Contents

1	Introduction	1
1.1	Hardware Setup	1
1.2	Communication Protocols	2
1.3	On Software Tools and Libraries	4
1.4	Lab Structure and Learning Objectives	4
2	NoteLab Setup Requirements	9
2.1	Hardware Requirements	9
2.1.1	Sensors	9
2.1.2	Actuators and Auxiliary Components	12
2.1.3	Auxiliary Components	13
2.1.4	Single Board Microcontrollers	14
2.1.5	Single Board Computers	14
2.1.6	Server	15
2.1.7	Workstation	16
2.1.8	WiFi Router	16
2.2	Software Requirements	17
2.2.1	Arduino Uno WiFi	17
2.2.2	Raspberry Pis	18
2.2.3	Server	19
2.2.4	Workstation	19
	References	20
3	Lab 0—Getting Started	23
3.1	Prerequisites	23
3.2	Setup	24
3.3	Part 1: On the Workstation	25
3.3.1	Task 1: Getting Started with Linux Command Line Interface (CLI)	25
3.3.2	Task 2: Wireless Router Configuration	28
3.3.3	Task 3: Remote Access Using SSH	32
3.4	Part 2: Introduction to Arduino Programing	33

3.4.1	Task 4: First Arduino Sketch	33
3.4.2	Task 5: Connect Arduino to the WLAN	36
3.5	Part 3: Getting Started with Raspberry Pi	41
3.5.1	Task 6: The First Python Script	41
3.5.2	Task 7: First Steps with Docker	44
3.6	Part 4: On the Server	46
3.6.1	Task 8: Getting Started with Kubernetes	46
	References	49
4	Lab 1—Interfacing Sensors and Actuators	51
4.1	Prerequisites	51
4.2	Setup	52
4.3	Part 1: Actuating an External LED	53
4.3.1	Task 1: Connecting External LED	53
4.3.2	Task 2: Making LED Blink	54
4.4	Part 2: Actuating an LCD Display	55
4.4.1	Task 3: Connecting LCD Display	55
4.4.2	Task 4: Printing Messages on LCD Display	56
4.5	Part 3: Temperature/Humidity Sensor	58
4.5.1	Task 5: Connecting DHT11 Sensor	58
4.5.2	Task 6: Reading Temperature and Humidity	59
4.6	Part 4: Ultrasonic Distance Sensor	61
4.6.1	Task 7: Connecting HC-SR04 Sensor	62
4.6.2	Task 8: Calculating Distance	63
4.7	Part 5: Motion Sensor	67
4.7.1	Task 9: Connecting HC-SR501 Sensor	67
4.7.2	Task 10: Detecting Motion	68
	References	71
5	Lab 2—Connecting IoT and Edge with MQTT	73
5.1	Prerequisites	73
5.2	Setup	74
5.3	Part 1: Deploying and Testing an MQTT Broker	76
5.3.1	Task 1: Running Mosquitto	76
5.4	Part 2: Implementing an MQTT Client in Arduino	78
5.4.1	Task 2: Implementing an MQTT Client	78
5.5	Part 3: Sensing and Actuating over MQTT	85
5.5.1	Task 3: Sending DHT11 Sensor Data over MQTT	86
5.5.2	Task 4: Activating an External LED and LCD Display over MQTT	90
	References	95

6	Lab 3—Connecting Edge and Cloud Tools with AMQP	97
6.1	Prerequisites	97
6.2	Setup	98
6.3	Part 1: Deploy and Test RabbitMQ	99
6.3.1	Task 1: Deploying RabbitMQ	99
6.3.2	Task 2: Testing RabbitMQ	102
6.4	Part 2: Implementing an AMQP Client in Python	104
6.4.1	Task 3: Implementing an AMQP Client	104
	References	110
7	Lab 4—Connecting Edge and Cloud Tools with REST HTTP	111
7.1	Prerequisites	111
7.2	Setup	112
7.3	Part 1: Deploy and Test Elasticsearch	113
7.3.1	Task 1: Deploying Elasticsearch	113
7.3.2	Task 2: Testing Elasticsearch	114
7.4	Part 2: Implementing an HTTP Client in Python	117
7.4.1	Task 3: Implementing an HTTP Client	118
	References	122
8	Lab 5—Prototyping Data Connectors	123
8.1	Prerequisites	123
8.2	Setup	124
8.3	Part 1: Implementing MQTT Client in Python	126
8.3.1	Task 1: Implementing an MQTT Client	126
8.4	Part 2: Data Connector—MQTT and AMQP	130
8.4.1	Task 2: Prototyping the Sensor Controller	131
8.5	Part 3: Data Connector—HTTP, AMQP and MQTT	135
8.5.1	Task 3: Prototyping the Actuator Controller	136
8.6	Part 4: Data Connector—AMQP and HTTP	140
8.6.1	Task 4: Deploying and Testing Logstash	140
	References	144
9	Lab 6—Integrating Sensors and Actuators with Edge and Cloud Tools	147
9.1	Prerequisites	147
9.2	Setup	148
9.3	Part 1: Sending Data from Sensors to Edge and Cloud Tools	149
9.3.1	Task 1: Implementing and Testing the Sensor Controller	150
9.4	Part 2: Triggering Actuators from Cloud and Edge Tools	156
9.4.1	Task 2: Implementing and Testing the Actuator Controller	157
	References	164

10	Lab 7—Network-of-Things Final Setup	165
10.1	Prerequisites	165
10.2	Setup	166
10.3	Part 1: Integrating Different Sensors and Actuators	168
10.3.1	Task 1: Integrating all Sensors	168
10.3.2	Task 2: Integrating all Actuators	172
10.4	Part 2: Final Setup—Integrating IoT, Edge to Cloud	178
10.4.1	Task 3: Testing and Benchmarking	179
	References	186
	Appendix A: Troubleshooting: Arduino Uno	189
	Appendix B: Troubleshooting: Raspberry Pi	191
	Appendix C: Troubleshooting: Server	195

About the Authors

Admela Jukan is Chair Professor in Communication Networks at TU Braunschweig. Prior to that, she worked in academia and industry, in the USA and Canada. She is Fellow of the IEEE and recipient of multiple industrial and research community awards. Admela received her Dr.tech. degree (cum laude) in Electrical and Computer Engineering from the Technische Universität Wien, M.Sc. degree in Information Technologies from the Politecnico di Milano, Italy, and her Dipl. -Ing. degree from the Fakultet Elektrotehnike i Racunarstva (FER), in Zagreb, Croatia.

Xavi Masip-Bruin is Full Professor at the Computer Architecture Department of the UPC and Director of the Center for Advanced Network Architectures (CRAAX). His research activity has driven several research contracts with public and industrial sectors, two start-ups exploiting technology transfer, several scientific contributions, and has been awarded with an IBM Faculty Award in 2016 for his contributions in the fog/edge/cloud computing field. Xavi Masip received his Telecommunication Engineering degree (B.Sc., M.Sc., and Ph.D.) from UPC in Barcelona, Spain.

Jasenka Dizdarević obtained her B.Sc. and M.Sc. engineering degrees in Telecommunications Engineering from Faculty of Electrical Engineering in Sarajevo, Bosnia, and Herzegovina. She is currently pursuing her Ph.D. (Dr.-Ing.) degree at Technische Universität Braunschweig, Germany. Her main research focus is in the area of application layer protocols communication protocols, its performance analysis, and experimentations. Jasenka is the main hardware and protocol architect in NoTE Lab.

Francisco Carpio received his M.Sc. and B.Sc. degrees in Telecommunications Engineering from Universitat Politècnica de Catalunya (UPC), Spain. He is currently pursuing his Ph.D. (Dr.-Ing.) degree at Technische Universität Braunschweig. He is Active Developer Contributor to the open source community and works in the area of optimizations and architecture design of IoT, edge, and cloud computing systems. He is the main software architect in NoTE Lab.

List of Figures

Fig. 1.1	NoTE Lab hardware configuration	2
Fig. 1.2	NoTE Lab communication protocols	3
Fig. 1.3	NoTE Lab structure and the individual lab modules	5
Fig. 2.1	DHT11 temperature/humidity sensor, icon, and PIN layout ...	11
Fig. 2.2	HCSR04 ultrasonic sensor, icon, and PIN layout	11
Fig. 2.3	Passive infrared sensor, icon, and PIN layout	12
Fig. 2.4	LED, icon, and PIN layout	13
Fig. 2.5	Liquid crystal display, icon, and PIN layout	13
Fig. 2.6	Auxiliary components and icons	14
Fig. 2.7	Arduino Uno WiFi rev2	15
Fig. 2.8	Raspberry Pi 4	15
Fig. 2.9	Server	16
Fig. 2.10	Workstation	16
Fig. 2.11	WiFi router	17
Fig. 3.1	Lab 0 setup	24
Fig. 3.2	Device connections	25
Fig. 3.3	Example of Router GUI	29
Fig. 3.4	Router GUI—Local Area Network (LAN) setup	29
Fig. 3.5	Router GUI—attached device	30
Fig. 3.6	Router GUI—wireless network	30
Fig. 3.7	Router GUI—attached devices	31
Fig. 3.8	Router GUI—assigning a static IP address	31
Fig. 3.9	Arduino IDE	34
Fig. 3.10	Arduino IDE <i>Verify</i> and <i>Upload</i>	35
Fig. 3.11	Arduino IDE <i>Verify</i> and <i>Upload</i>	35
Fig. 3.12	Install library in Arduino IDE	37
Fig. 4.1	Lab 1 setup	52
Fig. 4.2	Connecting an external LED to Arduino Uno WiFi Rev2	53
Fig. 4.3	Connecting LCD display to Arduino Uno WiFi Rev2	55
Fig. 4.4	Connecting DHT11 sensor to the Arduino	58
Fig. 4.5	Connecting HC-SR04 sensor to Arduino	62

Fig. 4.6	Connecting HC-SR501 sensor to Arduino	67
Fig. 5.1	Lab 2 setup	74
Fig. 5.2	Lab 2 components	75
Fig. 5.3	Lab 2, part 1 workflow	76
Fig. 5.4	Lab 2, part 2 workflow	78
Fig. 5.5	Lab 2, part 3 workflow	86
Fig. 5.6	Connecting DHT11 sensor to the Arduino	86
Fig. 5.7	Connecting external LEDs and LCD display to Arduino Uno WiFi Rev2	91
Fig. 6.1	Lab 3 setup	98
Fig. 6.2	Lab 3 components	98
Fig. 6.3	Lab 3, part 1 workflow	99
Fig. 6.4	RabbitMQ dashboard	102
Fig. 6.5	Lab 3, part 2 workflow	104
Fig. 6.6	RabbitMQ dashboard: Add binding to the <code>notelab</code> <code>queue</code>	108
Fig. 6.7	RabbitMQ dashboard: Message from the AMQP client	108
Fig. 7.1	Lab 4 setup	112
Fig. 7.2	Lab 4 components	112
Fig. 7.3	Lab 4, part 1 workflow	114
Fig. 7.4	Lab 4, part 2 workflow	118
Fig. 8.1	Lab 5 setup	124
Fig. 8.2	Lab 5 components	125
Fig. 8.3	Lab 5, part 1 workflow	126
Fig. 8.4	Lab 5, part 2 workflow	131
Fig. 8.5	RabbitMQ: <code>control_sensor_queue</code>	133
Fig. 8.6	Lab 5, part 3 workflow	136
Fig. 8.7	Lab 5, part 4 workflow	140
Fig. 8.8	Logstash subscribed to RabbitMQ	143
Fig. 9.1	Lab 6 setup	148
Fig. 9.2	Lab 6 components	149
Fig. 9.3	Lab 6, part 1 components	150
Fig. 9.4	RabbitMQ queues corresponding to the sensors	154
Fig. 9.5	Lab 6, part 2 components	157
Fig. 10.1	Lab 7 setup	166
Fig. 10.2	Lab 7 components	167
Fig. 10.3	Lab 7, part 1 components	169
Fig. 10.4	Interfacing and wiring all sensors to Arduino	169
Fig. 10.5	Connecting all actuators to Arduino	173
Fig. 10.6	Lab 7, part 2 components	178
Fig. 10.7	RabbitMQ queue corresponding to the Logstash configuration	180
Fig. 10.8	Motion data arriving at RabbitMQ	182
Fig. 10.9	Distance data arriving at RabbitMQ	183
Fig. 10.10	Temperature data arriving at RabbitMQ	183

List of Tables

Table 2.1	Hardware requirements and choice of devices	10
Table 2.2	Software prerequisites	17
Table 4.1	Mapping between the LED and Arduino	54
Table 4.2	Mapping between LCD display and Arduino	56
Table 4.3	Interfacing DHT11 with Arduino Uno WiFi Rev2	58
Table 4.4	Interfacing HC-SR04 with Arduino Uno WiFi Rev2	62
Table 4.5	Interfacing HC-SR501 sensor with Arduino Uno WiFi Rev2 ...	68

Chapter 1

Introduction



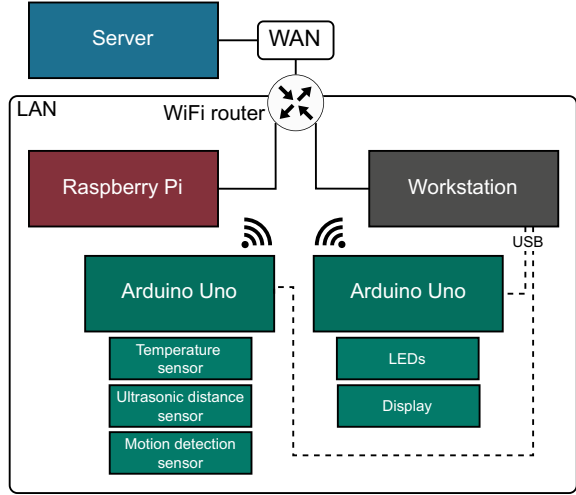
This section introduces NoTE Lab in more technical detail and also dedicates space to provide basics on communication protocols used. We also present each individual lab in the context of the overall NoTE Lab structure and discuss their learning objectives.

1.1 Hardware Setup

We start by describing the hardware configuration (Fig. 1.1) and their corresponding implementations, i.e.,

- *IoT devices* are implemented with two Arduino Uno WiFi which interface to sensors and actuators. One Arduino connects to three sensors: temperature/humidity sensor, motion detector, and ultrasonic distance sensor. Another Arduino interfaces two actuators: LEDs and LCD display. It should be noted that any Arduino compatible board is also suitable for the realization of the labs as long as it has the proper interfaces including WiFi capabilities.
- *Edge constrained devices* are implemented using a Raspberry Pi 4 (model B, 4 GB RAM). Other variants of the same board with larger or smaller RAM are also suitable. Older models, such as Raspberry Pi 3 model B, were tested and could be used as long as they run a 64-bit version of the operating system. Boards from other manufacturers have not been tested, but we expect that any models with ARM64-based chips are suitable.
- *Cloud/Edge Server* is implemented as a dedicated server machine located in the wide area network, albeit with a static public IP address. Alternatively, a virtual machine allocated in any public or private cloud service can also be used for this purpose.

Fig. 1.1 NoTE Lab hardware configuration



- *Workstation* is implemented as a dedicated desktop machine in the local area network to help students run programming and debugging tasks in various labs.
- *WiFi Router* used to connect all the devices in the local area network.

1.2 Communication Protocols

Communication protocols can be implemented in every layer of the Open Systems Interconnection (OSI) protocol stack. In NoTE Lab, we are interested in communication protocols in OSI Layer 7, also called the application layer. In the application layer, it is up to the system engineers to determine the most appropriate communication protocol solution, for instance, depending on application, hardware capabilities, amount of data to be processed, etc. Today, system engineers can choose from many different protocols, including Message Queue Telemetry Transport (MQTT), Hypertext Transfer Protocol (HTTP), Constrained Application Protocol (CoAP), Advanced Message Queuing Protocol (AMQP), Extensible Messaging and Presence Protocol (XMPP), and Data Distribution Service (DDS) protocol.

Figure 1.2 illustrates the application layer protocols chosen in NoTE Lab. The lightweight MQTT protocol is the most suitable solution for IoT devices due to its low communication overhead. Since MQTT follows a publish/subscribe model, MQTT broker is typically deployed in edge devices and works as a hub that collects sensor data from MQTT publishers and subscribers on IoT devices.

NoTE Lab implements AMQP protocol to send data from edge constrained devices to the cloud/edge server. To this end, MQTT broker connects to AMQP broker over a module we refer to as *sensor controller*, which not only connects var-

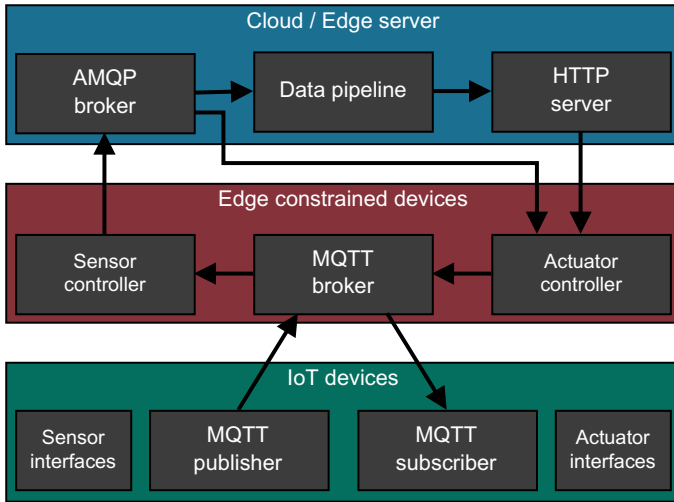


Fig. 1.2 NoTE Lab communication protocols

ious parts of the system but also enables sensor data processing. In the opposite direction, we choose HTTP protocol when retrieving data from the cloud for either data processing or actuation (see module *actuator controller*). Also, AMQP protocol can be used to communicate with the actuator controller. We choose to deploy both protocols, i.e., AMQP and HTTP, so students can learn both the publish/subscribe and client/server concepts and distinguish between the two. In the cloud, another connector needs to be deployed to collect data from the AMQP broker and forward it to the HTTP server.

The main characteristics of the application layer protocols used in NoTE Lab can be summarized as follows:

- Message Queue Telemetry Transport (MQTT) is an application layer protocol based on a publish/subscribe model. This model uses a broker where clients can publish or subscribe to the so-called *topics*. When a publisher sends a certain message to a certain topic, all subscribers to that topic receive the same message. In NoTE Lab, MQTT brokers run on edge constrained devices, i.e., Raspberry Pis, and MQTT publishers and subscribers run on either IoT devices or server and workstation. By configuring different topics, we can decide about the type of data to be collected and processed.
- Hypertext Transfer Protocol (HTTP) is the most widely used application layer protocol in the Internet. This protocol, which uses a client/server model, consists of a server responding to requests from a client based on the resource or information that the client requests. These requests can be of type POST, GET, PUT, or DELETE depending on the type of operation requested. In NoTE Lab, this protocol is used to interact with a database system, as it is typically the case in

cloud computing. This system offers a Representational State Transfer (REST) Application Programming Interface (API) to access data.

- **Advanced Message Queuing Protocol (AMQP)** is a protocol based on publish/subscribe model, with a broker entity ensuring that all messages published by AMQP clients (acting as publishers) are forwarded to other AMQP clients (acting as consumers). The AMQP broker consists of two main entities, *exchanges* and *queues*. *Exchanges* are used when publishers send messages, after which these messages are routed to queues, where they are stored until they are received by consumers. In NoTE Lab, AMQP broker is deployed in the cloud server and is used to receive data from edge constrained devices.

1.3 On Software Tools and Libraries

Generally, we choose software tools and external libraries based on the communications protocols implemented (Fig. 1.2) and the device categories (Fig. 1.1), i.e.,

- *IoT devices*: Arduinos are resource constrained devices, which requires us to choose programming languages with low memory footprint. In NoTE Lab, Arduino programming language is used when programming on these type of boards. The main library used for the communication part is the *arduino-mqtt* library, which allows the boards to use MQTT protocol.
- *Edge constrained devices*: Raspberry Pis are used to deploy a Mosquitto as MQTT broker. The connectors running on the Raspberry Pis to connect to the server are implemented in Python, using popular libraries, such as *Paho* for MQTT, *Pika* for AMQP, and *requests* for HTTP.
- *Cloud/edge server*: In the server, Kubernetes is used as a container orchestrator. Using containers, the server deploys a different set of popular cloud computing tools, including RabbitMQ as AMQP broker, and Elasticsearch which implements an HTTP server as a database system. Logstash tool is used as a pipeline connector between RabbitMQ and Elasticsearch.

1.4 Lab Structure and Learning Objectives

NoTE Lab course starts off with the overview of the entire IoT-edge-cloud computing system and is followed by the individual lab modules designed as individual puzzle pieces of a full system picture. Each individual lab module is focused on one cohesive part of the system (e.g., device, protocol, or software tool), as previously presented in Fig. 1.2. Except for *Lab 0*, which is the only lab module not shown here, since it introduces students to the basic concepts in hardware and software, each lab module is structured to fit into the full picture, as illustrated in Fig. 1.3.

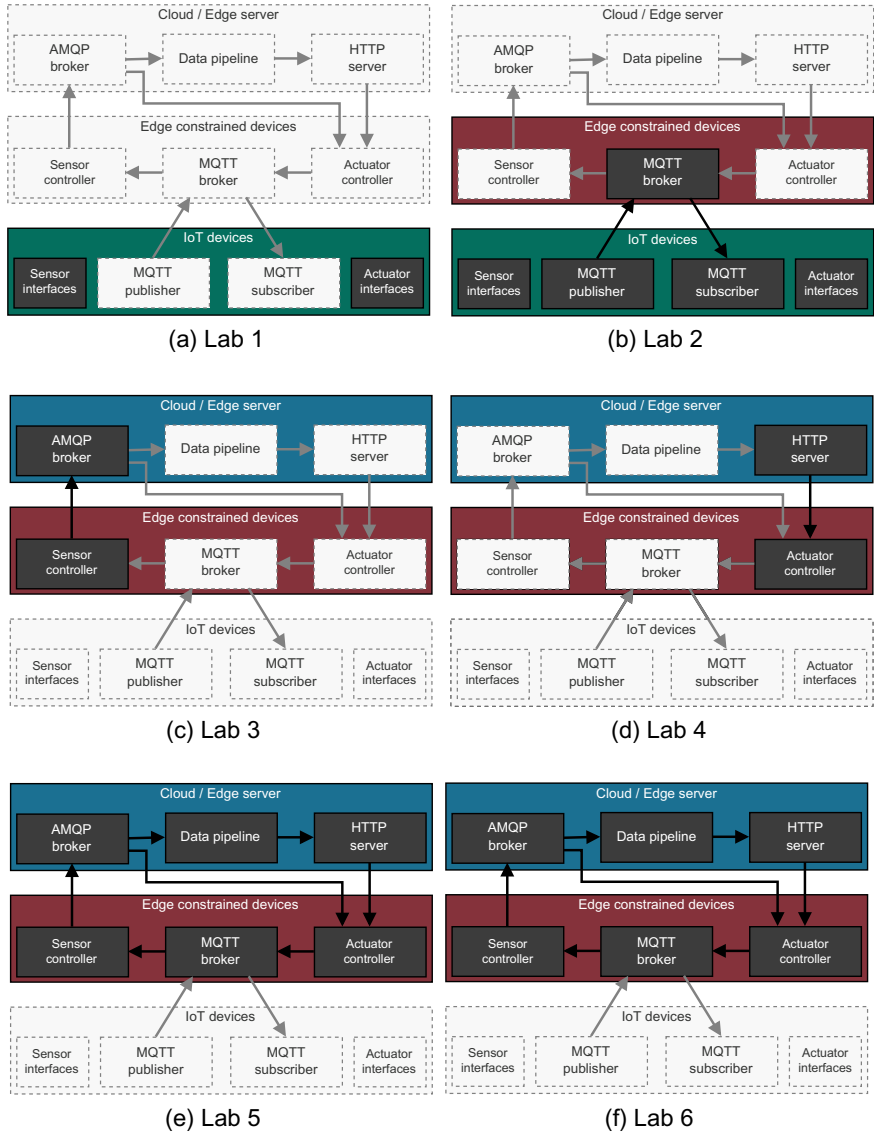
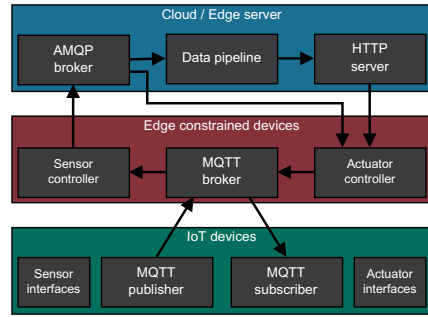


Fig. 1.3 NoTE Lab structure and the individual lab modules

Fig. 1.3 (continued)

(g) Lab 7

Lab 0—Getting started: This lab module helps students get familiar with the full system, with a focus on generic concepts useful in most lab modules. It includes the following parts:

- Part 1: On the Workstation
- Part 2: Introduction to Arduino programming
- Part 3: Getting started with Raspberry Pi
- Part 4: On the server

Lab 1—Interfacing sensors and actuators: This lab (see Fig. 1.3a) focuses on Arduinos. Students get familiar with Arduino IDE, as well as learn how to program different functionalities to different types of sensors. This lab includes the following parts:

- Part 1: Actuating an external LED
- Part 2: Actuating an LCD display
- Part 3: Temperature/humidity sensor
- Part 4: Ultrasonic distance sensor
- Part 5: Motion sensor

Lab 2—Connecting IoT and Edge with MQTT: This lab (see Fig. 1.3b) focuses on implementing MQTT clients on Arduinos and MQTT broker on Raspberry Pi and to establish communication between them. This lab includes the following parts:

- Part 1: Deploying and testing an MQTT broker
- Part 2: Implementing an MQTT client in Arduino
- Part 3: Sensing and actuating over MQTT

Lab 3—Connecting edge and cloud tools with AMQP: In this lab (see Fig. 1.3c), students get familiar with the implementation and testing of AMQP broker on the server and AMQP client on Raspberry Pi, as well as test their AMQP-based communication. It includes the following parts:

- Part 1: Deploy and test RabbitMQ
- Part 2: Implementing an AMQP client in Python

Lab 4—Connecting edge and cloud tools with REST HTTP: This lab (see Fig. 1.3d) focuses on configuring a database engine system on the server, with Elasticsearch tool, which offers a REST API and can be used as REST HTTP server. The HTTP communication is then tested between HTTP client on Raspberry Pi and HTTP server. This lab includes the following parts:

- Part 1: Deploy and test Elasticsearch
- Part 2: Implementing an HTTP client in Python

Lab 5—Prototyping data connectors: This lab (see Fig. 1.3e) focuses on multi-protocol connections with MQTT, AMQP, and HTTP protocol solutions. This includes establishing data connectors between MQTT clients running on Arduino and AMQP broker and database with REST API running on the server. This lab includes the following parts:

- Part 1: Implementing MQTT client in Python
- Part 2: Data connector between MQTT and AMQP brokers
- Part 3: Data connector between HTTP server and MQTT broker
- Part 4: Data connector between AMQP broker and HTTP server

Lab 6—Integrating sensors and actuators with edge and cloud tools: This lab (see Fig. 1.3f) focuses first on processing sensor data in the edge, before forwarding it up to the cloud server. In addition, it implements data processing and communication between the server and Arduinos toward the actuators. This lab includes the following parts:

- Part 1: Sending data from sensors to edge and cloud tools
- Part 2: Triggering actuators from cloud and edge tools

Lab 7—Network-of-Things final setup: This is the final lab (see Fig. 1.3g). It implements and tests the data processing and communication workflows which show how data can be processed in the edge, forwarded to the cloud, and in the same workflow sent back to actuators based on data processing. This lab includes the following parts:

- Part 1: Integrating different sensors and actuators
- Part 2: Final setup—integrating IoT, edge to cloud.

Chapter 2

NoteLab Setup Requirements



This chapter provides an overview of devices and software requirements in NoTE Lab. We first introduce all hardware devices and their related specifications. After that, we introduce all software tools and libraries that need to be installed in individual labs. All official datasheets and open-source codes of the hardware and software are listed at the end of this chapter.

2.1 Hardware Requirements

We created NoTE Lab conceptually such that no any specific devices or technologies are required and every of it component can be correspondingly replaced. In other words, every lab setup is open to different implementations. There are however some minimum requirements on devices used, as shown in Table 2.1, along with some off-the-shelf devices used in this book.

2.1.1 Sensors

We choose sensors not only based on how widely available they are, but also based on the quality and availability of open-access tutorials, open-source code, and the related developer's support. In NoTE Lab, we use temperature sensor, ultrasonic distance sensor, and passive infrared sensor.

Table 2.1 Hardware requirements and choice of devices

Device type	Min. requirements	NoTE Lab choice
<i>IoT devices</i>		
Sensors	Any Arduino compatible sensor	(1x) - DHT11 temperature humidity sensor (1x) - HC-SR04 Ultrasonic sensor (1x) - HC-SR501 PIR motion sensor
Actuators	Any Arduino compatible actuator	(3x) - 5 mm LEDs (of different colors) (1x) - 16x2 LCD module
Auxiliary components	Protoboard, wiring, resistors	(3x) - resistors (330 ohm) (1x) - resistor (10000 ohm) (2x) - breadboards (10x) - wire jumpers (min. 15 cm)
Single board microcontroller	Arduino-based board (5 V output and WiFi)	(2x) - Arduino Uno WiFi Rev2
<i>Edge constrained devices</i>		
Single board computer	ARM64-based device, 2 GB RAM, and 16 GB disk	(1x) - Raspberry Pi 4 Model B, 4 GB RAM, 16 GB SD storage
<i>Edge/cloud server</i>		
Server	4 vCPUs, 16 GB RAM, and 20 GB disk	(1x) - Desktop PC, 4 CPUs (Intel(R) Core(TM) i5-7600 CPU @ 3.50GHz), 32 GB RAM, 256 GB disk
<i>Auxiliary hardware</i>		
Workstation	2 vCPUs, 4 GB RAM, and 25 GB disk	(1x) - Desktop PC, 4 CPUs (Intel(R) Core(TM) i5-7600 CPU @ 3.50GHz), 16 GB RAM, 256 GB disk
Router	4 port WiFi router	(1x) - NetGear AC1200

Temperature Sensor

We choose DHT11 from the manufacturer Seeed Studio as the temperature sensor. This device uses a capacitive humidity sensor and a thermistor to measure the surrounding air, and as a result, it produces a digital signal on the data pin (i.e., no analog input pins needed). The DHT11 sensor includes a resistive humidity sensor, an NTC thermistor to measure the temperature of the humidity sensing substrate, and a microcontroller that combines the resistance measurements and factory calibration data into temperature and humidity measurements. It has four pins (shown in Fig. 2.1), and it operates from 3.5 to 5.5 V power supply, with Vcc and ground (GND) pin used to power the module. The sensor measures temperature between 0 and 50 °C ($\pm 2^{\circ}\text{C}$ accuracy) and humidity 20–80% (5% accuracy). The link to the datasheet used to gather this information can be found in [1].

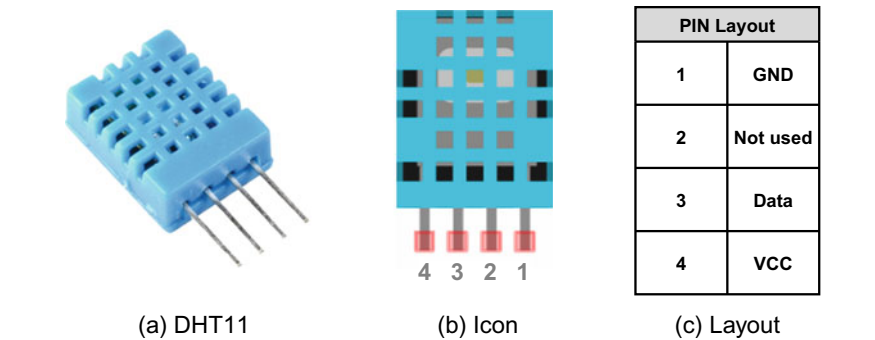


Fig. 2.1 DHT11 temperature/humidity sensor, icon, and PIN layout

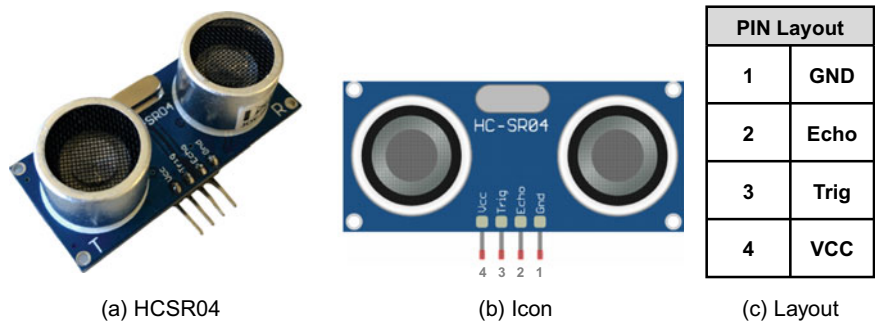


Fig. 2.2 HCSR04 ultrasonic sensor, icon, and PIN layout

Ultrasonic Distance Sensor

As a distance sensor, we choose HC-SR04 sensor from Joy-IT manufacturer. It measures distance in the range from 2 to 400 cm. This sensor includes two modules, ultrasonic transmitter and receiver. HC-SR04 uses four pins in total (shown in Fig. 2.2), two pins for communication, a trigger pin, and an echo pin. It operates at 5 V with 15 mA operating current, and with Vcc and ground (GND) pins used to power the device. With the power supply of the sensor, the trigger pin can be controlled and the measurement can be started, thus generating an ultrasonic pulse at a sound speed that travels until it hits an object. The receiver module then waits for the echo to be detected. After receiving an echo, the time between the trigger and the return of the echo is calculated using an Arduino, which converts it into a distance measurement. The link to the datasheet used to gather this information can be found in [2].

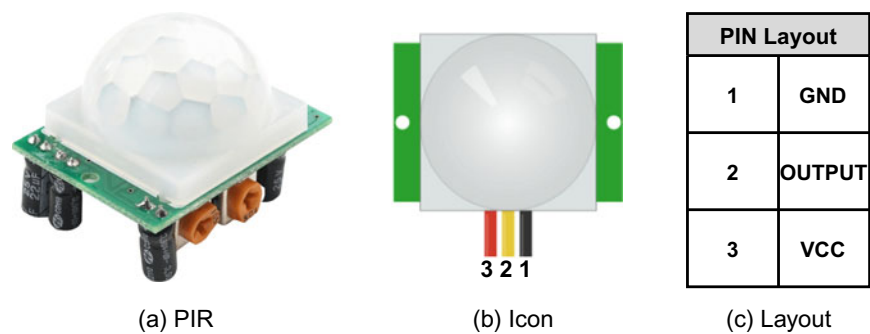


Fig. 2.3 Passive infrared sensor, icon, and PIN layout

Passive Infrared Sensor

As a motion sensor, we choose HC-SR501 PIR sensor from Iduino manufacturer. It detects motion through changes in the emission of infrared light. This sensor consists of two main components, a pyroelectric sensing component, which detects infrared radiation, and a fresnel lense, which focuses the radiation onto the sensor. The objects or humans are detected when they are in the sensing range, with the increasing intensity of the incoming radiation compared to the background. HC-SR501 operates typically at 5V, with Vcc and ground (GND) pins used to power the module (as shown in Fig. 2.3). In addition to Vcc and GND pins, it has a third pin, i.e., the output pin, which, depending on whether the sensor detects a moving object, outputs a low or high level signal. The link to the datasheet used to gather this information can be found in [3].

2.1.2 Actuators and Auxiliary Components

Light Emitting Diodes (LEDs)

We use standard 5mm LEDs of different colors as actuators. These LEDs have two different leads, a longer positive (anode) and shorter negative (cathode) lead, accordingly connected to the power supply (shown in Fig. 2.4). It is required to use a resistor to limit the current along with an LED. It is also important to get the polarization of the LED correct; the resistor, on the other hand, can be connected anyway around. The color of the LED determines the relevant resistor values, including power supply, LED voltage, and maximum current. Since we use Arduino pins as a power supply, the supply voltage is low (3.3–5 V). The links to the datasheet used to gather this information can be found in [4].

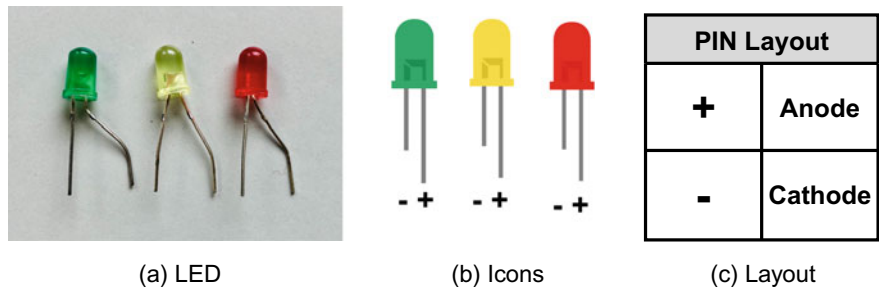


Fig. 2.4 LED, icon, and PIN layout

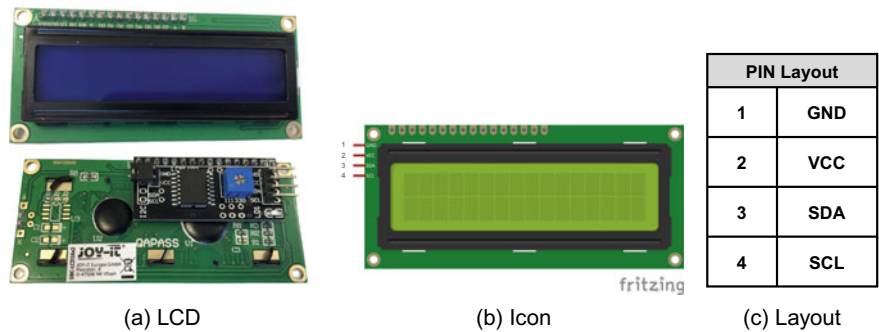


Fig. 2.5 Liquid crystal display, icon, and PIN layout

Liquid Crystal Display

We use a 16x2 LCD display (I2C version) from Joy-IT manufacturer as an actuator. These LCDs use the liquid crystals, whereby, by changing the orientation of liquid crystals, light is allowed to pass through, or it is blocked and they consist of two rows of 16 characters to display. To connect these types of LCD displays to Arduino boards, an I2C connector module is used, which, in addition to Vcc and GND pins, consists of two signal pins, SCL clock signal and SDA data signal. The chosen LCD operates typically at 5 V, with Vcc and ground (GND) pins used to power the module (shown in Fig. 2.5). The link to the datasheet used to gather this information can be found in [5].

2.1.3 Auxiliary Components

In order to interface sensors and actuators with Arduinos, a set of auxiliary components is required, including a breadboard, resistors, and a set of jumper wires. Breadboards, used as the backbone of most prototype circuits, are used to provide

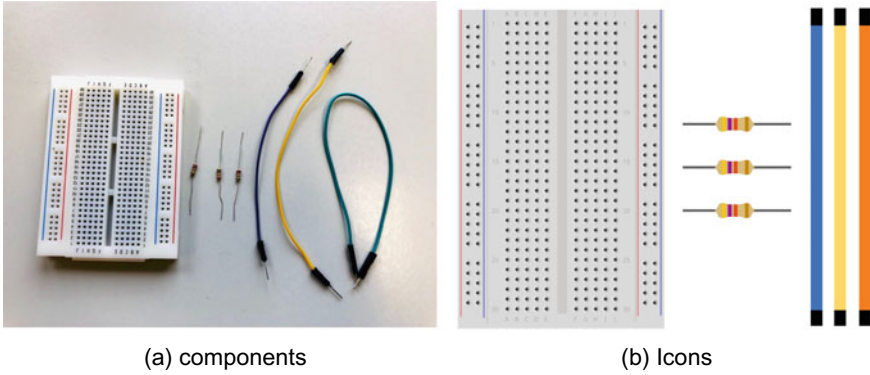


Fig. 2.6 Auxiliary components and icons

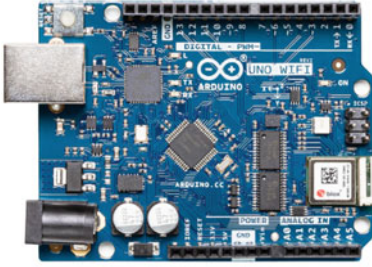
easy ways to connect and build temporary circuits. The breadboard has holes that are inter-connected, with the long lines/strips marked with + and -, denoting positive and negative power rails, respectively. The rows marked by numbers are connected internally. Also, as auxiliary components, we need some resistors to connect the LEDs and one of the sensors, as well as some jumper wires to connect all the components. The whole set of auxiliary components is shown in Fig. 2.6.

2.1.4 *Single Board Microcontrollers*

Single Board Microcontrollers can be best described as small, computer-like integrated circuit chips with programmable computation capabilities and many general-purpose input/output (GPIO) pins. In NoTE Lab, any boards compatible with the Arduino framework (with 5 V output) and equipped with a WiFi interface can be used. We opted for one of the official Arduino boards, which come with integrated WiFi interfaces. We specifically show Arduino Uno WiFi Rev2 board as an example that has been extensively tested (Fig. 2.7). The links to specifications (datasheet) can be found in [6]. It should be noted that other development boards may require different pin assignments, or operating voltage, which needs to be carefully considered. For each Arduino, a USB 2.0 Cable Type A is used for flashing and power purposes.

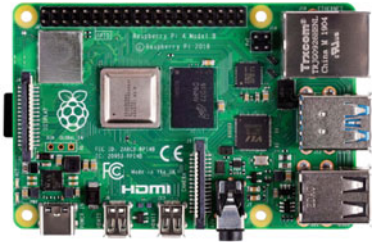
2.1.5 *Single Board Computers*

Single board computers (SBC) are actually full stack computers built on a single circuit board, hence the name. They also include external connections, such as USB, HDMI, RJ-45 connectors, audio, and external General-Purpose input and output



- Microcontroller: ATmega4809 (datasheet in [[Arduino micro. datasheet](#)])
- Clock speed: 16 MHz
- Radio module: u-blox NINA-W102 (datasheet in [[Arduino radio datasheet](#)])
- Firmware: Arduino NINA-W102 firmware
- Operating voltage: 5V.

Fig. 2.7 Arduino Uno WiFi rev2



- Board: Raspberry Pi 4 Model B
- Memory: 2, 4 or 8 GB of RAM
- Storage: 16 GB of free disk space
- Network: Ethernet interface (with Internet access)
- OS: 64 bits Linux distribution (recommended: Ubuntu Server LTS for ARM)

Fig. 2.8 Raspberry Pi 4

(GPIO) among others. These GPIO pins can be used to connect to external devices, such as sensors. While any SBC available for purchase can be used, in NoTE Lab we use Raspberry Pi 4 Model B as the baseline solution; see Fig. 2.8. Raspberry Pis typically get connected with a monitor, keyboard, and mouse, just like desktop computers. However, in NoTE Lab, we use a “headless” setup, so that students remotely interact with RPis using terminal commands, like SSH. For 4 GB Raspberry Pi 4 Model B, the link to the datasheet can be found in [9].

2.1.6 Server

In NoTE Lab, we use the server to run cloud computing tools in a “headless” setup and can be reached over the workstation via terminal commands (SSH). To this end, the server is expected to meet the following minimum requirements (See Fig. 2.9).



Fig. 2.9 Server

- CPU: 4 vCPUs
- Memory: 16 GB of RAM
- Storage: 20 GB of free disk space
- Network: Ethernet interface (with Internet access)
- OS: 64 bits Linux distribution (recommended: Ubuntu server LTS)



Fig. 2.10 Workstation

- CPU: 2 GHz dual-core processor
- Memory: 4 GB of RAM
- Storage: 25 GB of free disk space
- Video: VGA capable of 1024x768 screen resolution
- Network: Ethernet interface (with Internet access)
- OS: 64 bits Linux distribution (recommended: Ubuntu Desktop LTS)

2.1.7 Workstation

A workstation is used to access and configure Raspberry Pis as well as to provide additional support when developing and debugging scripts. The workstation with a connected monitor, mouse, and keyboard requires a traditional GUI-based desktop operating system to facilitate programming tasks. The minimum hardware requirements for the workstation are (See Fig. 2.10).

2.1.8 WiFi Router

In NoTE Lab, we use NetGear AC1200 WiFi router (model R6120). Generally, a WiFi router is expected to fulfill the following minimum requirements (See Fig. 2.11).

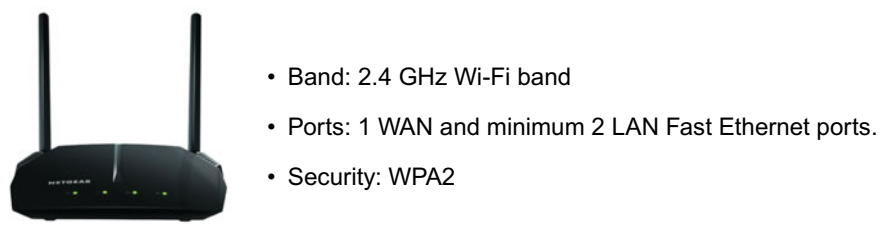


Fig. 2.11 WiFi router

2.2 Software Requirements

Table 2.2 shows the software prerequisites for each device used and the specific solution chosen in NoTE Lab.

2.2.1 Arduino Uno WiFi

Arduino Uno WiFi Rev2 comes with stock firmware Arduino NINA-W102 [10] and an integrated WiFi module. To update the firmware, follow the information in [11]. The following libraries are used when working with Arduino Uno:

Table 2.2 Software prerequisites		
Device type	Min. requirements	NoTE Lab solution
<i>IoT devices</i>		
Single board microcontroller	Arduino compatible firmware	Arduino NINA-W102 firmware
<i>Edge constrained device</i>		
Single board computer	Linux-based OS 64-bit	Ubuntu Server 20.04 LTS 64-bit ARM
<i>Edge/cloud server</i>		
Server	Linux-based OS 64-bit	Ubuntu Server 20.04 LTS 64-bits
<i>Others</i>		
Workstation	Linux OS 64-bit	Ubuntu Desktop 20.04 LTS 64-bits

Libraries

- Wi-FiNINA library: Enables network connection for Arduino UNO Wi-Fi Rev.2 [29].
- Liquid Crystal I2C library: Allows the control of I2C displays [31].
- DHT sensor library: Support for DHT series of low-cost temperature/humidity sensors [30].
- MQTT library: Support for MQTT 3.1.1 client [32].
- JSON library: Simple JSON library for embedded C++ [33].

2.2.2 *Raspberry Pis*

Raspberry Pi operating system usually comes preinstalled on the device. As previously mentioned, however, we recommend to set up a headless Raspberry Pi. Therefore, we need to install the long-term support (LTS) version of Ubuntu Server for ARM, which comes with a comparably fewer preinstalled packages and thus frees up system resources, which we can use for computing purposes. The official documentation for this installation can be found in [12]. The following tools and libraries are used when working with Raspberry Pi Uno:

Tools

- Docker engine: Containerization tool (installation instructions in [16]).
- Python interpreter, preinstalled in most Linux distros [17].
- `pip`: Tool for the installation of Python modules (since Python 3.4, already included in Python binary installers, otherwise refer to instructions in [15]).
- SSH server (preinstalled in most Linux distros, refer to installation instructions in [18] for Ubuntu to verify `ssh` is installed and the service up and running).
- `mosquitto-clients`: Mosquitto command line MQTT clients (installation using `apt` package manager, the link to the source is provided in [23]).
- `amqp-tools`: Command line utilities for interacting with AMQP servers (installation using `apt` package manager, see the links to the source in [24]).

Libraries

- `paho-mqtt`: MQTT library for Python (installation using `pip` tool, details in [35]).
- `pika`: AMQP library for Python (installation using `pip` tool, details in [34]).
- `requests`: HTTP library for Python (installation using `pip` tool, details are in [36]).

2.2.3 *Server*

We choose Ubuntu Server long-term support (LTS) version due to its stability (see the links to the official documentation in [13]). The following tools and libraries are used when working with the server:

Tools

- `curl`: Command line tool and library for transferring data with URLs (installation using `apt` package manager, see the link to the source in [20]).
- `helm`: A package manager for Kubernetes (refer to installation instructions from the links to scripts [21]).
- SSH server (preinstalled in most Linux distros, refer to installation instructions in [18] for Ubuntu to verify `ssh` is installed and the service up and running).
- `k3s`: A certified Kubernetes distribution built for IoT and Edge computing (refer to installation instructions in links listed [25]).
- RabbitMQ: A message broker with support for AMQP protocol [26].
- Elasticsearch: A distributed search engine based on schema-free JSON documents [27].
- Logstash: A server-side data processing pipeline [28].

2.2.4 *Workstation*

Also here, we recommend Ubuntu Desktop long-term support (LTS) version as the operating system for the workstation (see the links for installation in [14]). Since this machine is intended to be used for debugging, testing, and system management purposes, the tools and libraries required are the same as in Raspberry Pi, but adding the following specific ones for remotely accessing the Raspberry Pi and server, as well as for programming for the Arduinos:

Tools

- `curl`: Command line tool and library for transferring data with URLs (installation using `apt` package manager, see the link to the source in [20]).
- SSH client (preinstalled in most Linux distros, refer to installation instructions for Ubuntu [18]).
- Arduino IDE: Arduino integrated development environment (refer to installation instructions from [22]).

References

Datasheets

1. Temperature sensor DHT11 datasheet: <https://asset.conrad.com/media10/add/160267/c1/-/en/001405544DS01/datenblatt-1405544-seeed-studio-dht11-ntc-temperatursensor-0-bis-50-c-radial-bedrahtet.pdf>
2. Ultrasonic distance sensor datasheet: [https://joy-it.net/files/files/Produkte/SEN-US01/SEN-US01%20Datasheet%20\(English\).pdf](https://joy-it.net/files/files/Produkte/SEN-US01/SEN-US01%20Datasheet%20(English).pdf)
3. Motion detector PIR sensor datasheet: <http://www.openplatform.cc/index.php/home/index/details/apiid/38>
4. LEDs datasheet: https://asset.re-in.de/add/160267/c1/-/en/001661532DS01/DA_Barthelme-LED-Sortiment-Kaltweiss-Warmweiss-Gelb-Blau-Rot-Gruen-Rund-5mm.pdf
5. 16×2 LCD datasheet: https://joy-it.net/files/files/Produkte/SBC-LCD16x2/SBC-LCD16x2_Manual_2021-04-07.pdf
6. Arduino Uno WiFi rev2 datasheet: https://content.arduino.cc/assets/Pinout-UNOwifirev2_latest.pdf
7. Arduino Uno WiFi rev2 (microcontroller) datasheet: <https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega4808-4809-Data-Sheet-DS40002173A.pdf>
8. Arduino Uno WiFi rev2 (radio module) datasheet: https://content.arduino.cc/assets/Arduino_NINA-W10_DataSheet_%28UBX-17065507%29.pdf
9. Raspberry Pi 4 model B datasheet: <https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-product-brief.pdf>

Firmware and Operating Systems

10. Arduino NINA-W102 firmware: <https://github.com/arduino/nina-fw>
11. Arduino firmware updater: <https://docs.arduino.cc/tutorials/generic/firmware-updater>
12. How to install Ubuntu server on a Raspberry Pi: <https://ubuntu.com/tutorials/how-to-install-ubuntu-on-your-raspberry-pi>
13. How to install Ubuntu Server: <https://ubuntu.com/tutorials/install-ubuntu-server>
14. How to install Ubuntu desktop: <https://ubuntu.com/tutorials/install-ubuntu-desktop>

Tools

15. How to install python packages: <https://packaging.python.org/en/latest/tutorials/installing-packages/>
16. Docker Engine installation on Ubuntu: <https://docs.docker.com/engine/install/ubuntu/>
17. Python: <https://www.python.org/downloads/>
18. OpenSSH Server: <https://ubuntu.com/server/docs/service-openssh>
19. Git for linux: <https://git-scm.com/download/linux>
20. CURL: <https://curl.se/download.html>
21. Helm: <https://helm.sh/docs/intro/install/#from-script>
22. How to install Arduino IDE: <https://www.arduino.cc/en/Guide/Linux>
23. mosquitto-clients (v1.6.9-1): <https://packages.ubuntu.com/focal/mosquitto-clients>
24. Package: amqp-tools (0.10.0-1): <https://packages.ubuntu.com/focal/amqp-tools>

- 25. K3s container orchestrator: <https://rancher.com/docs/k3s/latest/en/installation/install-options/>
- 26. RabbitMQ: <https://www.rabbitmq.com/>
- 27. Elasticsearch: <https://www.elastic.co/elasticsearch/>
- 28. Logstash: <https://www.elastic.co/logstash/>

Libraries

- 29. Arduino Wi-FiNINA library source: <https://github.com/arduino-libraries/WiFiNINA>
- 30. Source code DHT11 library: <https://github.com/adafruit/DHT-sensor-library>
- 31. Source code LiquidCrystal_I2C library https://github.com/johnrickman/LiquidCrystal_I2C
- 32. Arduino MQTT library: <https://github.com/256dpi/arduino-mqtt>
- 33. Arduino JSON library: <https://github.com/bblanchon/ArduinoJson>
- 34. Pika library: <https://pika.readthedocs.io/en/stable/>
- 35. Paho Python Client: <https://www.eclipse.org/paho/index.php?page=clients/python/index.php>
- 36. Requests library: <https://requests.readthedocs.io/en/latest/user/install/>

Chapter 3

Lab 0—Getting Started



3.1 Prerequisites

Hardware

- (1x) - Arduino Uno WiFi Rev2
- Raspberry Pi 4 Model B
- Server
- Workstation
- Router

Software

- Raspberry Pi 4 Model B
 - Docker v20.10.8 [2]
 - Python v3.9 [3]
 - OpenSSH v8.4 [4]
- Server
 - curl v7.81.0 [5]
 - OpenSSH v8.4 [4]
- Workstation
 - curl v7.81.0 [5]
 - Arduino IDE v1.8.13 [6]
 - OpenSSH v8.4 [4]
 - Python v3.9 [3]

3.2 Setup

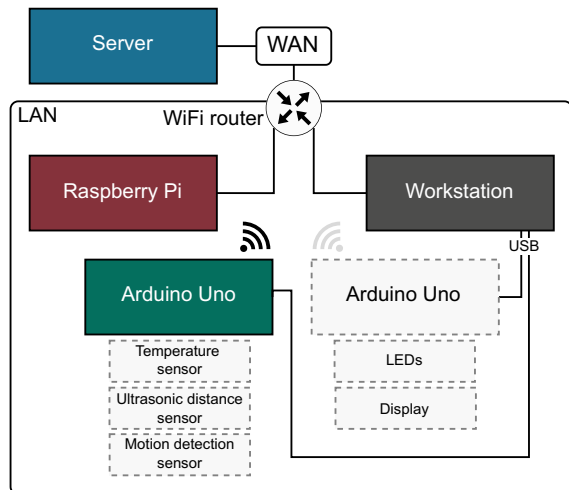
This is an introductory lab for students to get familiar with a variety of hardware in NoTE Lab. In addition, we provide a refresher tutorial on basic Linux CLI commands, as well as network and device configurations. Figure 3.1 shows the hardware connections used in this lab. The lab is divided into four main parts. We start off with Linux CLI on workstation. We then show how to configure the wireless router and run SSH commands. We continue with the basics of programming in Arduino and Python. We finish with the basic instructions on how to install a lightweight distribution of Kubernetes, a container orchestrator tool used in cloud computing. For more advanced students, this introductory lab already includes the assignments. This lab uses the following library and provides instructions on how to install it:

- Arduino Uno WiFi
 - Wi-FiNINA library v1.8.13 [1]

Before starting with the Lab, make sure all the components are connected as follows:

The workstation and Raspberry Pi devices (the latter powered through an USB Type-C charging port) are connected to the assigned router in a wired fashion, via Ethernet cables. Through the workstation, students are going to change the default settings of the router and set up a local area network, to which the workstation and Raspberry Pi are going to connect to. Next, Arduinos are connected to the workstation using USB Type A/B cables, which are going to be used as both their power supply and for flashing the code. Finally, Arduino boards are configured so they can also connect to the same local area network using the WiFi interface (see Fig. 3.2).

Fig. 3.1 Lab 0 setup



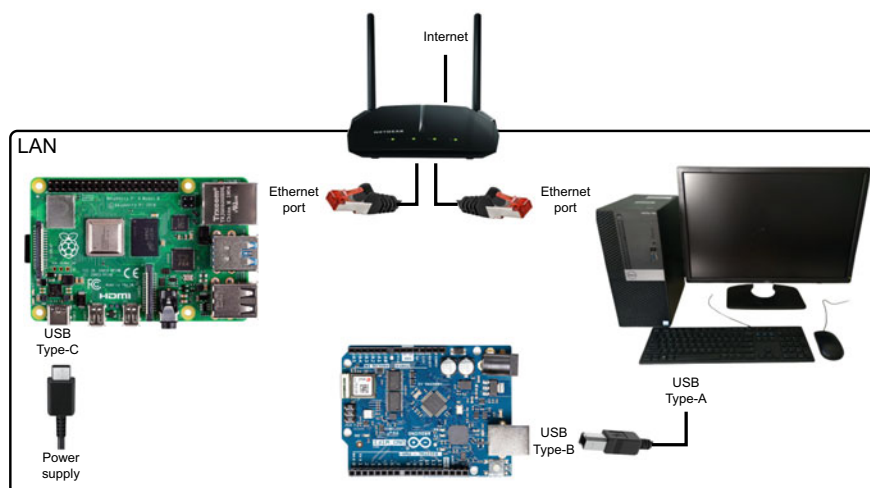


Fig. 3.2 Device connections

3.3 Part 1: On the Workstation

We start off with three tasks: getting familiar with Linux CLI commands, configuring a wireless local area network, and learning how to remotely access machines with SSH commands.

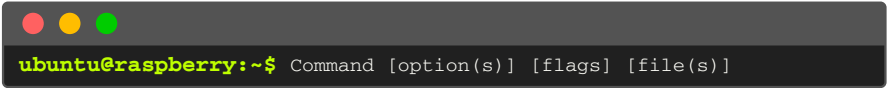
3.3.1 Task 1: Getting Started with Linux Command Line Interface (CLI)

Since the workstation, the Raspberry Pi, and the server device, all three use Linux-based operating systems, throughout different tasks it is going to be required to use CLI commands from their corresponding prompts.

When executing commands from the workstation, the following prompt is used:

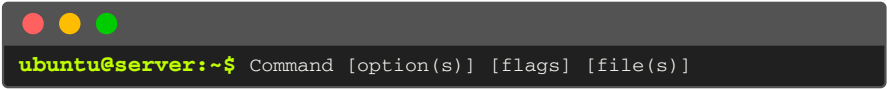
```
student@workstation:~$ Command [option(s)] [flags] [file(s)]
```

When executing commands from the Raspberry Pi (through a remote connection), the following prompt is used:



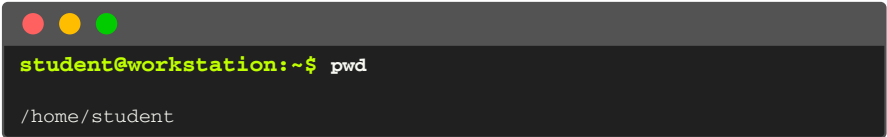
```
ubuntu@raspberrypi:~$ Command [option(s)] [flags] [file(s)]
```

And finally, when executing commands from the server (through remote connection), the following prompt is used:



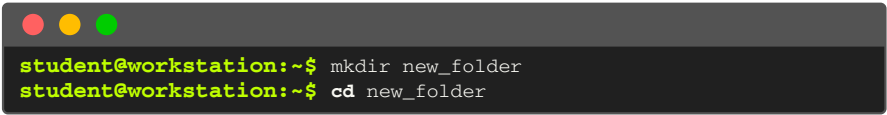
```
ubuntu@server:~$ Command [option(s)] [flags] [file(s)]
```

Now, create a terminal on the workstation and type `pwd` to see the current working directory:



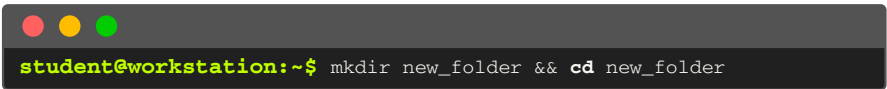
```
student@workstation:~$ pwd
/home/student
```

Create a new directory by typing `mkdir` and get into the directory by typing `cd`:



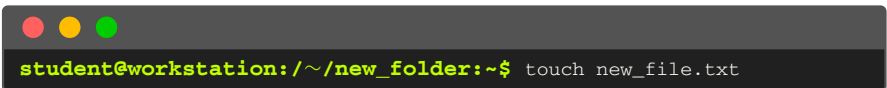
```
student@workstation:~$ mkdir new_folder
student@workstation:~$ cd new_folder
```

The same can be achieved by a single command making use of an AND operator:



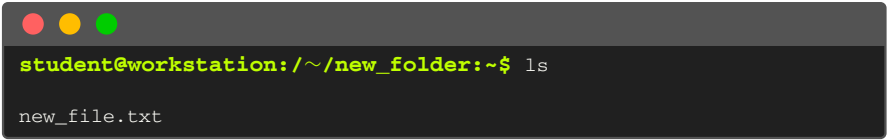
```
student@workstation:~$ mkdir new_folder && cd new_folder
```

Now, create a new file with `touch` command:



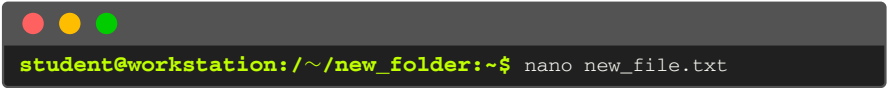
```
student@workstation:~/new_folder:~$ touch new_file.txt
```

Use `ls` command to list files within the current directory:



```
student@workstation:~/new_folder:~$ ls
new_file.txt
```

Many tools are available to edit files from CLI. In NoTE Lab, we choose `nano`, a common and easy-to-use text editor which is sufficient for our purposes. Open the newly created file with



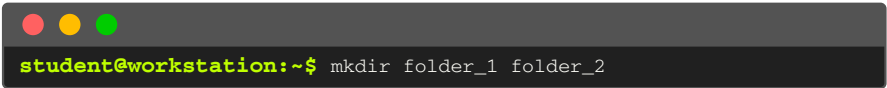
```
student@workstation:~/new_folder:~$ nano new_file.txt
```

After the file is edited, `ctrl + x` asks for saving the file and exits afterward. Use `cd ..` to move up to the parent directory (use just `cd` to move to the home directory). Check if you are in the parent directory:



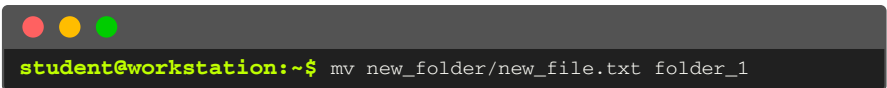
```
student@workstation:~$ pwd
/home/student
```

Create two additional directories:



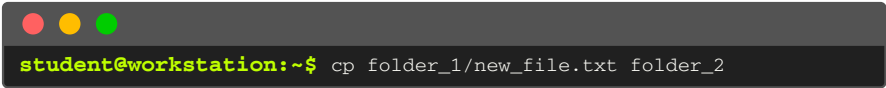
```
student@workstation:~$ mkdir folder_1 folder_2
```

With command `mv`, move the previously created file (`new_file.txt`) to one of the new directories:



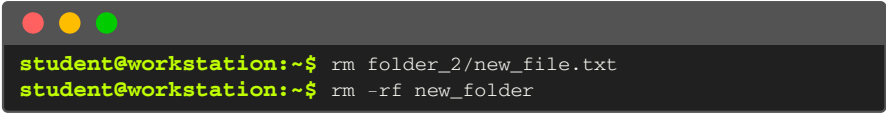
```
student@workstation:~$ mv new_folder/new_file.txt folder_1
```

Check with `ls` command if the file was moved from the old to the new directory. Now, copy the same file to the other directory with `cp` command:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The prompt is `student@workstation:~$` and the command entered is `cp folder_1/new_file.txt folder_2`.

```
student@workstation:~$ cp folder_1/new_file.txt folder_2
```

To delete the created file, use `rm` command. To delete the folder with all sub-files and directories, use `rm -rf`:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The prompt is `student@workstation:~$` and the commands entered are `rm folder_2/new_file.txt` and `rm -rf new_folder`.

```
student@workstation:~$ rm folder_2/new_file.txt
student@workstation:~$ rm -rf new_folder
```

Assignment 1

Complete the following assignment:

Create two new files and list all files from the current folder using one single command with the logical AND operator.

Save the command and output to the lab report.

3.3.2 Task 2: Wireless Router Configuration

Let us connect the workstation to the router using an Ethernet cable to any of the available Ethernet ports. In this task, we choose NetGear wireless router with the stock firmware already preinstalled. Let us check network settings by connecting to NetGear Graphical User Interface (GUI) from a workstation browser using the router default IP address, here <http://192.168.1.1> (Note that other router brands may have different default IP addresses). Enter the requested username and password, as it is typically written on the back of the router. Connect the router to the Internet with an Ethernet cable using the Internet modem port. Select *Advanced* to check the status of *Router Information* and *Internet port* fields, as shown in Fig. 3.3.

Under *Router Information*, verify that the router IP Address is the default one as defined by the manufacturer, in our case it should be 192.168.1.1. Under *Internet Port*, verify that the router is connected to the Internet. In case there is no access to the public Internet, the values in all fields under *Internet Port* would be set to

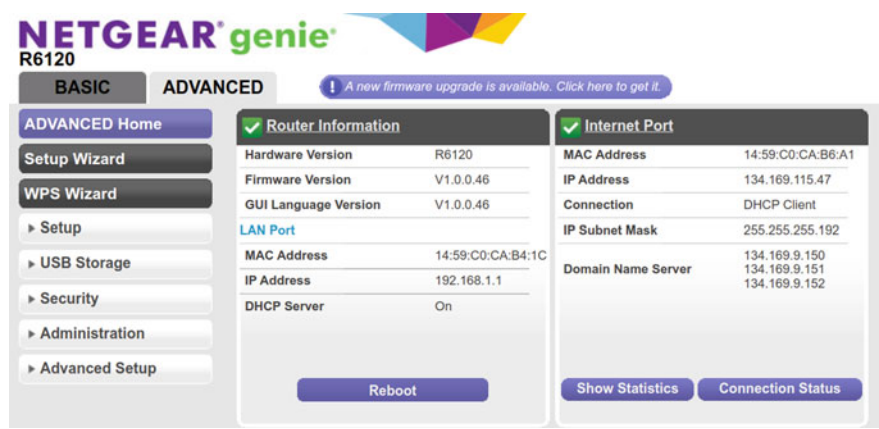


Fig. 3.3 Example of Router GUI

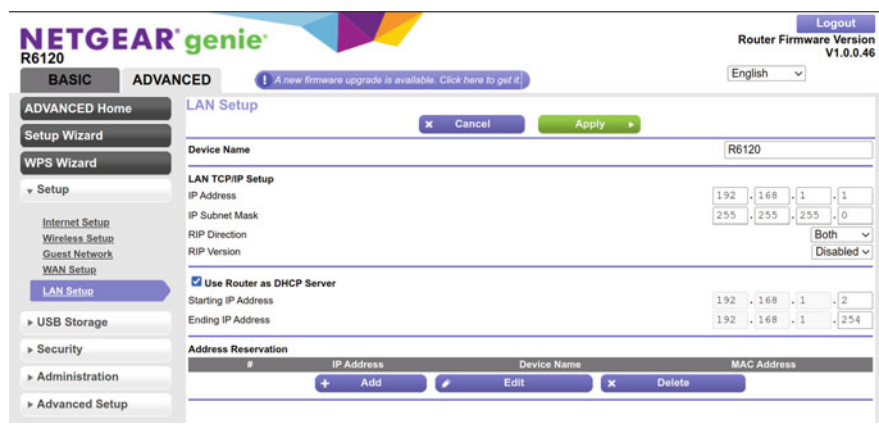


Fig. 3.4 Router GUI—Local Area Network (LAN) setup

0.0.0.0. While still under *Advanced* option, go to *Setup* field, and under this field, find *LAN setup*. Check the information under *LAN setup*, which shows the IP address and subnet mask, and make sure the DHCP server functionality is enabled in the router, as shown in Fig. 3.4.

In *Attached Devices* option, and by going back to the main NetGear GUI from *Advanced* tab to *Basic* tab, we should be now able to see all devices listed that are connected to the local area network, e.g., Fig. 3.5. In this example, we see that there is currently only one device in LAN, i.e., our workstation, with its corresponding MAC address, and its automatically assigned IP address (e.g., 192.168.1.2) as well as its connection type (wired).

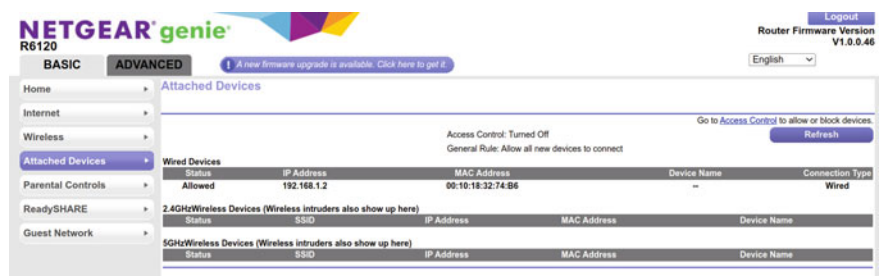


Fig. 3.5 Router GUI—attached device

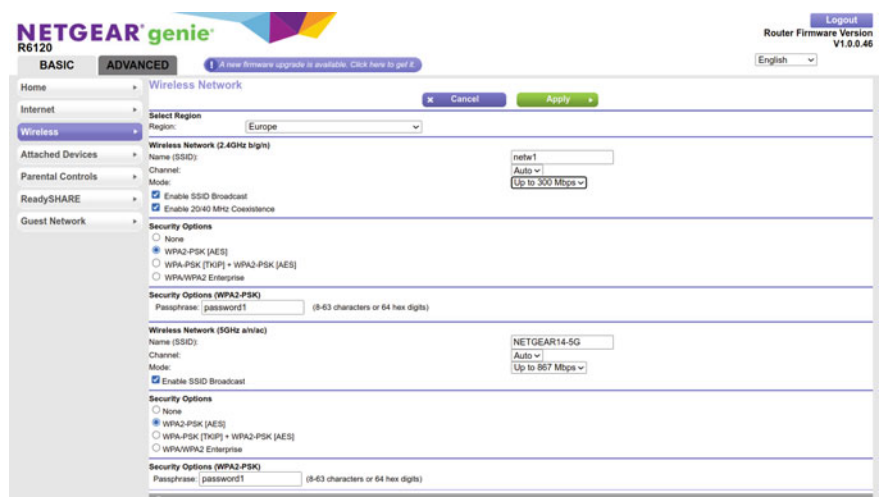


Fig. 3.6 Router GUI—wireless network

Now configure wireless access to the LAN. To this end, we first change the default network name (SSID) and password under *Wireless* option. In the example shown in Fig. 3.6, the SSID and password match *netw1* and *password1*, respectively.

Next, connect a second device to the router, this time a Raspberry Pi device, using Ethernet cable which we connect to any of the remaining Ethernet ports available. Check again the option *Attached Devices* (under *Basic* tab) in NetGear GUI to see if there is a new device connected to the LAN. As can be seen in Fig. 3.7, we now see that there are two devices in LAN, workstation and a Raspberry Pi, the latter also with its MAC address. Note that Raspberry Pi (with a device name *UBUNTU*) is automatically assigned an IP address (e.g., 192.168.1.3) and its connection type (wired).



Fig. 3.7 Router GUI—attached devices

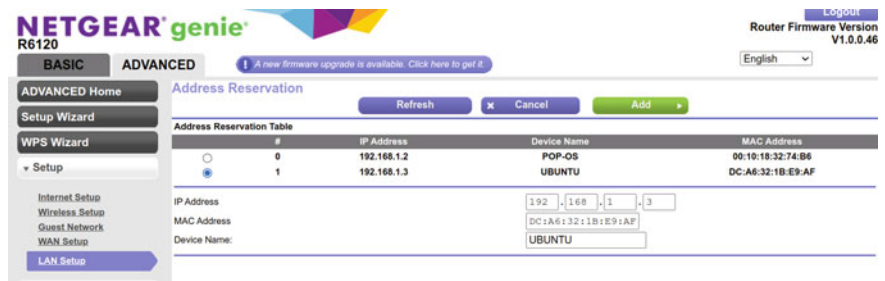


Fig. 3.8 Router GUI—assigning a static IP address

Next step is to assign a static IP address to Raspberry Pi device in order to easily access remotely to the device. Go again to *LAN Setup* option and choose *Address Reservation* and *Add*. In the opened window, choose Raspberry Pi device in *Address Reservation Table*. To reserve the assigned static IP address (e.g., 192.168.1.3), simply press (*Add*), as shown in Fig. 3.8.

Assignment 2

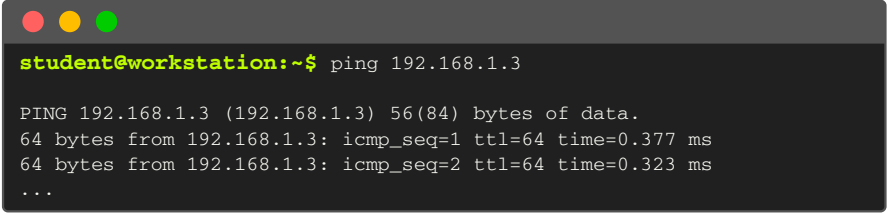
Answer the following questions:

- (a) What are private IP and what are public IP addresses?
- (b) Which ranges are reserved for private IP addresses?
- (c) What is a DHCP server?

Save the answers to the lab report.

3.3.3 Task 3: Remote Access Using SSH

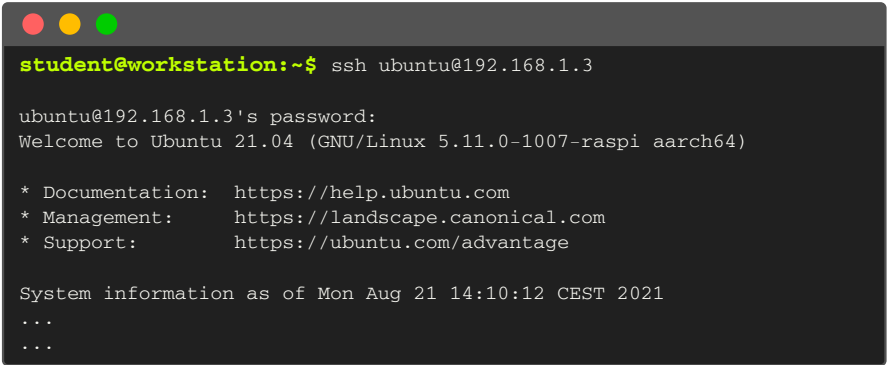
We are now ready to remotely access devices. Since we have already assigned a static IP address to Raspberry Pi, let us test that the assignment was successful by using the `ping` command on the workstation:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The prompt is `student@workstation:~$`. The command `ping 192.168.1.3` has been entered. The output shows a successful ping to 192.168.1.3 with 56(84) bytes of data, showing two successful responses with times of 0.377 ms and 0.323 ms.

```
student@workstation:~$ ping 192.168.1.3

PING 192.168.1.3 (192.168.1.3) 56(84) bytes of data:
64 bytes from 192.168.1.3: icmp_seq=1 ttl=64 time=0.377 ms
64 bytes from 192.168.1.3: icmp_seq=2 ttl=64 time=0.323 ms
...
```

If the ping was successful, we can now SSH into Raspberry Pi. Since Ubuntu on Raspberry Pi already comes with SSH server enabled by default, we can SSH using the default username `ubuntu` like this:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The prompt is `student@workstation:~$`. The command `ssh ubuntu@192.168.1.3` has been entered. The output shows the password prompt, a welcome message for Ubuntu 21.04, and system information.

```
student@workstation:~$ ssh ubuntu@192.168.1.3

ubuntu@192.168.1.3's password:
Welcome to Ubuntu 21.04 (GNU/Linux 5.11.0-1007-raspi aarch64)

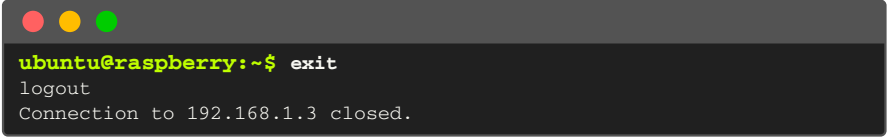
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage

System information as of Mon Aug 21 14:10:12 CEST 2021
...
...
```

Tips

In case the attempt to connect fails, please refer to Appendix [B.3](#).

In this case, since this is the first time accessing the device, the OS is asking to set up a new password. (When using SSH to access the server with a public IP, just follow the same steps except that IP needs to be replaced with the server IP address). To exit the established SSH session and return to your workstation session type `exit`:



```
ubuntu@raspberrypi:~$ exit
logout
Connection to 192.168.1.3 closed.
```

For troubleshooting, please check Appendix [B.1](#).

Assignment 3

Answer the following questions:

- (a) What are the benefits of assigning a static IP address to Raspberry Pi?
- (b) What are the requirements to establish SSH client-server remote access?

Save the answers to the lab report.

3.4 Part 2: Introduction to Arduino Programming

To program on Arduino boards, we use Arduino programming language, which is a simplified version of C++. To this end, we use Arduino IDE tool for writing the so-called *sketches* and compiling and flashing them to the Arduino boards.

3.4.1 Task 4: First Arduino Sketch

Assuming Arduino IDE is already installed on the workstation and the Arduino board already is connected to it with an USB cable, let us open Arduino IDE and start with the first project. The main file in this project is an Arduino sketch which always consists of at least two main functions we show here (see Fig. 3.9).

The first function `setup()` is only triggered once, at the beginning of the program execution. It usually contains initialization-related tasks. The `loop()` function is triggered in an iterative process, such that the code is executed repeatedly until the board is either powered off or restarted.

Let us now run a simple Arduino program, for instance, a program that let us control an on-board LED on Arduino. First of all, we check if Arduino UNO WiFi Rev2 board is connected to the workstation and on which serial port. We then launch Arduino IDE. As soon as Arduino IDE starts, it polls all devices connected with



Fig. 3.9 Arduino IDE

USB and builds a list of these devices. Select the serial device of the board from the *Tools -> Port -> Serial ports* menu. This is likely going to be */dev/ttyACM0* or */dev/ttyACM1*.

Let us now create a new folder called `int_led`. Open a text editor and save it as `int_led.ino`: this will be our main sketch file in the task to control an on-board LED. The names of the sketch and folder have to match in order for Arduino IDE to recognize it as a *project*. Add the following code to the above sketch file:

Source code: [9]

```
1 #define LED 25 // Arduino UNO WiFi Rev2's builtin LED is  
   at digital pin 25  
2  
3 void setup()  
4 {  
5     pinMode(LED, OUTPUT); // LED pin as output.  
6 }  
7  
8 void loop()  
9 {  
10     digitalWrite(LED, HIGH); // turn the LED off  
11     delay(1000); // wait for 1 second  
12     digitalWrite(LED, LOW); // turn the LED on  
13     delay(1000); // wait for 1 second  
14 }
```

Code descriptions: [9]

For Arduino UNO WiFi Rev2, the builtin LED is on digital pin 25 (first line). Function `setup()` then simply configures LED builtin pin as output. The `loop()` function alternates between powering on and off the builtin LED, by combining `digitalWrite()` and `delay()` functions.

Next, use *Verify* button in Arduino IDE (see Fig. 3.10) to check if the above sketch code compiles without errors. Verification of the code allows us to work on the sketch without having the board connected to the PC, before actually flashing the code.

After verification, configure the correct parameters with option *Tools*. The examples of correct parameters are board to use, baud rate, and correct serial ports. First, install the required package from Boards Manager from Tools -> Board menu and find Arduino megaAVR Boards, which includes Arduino Uno WiFi Rev2 (see Fig. 3.8) and click *Install* button (see Fig. 3.11).



Fig. 3.10 Arduino IDE *Verify* and *Upload*

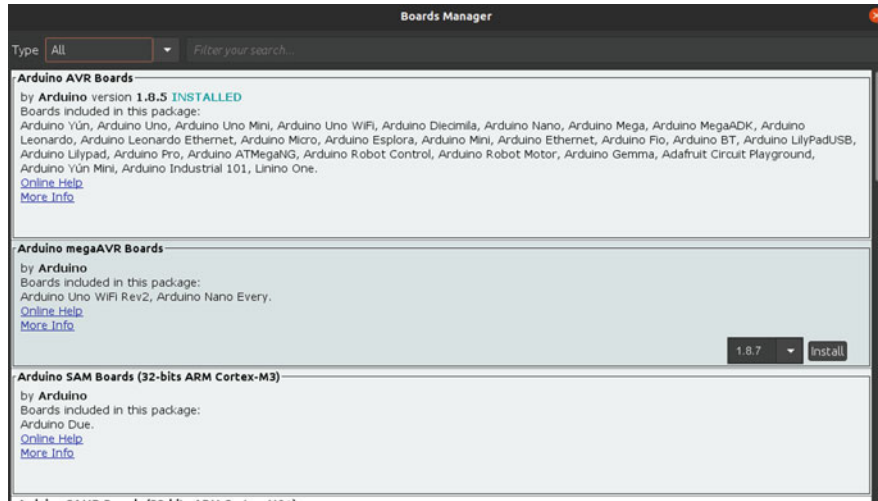


Fig. 3.11 Arduino IDE *Verify* and *Upload*

After installation, configure the following in *Tools* menu:

1. Select Arduino Uno WiFi Rev2 board in *Board*.
2. Select the serial device of the board from the *Tools* -> *Port* -> *Serial ports* menu (for example, `/dev/ttyACM0`).

Finally, flash the code with *Upload* button, and once the process is done, you should see the builtin LED blinking every second.

Tips

In case that during the upload Arduino IDE points to an error while reading the serial port, the application needs root privileges to attach to the port. Check Appendix [A.2](#) for more details.

Assignment 4

Complete the following assignment:

Modify the above example sketch by changing the rate of blinking to 500 ms ON and 500 ms OFF in two consecutive blinks on the LED. After that, add a waiting time of 2 seconds.

Add the modified code to the lab report.

3.4.2 Task 5: Connect Arduino to the WLAN

In this task, we connect the board to the local area network using the WiFi router. In this case, instead of adding all the code into a single sketch file, we divide it into three different files. This comes in handy since we can create our own custom libraries and later reuse them in other labs. The three files consist of a main sketch file `*.ino`, a header file `*.h`, containing definitions of the functions, and a source file `*.cpp`, containing the main functions of the library.

As a first step for this task, create a new folder, e.g., `WifiClient` and add a main sketch file with the same filename, e.g., `WifiClient.ino` using the CLI. After that, we add new files with the same filename, but with extensions `*.h` and `*.cpp` (`WifiClient.h` and `WifiClient.cpp`). Now open the main sketch file using Arduino IDE, which also opens the source and the header files automatically, `*.h` and `*.cpp`.

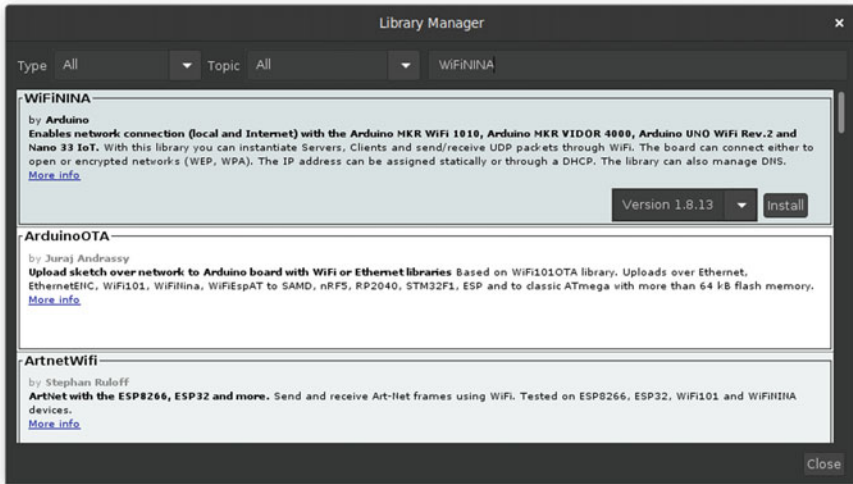


Fig. 3.12 Install library in Arduino IDE

Before following up, we now learn how to need to make sure that the external library for enabling network connection for Arduino UNO WiFi Rev.2 board is installed in the system. To this end, in Arduino IDE under *Tools*, let us find the option *Manage Libraries* for opening the *Library Manager*. Here, we can search for the required libraries and have them automatically downloaded and installed by Arduino IDE. In this case, the required library can be found under its full name *WiFiNINA* (source code available in [1]), as shown in Fig. 3.12.

Let us now add the following code into `WifiClient.h` header file:

Source code: [12]

```

1 class WifiClient
2 {
3 public:
4     WifiClient(char *ssid, char *pass);
5     void connect();
6
7 private:
8     char *_ssid;
9     char *_pass;
10 };

```

Code descriptions: [12]

This header file contains the definitions of the functions and variables required for creating the library, wrapped in a class named `WifiClient`. These functions and variables can be either public (access possible for anyone using created library) or private (access only within the class). As a first step, we use a constructor specifying network name and password (character array variables `ssid` and `password pass`, respectively), to be triggered when creating class instances. We also define a function `connect()`, which does not return anything. Finally, we define two private character variables, `_ssid` and `_pass`. In addition, the header file also includes a statement for accessing standard types, functions, and constants of the Arduino language (installed with Arduino core), as this is required in order to use a few Arduino-specific classes.

We now follow up by adding the actual code into `WifiClient.cpp` source file:

Source code: [13]

```
1 #include <WiFiNINA.h>
2 #include "WifiClient.h"
3
4 WifiClient::WifiClient(char *ssid, char *pass)
5 {
6     _ssid = ssid;
7     _pass = pass;
8 }
9
10 void WifiClient::connect()
11 {
12     WiFiClient wifiClient;
13     Serial.print("Connecting to WPA SSID [" + String(_ssid)
14         + "...");
15     while (WiFi.begin(_ssid, _pass) != WL_CONNECTED)
16     {
17         Serial.print(".");
18         delay(500);
19     }
20     IPAddress ip = WiFi.localIP();
21     Serial.print(" done, IP: ");
22     Serial.println(ip);
23 }
```


Code descriptions: [13]

This code starts by including the header file `WifiClient.h` for the definitions and the library `WifiNINA.h` for WiFi support of Arduino Uno WiFi Rev2 boards. Next, we define the constructor, which is going to be executed each time an object of this class is instantiated in the main sketch. Here, we save SSID value of the WiFi network and its corresponding password (`ssid` and `pass`) as private variables, for use in other functions. After this, we define the function that connects the board to the wireless router (including the name of the class to which it belongs), i.e.,

- `connect()`: In this function, we first define an instance of class `WifiClient`. Then we use `Serial.print()` function that prints out the message on connecting to the network; the name of the network is passed to function from parameter `_ssid`. Then, in `while` loop we use `begin()` function from the imported `WifiNINA` library which initializes the library's network settings and provides the current status. Here, we use `Serial.print()` function to print out a message that signals waiting for connection until the status returned is `WL_CONNECTED`, which is assigned when connected to wireless network. Finally, we use `Serial.print()` to print out that the connection was established, using `localIP()` function.

Once the function is created, we add the following code to the main sketch file `WifiClient.ino`:

Source code: [14]

```
1 #include "WifiClient.h"
2
3 char *ssid = "netw1";
4 char *pass = "password1";
5 WifiClient wifi_client(ssid, pass);
6
7 void setup()
8 {
9     Serial.begin(9600);
10    wifi_client.connect();
11 }
12
13 void loop()
14 {
15 }
```

Code descriptions: [14]

We start by including the header file `WifiClient.h`, which also includes the source code of `WifiClient.cpp`. After this, we define two types of character array variables, `ssid` and `pass`, and assign them values which correspond to wireless network configuration. Next, in order to use the functions defined in class `WifiClient`, we need to create an instance of this class, here named `wifi_client`, with the network name and password variables as its arguments. These arguments are passed to the constructor. In `setup()` function, we open serial communication at a baud rate of 9600 *bps*, and we trigger the `connect()` function. The `loop()` function is here empty, since it is not needed.

Also here, we use *Verify* to check if there are errors in the code. After verification, we configure the correct parameters in *Tools* menu. We select *Arduino Uno WiFi Rev2* in *Board* menu and the serial device of the board from the *Tools -> Port -> Serial ports* menu. Now we can upload the code and, when finished, use *Serial Monitor* to check the output. We set the baud rate in *Serial Monitor* to match the established serial communication. If everything goes ok, the output should now show the assigned network parameters as follows:

```
Connecting to WPA SSID [netw1]... done, IP: 192.168.1.5
```

The output matches SSID *netw1*, and we can see that the assigned IP address is in 192.168.1.0/24 range.

Assignment 5

Answer the questions and complete the following assignment:

- (a) Let us assume that we need to add a new functionality in the current empty `loop()` function that requires the WiFi connection to be up and running. Would this be the correct approach? What happens if WiFi access point becomes unavailable?
- (b) Change the current code to make it reliable against WiFi connection failures (hint: check WiFi connection status within the `loop()` function.)

Save the answers and the code to the lab report.

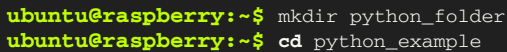
3.5 Part 3: Getting Started with Raspberry Pi

The objective of this part is twofold: to introduce the basics of Python programming and to introduce Docker basic commands. These basics are necessary for a few development tasks with Raspberry Pi in NoTE Lab.

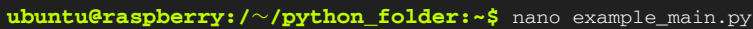
3.5.1 Task 6: The First Python Script

Most Linux distributions have Python 3.x interpreter already preinstalled. To program in Python, we can use any text editor; in our case, we make use of nano (see the full list of available options in [8]).

First, create a new directory in your Raspberry Pi and then create a python script as follows: `example_main.py`.



```
ubuntu@raspberrypi:~$ mkdir python_folder
ubuntu@raspberrypi:~$ cd python_example
```



```
ubuntu@raspberrypi:~/python_folder:~$ nano example_main.py
```

Then, add the following code using nano editor:

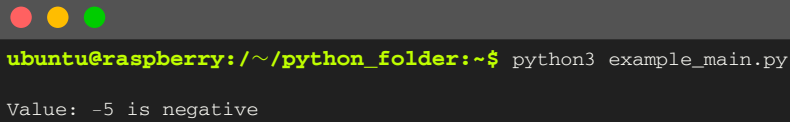
Source code: [10]

```
4 def check_sign(value):
5     if value > 0:
6         print("Value: ", value, "is positive")
7     elif value == 0:
8         print("Value:", value, "is equal to zero")
9     else:
10        print("Value:", value, "is negative")
11
12
13 check_sign(-5)
```

Code descriptions: [10]

In this script, we first define the function `check_sign()` that expects a parameter named `value`. This function checks whether that value is positive, negative, or zero making use of an `if/else` conditional statement. Then we just call the function passing the value to check as parameter, in this case `-5`.

Run the script as follows:



```
ubuntu@raspberrypi: ~/python_folder ~$ python3 example_main.py
Value: -5 is negative
```

Create a new file `example_module.py` in the same directory and add the following code:

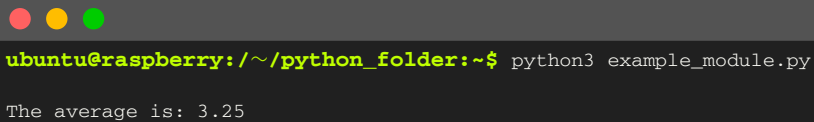
Source code: [11]

```
1 def calc_sum(values):
2     sum_result = 0
3     for value in values:
4         sum_result += value
5     return sum_result
6
7
8 def calc_avg(values):
9     avg = calc_sum(values) / len(values)
10    return avg
11
12
13 if __name__ == "__main__":
14     my_avg = calc_avg([1, 1, 1, 10])
15     print("The average is: " + str(my_avg))
```

Code descriptions: [11]

Here, we define two functions, `calc_sum()` and `calc_avg()`. Both functions take as a parameter a list of values. Function `calc_sum()` calculates the sum of these numbers using a `for` statement and returns the calculated value. Function `calc_avg()` calculates the average value by first triggering `calc_sum()` and then dividing by the length of the list. Since this script is intended to be a module, in case we want to run it as the main program, for instance, for testing purposes, as a convention we add `if __name__ == "__main__":` followed by the code to be executed. In this case, we just trigger `calc_avg()` function passing a list of integers and print the result.

Run the script as follows:



```
ubuntu@raspberrypi: ~/python_folder: ~$ python3 example_module.py
The average is: 3.25
```

Let us now go back to the previous script `example_main.py` and use it to trigger functions from `example_module.py`. For that, update the previous code to the following one:

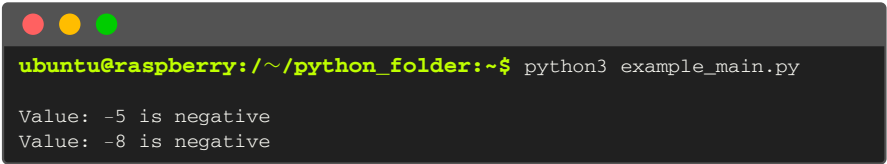
Source code: [10]

```
1 import example_module as em
2
3
4 def check_sign(value):
5     if value > 0:
6         print("Value: ", value, "is positive")
7     elif value == 0:
8         print("Value:", value, "is equal to zero")
9     else:
10        print("Value:", value, "is negative")
11
12
13 check_sign(-5)
14
15 value = em.calc_sum([2, -5, 8, -13])
16 check_sign(value)
```

Code descriptions: [10]

In this case, we add `import example_module` to import all the functions from the module we just created. Then, in line 15, we trigger function `calc_sum()` and the result value returned into variable `value`. Then, we `check_sign()` as in the previous case passing the calculated value instead.

Run this updated version of the script as follows:



```
ubuntu@raspberrypi: ~/python_folder: ~$ python3 example_main.py
Value: -5 is negative
Value: -8 is negative
```

Assignment 6

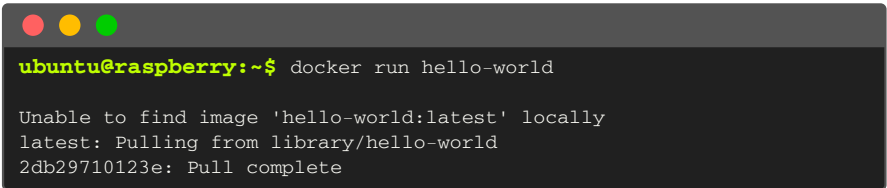
Complete the following assignment:

Write a module in which you define a function that accepts a list of values as arguments and returns the smallest number among them. Trigger the function from a separate main script by importing the module.

Add the code to the lab report.

3.5.2 Task 7: First Steps with Docker

Assuming `docker` is already installed on Raspberry Pi, we follow up with basic commands to manage the containers. First deploy a `hello-world` container with



```
ubuntu@raspberrypi: ~$ docker run hello-world

Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
2db29710123e: Pull complete
```

```
Digest: sha256:13
e367d31ae85359f42d637adf6da428f76d75dc9afeb3c21faea0d976f5c651
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working
correctly.

...
```

To check the status of all containers, run

```
ubuntu@raspberrypi:~$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	PORTS	NAMES
66b70ecc5277	hello-world	"/hello"		36
seconds ago	Exited (0) 36 seconds ago			modest_edison

In this case, only one container is currently running, named `hello-world`. Knowing the `CONTAINER_ID` tag, we can also stop and remove containers with

```
ubuntu@raspberrypi:~$ docker stop CONTAINER_ID
ubuntu@raspberrypi:~$ docker rm CONTAINER_ID
```

For further instructions, please refer to the official documentation in [7].

Assignment 7

Answer the following questions:

- What is the difference between a container and a virtual machine?
- What are the advantages and disadvantages of containers?
- What is the difference between a container and container images?
- From where does Docker pull container images by default?
- What is container registry?

Add the answers to the lab report.

3.6 Part 4: On the Server

The objective of this part is to get familiar with the basic commands for managing containers in the server with K3s.

3.6.1 Task 8: Getting Started with Kubernetes

Let us as the first step establish a remote connection to the server using SSH, and let us install `k3s` using the official installation script ():

```
ubuntu@server:~$ curl -sL https://get.k3s.io | INSTALL_K3S_VERSION=
v1.20.9+k3s1 sh -

[INFO] Finding release for channel stable
[INFO] Using v1.20.9+k3s1 as release
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/
download/v1.20.9+k3s1/sha256sum-amd64.txt
[INFO] Downloading binary https://github.com/k3s-io/k3s/releases/
download/v1.20.9+k3s1/k3s
...
...
```

Tips

In case this installation method fails because of either the script or the binaries are not available anymore, please refer to Appendix [C.4](#) for an alternative installation method.

Wait some time until `k3s` is up and running. This process may take up to 30 seconds to finalize. To check whether the installation was successful, run

```
ubuntu@server:~$ sudo kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
ubuntu-server-1106	Ready	control-plane,master	46s	v1.20.9+k3s1

If `STATUS` label is `Ready`, then the installation was successful. In order to avoid adding `sudo` in every `kubectl` command, we add `k3s` configuration to the user space with

```
ubuntu@server:~$ mkdir ~/.kube
ubuntu@server:~$ sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/k3s-config && sudo chown
$USER: ~/.kube/k3s-config && export KUBECONFIG=~/.kube/k3s-config
```

It is to be noted that we are setting an environment variable using `export` command, so you need to repeat the same command in every new terminal you open or add it to `.bashrc` or `.zshrc` files in order to be able to use `kubectl` commands without `sudo`.

The following command is useful to list all pods in all namespaces in order to check their status:

```
ubuntu@server:~$ kubectl get pods --all-namespaces
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	metrics-server-...	1/1	Running	0	2m51s
kube-system	local-path-prov...	1/1	Running	0	2m51s
kube-system	coredns-64888...	1/1	Running	0	2m51s
kube-system	helm-install-tra...	0/1	Completed	0	2m51s
kube-system	svclb-traefik-4...	2/2	Running	0	2m34s
kube-system	traefik-6f9cbd...	1/1	Running	0	2m34s

Tips

In case you need to uninstall `k3s`, please refer to [Appendix C.2](#).

In our labs, we also make use of `Helm` which is a package manager that helps managing applications on `Kubernetes`. We will use `Helm` in the following Labs to install `RabbitMQ`, `Logstash`, and `Elasticsearch`. Now, as an example of how to install an application, we are going to install a *hello-world* simple application. To do this, we need to first add the repository containing the `helm` chart as follows:

```
ubuntu@server:~$ helm repo add crowdsec https://crowdsecurity.github.io/helm-charts
```

Then, we install the helm chart with the definition of the application, in this case a *hello-world*, as follows:

```
ubuntu@server:~$ helm install my-helloworld crowdsec/helloworld --version 0.2.1

NAME: my-helloworld
LAST DEPLOYED: Sat Jul  2 12:25:28 2022
NAMESPACE: default
STATUS: deployed
REVISION: 1
...
```

Tips

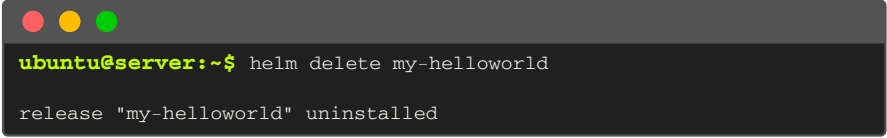
In case this installation method fails because of either the container image or the repository are not available anymore, please refer to Appendix [C.5](#) for an alternative installation method.

To list all current installed applications, use the following command:

```
ubuntu@server:~$ helm list
```

NAME	NAMESPACE	REVISION	UPDATED	APP
	STATUS	CHART		
my-helloworld	default	1	2022-07-02	helloworld-0.2.1
12:25:28.10388469 +0000 UTC	deployed			
1.16.0				

To uninstall an application, you can use the following:



```
ubuntu@server:~$ helm delete my-helloworld  
  
release "my-helloworld" uninstalled
```

Assignment 8

Answer the following questions:

- (a) What is a container orchestrator and what is it used for?
- (b) What is the difference between k3s and kubernetes?
- (c) What alternatives exist to kubernetes?
- (d) What is helm and what is it used for?

Add the answers to the lab report.

References

Libraries

1. Arduino Wi-FiNINA library source: <https://github.com/arduino-libraries/WiFiNINA>

Tools

2. Docker Engine installation on Ubuntu: <https://docs.docker.com/engine/install/ubuntu/>
3. Python: <https://www.python.org/downloads/>
4. OpenSSH Server: <https://ubuntu.com/server/docs/service-openssh>
5. CURL: <https://curl.se/download.html>
6. How to install Arduino IDE: <https://www.arduino.cc/en/Guide/Linux>

Other Resources

7. Docker official documentation: <https://docs.docker.com/engine/reference/commandline/docker/>
8. Text editors for Python: <https://wiki.python.org/moin/PythonEditors>
9. Internal LED Arduino sketch: https://github.com/tubsksns/notelab-code/blob/master/arduino/actuators/int_led/int_led.ino

10. Example Python script: https://github.com/tubskns/notelab-code/blob/master/rpi/example_main.py
11. Example Python module: https://github.com/tubskns/notelab-code/blob/master/rpi/example_module.py
12. WiFi client Arduino header file: <https://github.com/tubskns/notelab-code/blob/master/arduino/WifiClient/WifiClient.h>
13. WiFi client source file: <https://github.com/tubskns/notelab-code/blob/master/arduino/WifiClient/WifiClient.cpp>
14. WiFi client Arduino sketch: <https://github.com/tubskns/notelab-code/blob/master/arduino/WifiClient/WifiClient.ino>

Chapter 4

Lab 1—Interfacing Sensors and Actuators



4.1 Prerequisites

Hardware

- Sensors
 - (1x) - DHT11 temperature humidity sensor
 - (1x) - HC-SR04 Ultrasonic sensor
 - (1x) - HC-SR501 PIR motion sensor
- Actuators
 - (3x) - 5 mm LEDs (red, green, and yellow)
 - (1x) - 16 × 2 LCD module
- Auxiliary components
 - (3x) - resistors (330 ohm)
 - (1x) - resistor (10000 ohm)
 - (2x) - breadboards
 - (10x) - wire jumpers (min. 15 cm)
- (2x) - Arduino Uno WiFi Rev2
- Workstation
- Router

Software

- Workstation
 - Arduino IDE v1.8.13 [4]
 - OpenSSH v8.4 [3].

4.2 Setup

In *Lab 1*, we connect sensors and actuators to Arduino and program code for interfaces to read values from sensors. In addition, we learn how to trigger the actuators. The hardware setup for this lab is shown in Fig. 4.1, whereby Arduinos connect to the workstation via USB ports.

This lab is divided into five different parts. The first two parts are dedicated to writing interfaces to control the actuators, and the remaining three parts focus on writing interfaces for reading values from the sensors. This lab uses following libraries:

- Sensors interfaces
 - DHT11 library v1.4.3 [1]
- Actuator interfaces
 - Liquid Crystal_I2C library v1.1.2 [1]

Make sure the required external libraries are installed in the system. In Arduino IDE under *Tools*, we find the option *Manage Libraries*, from where we can search for the required libraries and have them automatically download and installed by Arduino IDE. DHT11 Library can be found under the full name *DHT sensor library* (source code available in [2]), while Liquid Crystal I2C library can be found under the full name *Liquid Crystal I2C* (source code available in [1]).

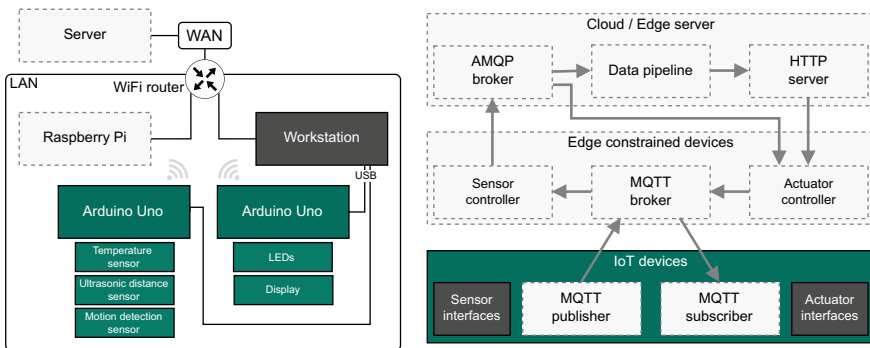


Fig. 4.1 Lab 1 setup

4.3 Part 1: Actuating an External LED

In this part of the lab, we connect an external LED (red) to Arduino Uno and write the code to make a LED blink periodically.

4.3.1 Task 1: Connecting External LED

Follow the schematics shown in Fig. 4.2 to connect Arduino Uno WiFi Rev2 board and a LED.

The standard 5 mm LEDs have two-pin interfaces with a long leg (anode) for positive voltage supply and a smaller leg (cathode) for negative voltage supply. A LED is used in series with a resistor, thus preventing its damage. Connect a red LED with a resistor of 330 ohm to the Arduino according to Table 4.1.

Assignment 1

Answer the following questions:

- (a) Is the resistor of 330 ohm adequate for red LED? Calculate the proper resistance based on the LED and Arduino datasheets.
- (b) What happens to the LED if the resistance used is higher than the calculated one? And what, if it is lower?

Add the answers to the lab report.

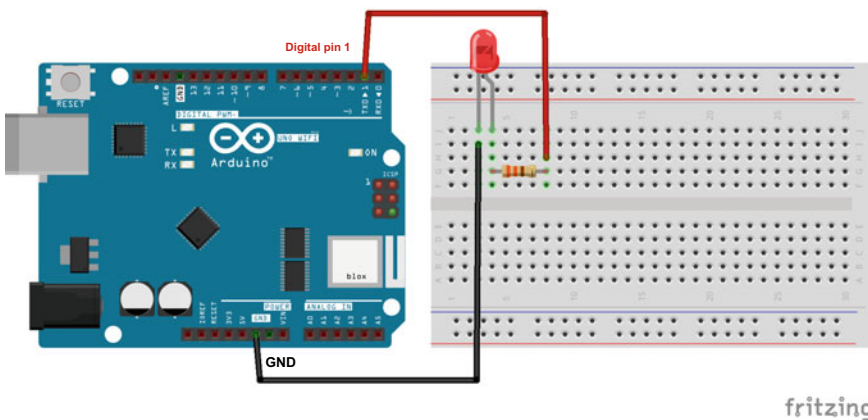


Fig. 4.2 Connecting an external LED to Arduino Uno WiFi Rev2

Table 4.1 Mapping between the LED and Arduino

LED	Arduino Uno WiFi Rev2
Anode	Digital pin 1
Cathode	GND

4.3.2 Task 2: Making LED Blink

To make a LED blink, we start with programming a sketch in Arduino that can actuate and test the LED. In Arduino IDE, let us create a new folder for the project (e.g., `ext_led`), and within the folder create a file with `(*.ino)` extension (e.g., `ext_led.ino`). This file is now the main sketch file for this task (*remember*: the name of the sketch and folder need to match). Then, add the following code to this sketch file:

Source code: [7]

```
1 #define LED 1 // external LED is connected to
   board's digital pin 1
2
3 void setup()
4 {
5     pinMode(LED, OUTPUT); // initialize LED as
       an output
6 }
7
8 void loop()
9 {
10     digitalWrite(LED, HIGH); // turn the LED off
11     delay(1000);              // wait for 1 second
12     digitalWrite(LED, LOW);  // turn the LED on
13     delay(1000);              // wait for 1 second
14 }
```

Code descriptions: [7]

In the `setup` function, we initialize the output of the Arduino that activates the LED, in this case digital pin 1 of Arduino Uno WiFi Rev2 board. The `loop()` function alternates between powering on and off the external LED, by combining `digitalWrite()` and `delay()` functions.

Now, proceed to *Verify* to check the code is without errors and then *Upload* the code in order to flash it on the Arduino. For more details about this process, please, go back to Lab 0, Sect. 3.4.1.

We can now verify that the external LED is properly blinking.

Assignment 2

Complete the following assignment:

Modify `loop()` function to make the LED blink every second by only triggering `digitalWrite()` function once (hint: use a global variable to store the state of LED).

Save the code to the lab report.

4.4 Part 2: Actuating an LCD Display

We now connect an I2C Serial LCD display to the Arduino and program a small piece of code to actuate it, by displaying a test message on its display.

4.4.1 Task 3: Connecting LCD Display

Connect Arduino Uno WiFi Rev2 board to I2C Serial 16×2 LCD display as illustrated in Fig. 4.3. Note that 16×2 refers to the number of columns and rows on which the display can print characters. Check the datasheet in [LCD datasheet] for more information.

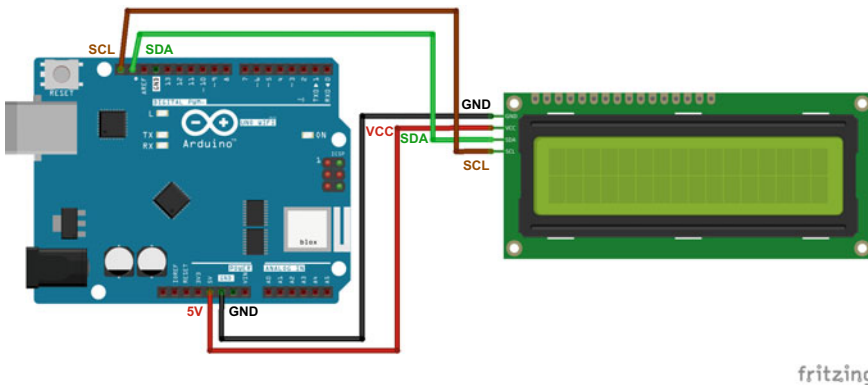


Fig. 4.3 Connecting LCD display to Arduino Uno WiFi Rev2

Table 4.2 Mapping between LCD display and Arduino

16 × 2 LCD	Arduino Uno WiFi Rev2
GND	GND
VDD	5 V
SDA	SDA
SCL	SCL

Recall that I2C Serial 16 × 2 LCD display has a four-pin interface through its I2C connector. Connect the display to Arduino according to Table 4.2.

Assignment 3

Answer the following questions:

- (a) What is I2C? How does it work? What is its main advantage?
- (b) Check the provided I2C Serial 16×2 LCD display. What makes this display different from traditional LCD displays which are not using the I2C interface? Is it possible to make traditional LCD display to work using I2C communication? How?

Add the answers to the lab report.

4.4.2 Task 4: Printing Messages on LCD Display

To test the LCD display, we need a new sketch in Arduino IDE. Also here, we create a new folder for the project (e.g., LCD), and within the folder a file with (*.ino) extension (e.g., LCD.ino). This file is now the main sketch file for this task (*remember*: the name of the sketch and folder need to match). Then, add the following code to the sketch file:

Source code: [5]

```
1 #include <Wire.h>
2 #include <LiquidCrystal_I2C.h>
3
4 LiquidCrystal_I2C lcd(0x27,16,2);
5
6 void setup() {
7   lcd.init();
8   lcd.backlight();
9   lcd.setCursor(0,0);
10  lcd.print("Test message");
11  lcd.setCursor(0,1);
```

```
12 lcd.print("from NoteLab");  
13 }  
14  
15 void loop() {  
16  
17 }
```

Code descriptions: [5]

This file starts with including `Wire.h` library which is required to write data to I2C peripheral devices. This is an inbuilt Arduino library so there is no need to install it separately. We also include `LiquidCrystal_I2C.h` library to allow an Arduino board to control liquid crystal displays (LCDs). Next, we initialize an object of this library `lcd`, setting I2C address (0×27) and display's dimensions (16 columns and 2 rows).

In function `setup()`, we first use `init()` function to initialize LCD, followed with `backlight()` function, which turns on the backlight. Next, with function `setCursor()`, we define a starting point for printing, so that the first integer defines the column and the second the row. In this example, we start from the first column and first row, following up with function `print()` to print the first part of the message. Finally, we print the second part of the message in the second row.

Now, proceed to *Verify* to check the code is without errors and then *Upload* the code in order to flash it on the Arduino. For more details about this process, please, go back to Lab 0, Sect. 3.4.1.

We can now verify that the LCD display shows now the test message.

Assignment 4

Answer the following questions:

- Which integer parameters would we pass to function `lcd.setCursor()` in order to start printing in the tenth column of the first row.
- Assuming we set `lcd.setCursor()` to the proper values (tenth column, first row), what happens if we try to print "hello world" from that position?

Add the answers to the lab report.

4.5 Part 3: Temperature/Humidity Sensor

We now connect a temperature and humidity sensor (DHT11) to Arduino and read and print the measured values to the console output on the workstation.

4.5.1 Task 5: Connecting DHT11 Sensor

Connect DHT11 sensor to Arduino Uno WiFi Rev2 board as shown in Fig. 4.4. Recall that in case that another model of the board is used, the pins need to be updated according to the corresponding datasheets.

Just like LED previously explained, DHT11 is also used in series with a resistor to avoid damages. Connect DHT11 to Arduino according to Table 4.3, with 10000 ohms resistor between data pin and VCC pin.

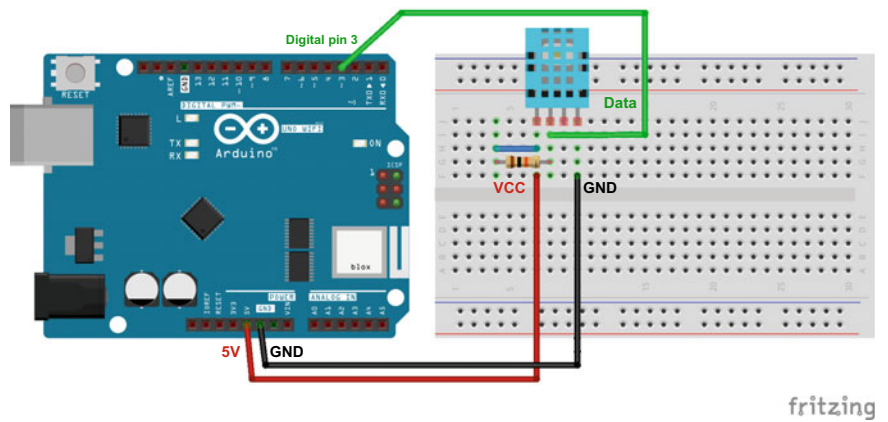


Fig. 4.4 Connecting DHT11 sensor to the Arduino

Table 4.3 Interfacing DHT11 with Arduino Uno WiFi Rev2

DHT11	Arduino Uno WiFi Rev2
VCC	5 V
GND	GND
Data	Digital pin 3

Assignment 5

Answer the following questions:

- (a) What does it mean by *relative humidity*?
- (b) Can DHT11 return humidity values 100% or 0%?
- (c) What is the accuracy of DHT11 for temperature? And the sampling rate?

Add the answers to the lab report.

4.5.2 Task 6: Reading Temperature and Humidity

To read and print sensor data, we need a new sketch file. We first create a new folder for the project (e.g., DHT11) and in the folder we create a file with (*.ino) extension (e.g., DHT11.ino). This file is now the main sketch file for this task. We then add the following code to the sketch file:

Source code: [6]

```
1 #include "DHT.h" // DHT module library
2
3 #define DHTPIN 3 // DHT11's data pin is
   connected to board's digital pin 3
4
5 #define DHTTYPE DHT11 // DHT type
6 DHT dht(DHTPIN, DHT11); // instantiate DHT object
7
8 void setup()
9 {
10     Serial.begin(9600);
11     dht.begin(); // initialize the sensor
12 }
13
14 void loop()
15 {
16     float humidity = dht.readHumidity();
17     if (!isnan(humidity)) {
18         String humidity_str = String(humidity);
19         //convert float to string
```

```

19     Serial.print("Humidity: " + humidity_str + "
        %\n");
20 } else {
21     Serial.println("Humidity reading from DHT11
        sensor failed!");
22 }
23
24 float temperature = dht.readTemperature();
25 if (!isnan(temperature))
26 {
27     String temperature_str =
        String(temperature); //convert float to
        string
28     Serial.print("Temperature: " +
        temperature_str + " *C\n");
29 } else {
30     Serial.println("Temperature reading from
        DHT11 sensor failed!");
31 }
32 delay(1000);
33 }

```

Code descriptions: [6]

The code first includes `DHT.h` header file for the definitions, and the library for DHT sensor. After this, according to Fig. 4.4 and following Table 4.3, we define the data pin on DHT11. In this example, DHT data pin is connected to Arduino Uno WiFi Rev2 board digital on pin 3. We also define the type of DHT sensor to be used (in this case DHT11) and initialize a DHT object with the following two parameters: DHT type and data pin.

Let us explain the functions used:

- `setup()`: This function opens serial communication at baud rate of 9600 *bps* and initializes the sensor by calling function `begin()` from the imported DHT sensor library.
- `loop()`: This function triggers `readHumidity()` function from the imported DHT sensor library, which reads humidity data and returns a float value as percentage. We save the returned value into a float variable. We also include conditional statement to check if the humidity readings are valid. In case of failed readings, a message is printed and the value returned is 0. Otherwise, the values from humidity variable are read. We then convert the said float value into a string type before printing it by using `Serial.print()` function.

To read and print the temperature values, we follow a similar procedure. We repeat the process for reading and printing, where we trigger `readTemperature()` function from the imported DHT sensor library. This function reads temperature data and returns temperature values in the selected temperature scale (Celsius or Fahrenheit). Before reading again the new values, we add a delay of 1 s using `delay()` function.

Now, proceed to *Verify* to check the code is without errors and then *Upload* the code in order to flash it on the Arduino. For more details about this process, please, go back to Lab 0, Sect. 3.4.1.

We can now use *Serial Monitor* to check the output. Set the baud rate in *Serial Monitor* to match the established serial communication. If everything goes ok, the output should show the temperature and humidity values in this form:

```
Humidity: 29.00 %  
Temperature: 26.00 *C  
Humidity: 29.00 %  
Temperature: 26.30 *C  
...
```

Assignment 6

Complete the following assignment:

Connect both red LED and DHT11 sensor to Arduino. Combine the code from this task with the code developed in the previous tasks to turn on LED for 5 seconds but only upon a change in the sensing temperature (hint: use global variable to store the measured temperature).

Add the code to the lab report.

4.6 Part 4: Ultrasonic Distance Sensor

In this part of the lab, we connect an ultrasonic distance sensor (HC-SR04) to Arduino and read and print the measured values to the console output.

4.6.1 Task 7: Connecting HC-SR04 Sensor

Connect the ultrasonic distance sensor HC-SR04 to Arduino Uno WiFi Rev2 board as shown in Fig. 4.5. Interface HC-SR04 to Arduino according to Table 4.4.

Assignment 7

Answer the following questions:

(a) Knowing that HC-SR04 emits an ultrasound at 40kHz which bounces back to the sensor and considering the travel time and the speed of sound, how can distance be calculated?

(b) What is the accuracy of an HC-SR04 sensor?

Add the answers to the lab report.

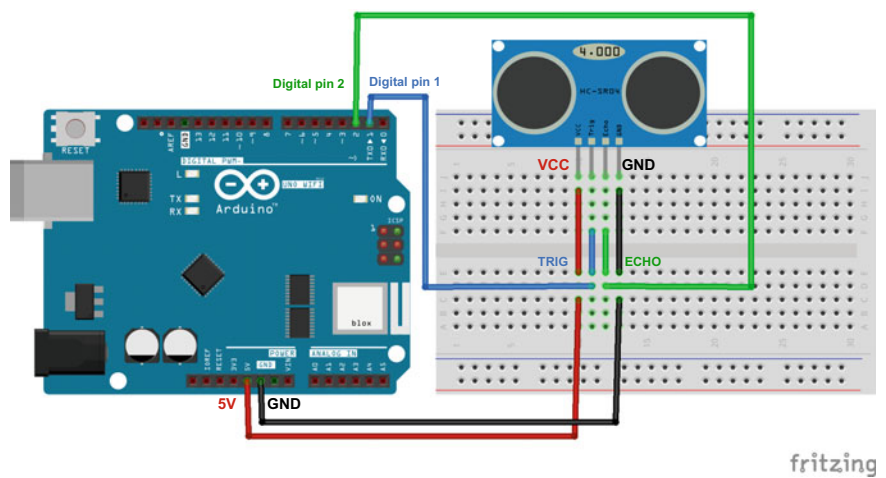


Fig. 4.5 Connecting HC-SR04 sensor to Arduino

Table 4.4 Interfacing HC-SR04 with Arduino Uno WiFi Rev2

HC-SR04	Arduino Uno WiFi Rev2
VCC	5 V
GND	GND
Trigger	Digital pin 1
Echo	Digital pin 2

4.6.2 Task 8: Calculating Distance

Instead of writing code into a single sketch file, in this task, we divide the code into different files and use them either individually, or altogether. Let us start by creating a new folder for the project (e.g., HCSR04), and in this folder, we create the main sketch file with the same filename (e.g., HCSR04.ino). Then, let us add new files with the same filename but with extensions *.h and *.cpp (e.g., HCSR04.h and HCSR04.cpp). As a result, when opening the main sketch with Arduino IDE, all three files automatically open. Let us add the following code into HCSR04.h header file:

Source code: [8]

```
1
2 class HCSR04
3 {
4     public:
5         HCSR04(int trigPin, int echoPin);
6         float calculate_distance();
7
8     private:
9         int _trigPin;
10        int _echoPin;
11 };
```

Code descriptions: [8]

This header file contains the definitions of the functions and variables required for creating the library and wrapped in a class called HCSR04. These functions and variables can be either public (accessed by anyone using created library) or private (access only within class). Let us create a constructor specifying which pins are used for trigger and echo pins of the sensor and are called when creating class instances. We also define function `calculate_distance()`, which returns an integer value of the measured distance. Finally, we define two private integer variables, `_trigPin` and `_echoPin`.

Then, we add the actual code into the HCSR04.cpp source file:

Source code: [9]

```

1 #include "HCSR04.h"
2 #include <Arduino.h>
3
4
5 HCSR04::HCSR04(int trigPin, int echoPin)
6 {
7     pinMode(trigPin, OUTPUT); // set the trigger
8     pin as output
9     pinMode(echoPin, INPUT); // set the echo pin
10    as input
11    _trigPin = trigPin;
12    _echoPin = echoPin;
13 }
14
15 float HCSR04::calculate_distance()
16 {
17     // write a pulse: when trigger pin is set to
18     HIGH, start transmitting
19     digitalWrite(_trigPin, LOW); // clear the
20     trigger pin
21     delayMicroseconds(10);
22     digitalWrite(_trigPin, HIGH); //set the
23     trigger pin to HIGH for 10 ms
24     delayMicroseconds(10);
25     digitalWrite(_trigPin, LOW);
26
27     long duration = pulseIn(_echoPin, HIGH); //
28     time elapsed for sending and receiving
29     pulse wave
30     float distance = duration * 0.034 / 2; //
31     calculate distance in cm
32
33     return distance; //return distance
34 }

```

Code descriptions: [9]

This file starts with including `HCSR04.h` header file for the definitions and the library for Arduino functions. Since this library is installed with Arduino core, it does not need to be installed separately. Next, we define the constructor, and this code is going to be executed each time an object of this class is created in the main sketch program. Here, we configure trigger pin and echo pin to act

as an output and an input, respectively, after which we save them in private variables for use in other functions. We then define the following functions (including the name of the class to which it belongs):

- `calculate_distance()`: This function first triggers the sensor, which is achieved with Arduino functions `digitalWrite()`; `delayMicroseconds()`. These two functions are used interchangeably, writing the LOW pulse value to clear the pin and then HIGH pulse value for 10ms to trigger the sensor. Function `calculate_distance()` also includes one variable of type long named `duration` and one variable of type integer named `distance`. Variable `duration` stores the time elapsed between sending and receiving of the pulse wave. Based on this time, a distance in cm is calculated, multiplying it by 0.034s/cm (speed of sound) divided by 2, and stored in variable `distance`, which is then returned.

Once the function is created, we add the following code into the main sketch file `HCSR04.ino`:

Source code: [10]

```
1 #include "HCSR04.h"
2
3 const int trigPin = 1; // HC-SR04's trigger pin
  is connected to board's digital pin 1
4 const int echoPin = 2; // HC-SR04's echo pin is
  connected to board's digital pin 2
5
6 HCSR04 hcsr04(trigPin, echoPin);
7
8 void setup()
9 {
10     Serial.begin(9600);
11 }
12
13 void loop()
14 {
15     float distance = hcsr04.calculate_distance();
16     String distance_str = String(distance); //
      convert int to string
17     Serial.print("Distance: " + distance_str + "
      cm\n");
18     delay(1000);
19 }
```

Code descriptions: [10]

The first step is to include `HCSR04.h` header file with source code in `HCSR04.cpp` as well. We then define the trigger and echo pin on HC-SR04. In this example, trigger pin is connected to Arduino Uno WiFi Rev2 digital pin 1 and echo pin to digital pin 2. Next, in order to use the functions defined in `HCSR04` class, we create an object of this class, which we name `hcsr04`. In this class, trigger and echo pin variables are its arguments that need to be passed to the constructor. We then use the following functions:

- `setup()`: In this function, we open serial communication at a baud rate of 9600 *bps*.
- `loop()`: In this function, we first trigger the `calculate_distance()` function and save the returned value into an integer variable `distance`. We then convert that integer value into a string type before printing, by using `Serial.print()` function. After that, we add a delay of 1 s using the `delay()` function, before reading again the new values.

Now, proceed to *Verify* to check the code is without errors and then *Upload* the code in order to flash it on the Arduino. For more details about this process, please, go back to Lab 0, Sect. 3.4.1.

We can now use *Serial Monitor* to check the output. If everything goes ok, the output should now show the distance values in the following form:

```
Distance: 4 cm
Distance: 5 cm
...
```

Assignment 8

Complete the following assignment:

Connect both red LED and HC-SR04 sensors to the Arduino. Combine the code used in this Task with the code implemented previously in this lab in order to turn the LED on when the calculated distance is shorter than 10 cm; turn off LED otherwise.

Add the code to the lab report.

4.7 Part 5: Motion Sensor

We now connect a motion sensor (PIR, HC-SR501) to Arduino and read and print the measured values to the console output.

4.7.1 Task 9: Connecting HC-SR501 Sensor

Connect the motion sensor HC-SR501 to Arduino Uno WiFi Rev2 board as shown in Fig. 4.6 and Interface HC-SR501 to Arduino according to Table 4.5.

Assignment 9

Answer the following questions:

- What is the infrared radiation which HC-SR501 sensor is expected to detect?
- What is the sensitivity of HC-SR501 sensor? Can it be adjusted?

Add the answers to the lab report.

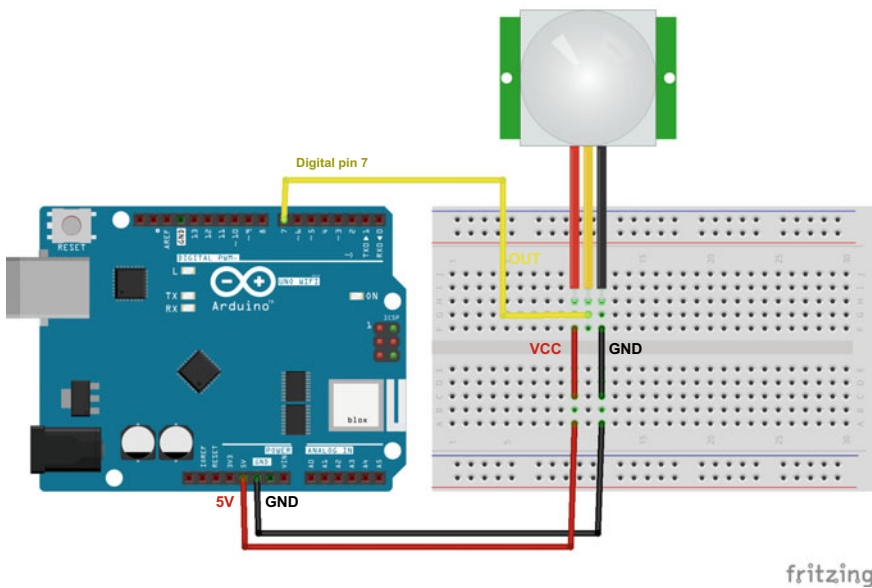


Fig. 4.6 Connecting HC-SR501 sensor to Arduino

Table 4.5 Interfacing HC-SR501 sensor with Arduino Uno WiFi Rev2

PIR	Arduino Uno WiFi Rev2
VCC	5 V
GND	GND
OUT	Digital pin 7

4.7.2 Task 10: Detecting Motion

We now program a sketch in Arduino to read the values from the sensor. Also here, the code is divided into different files (main sketch file *.ino, header file *.h, and source file *.cpp). In this exercise, we start by creating a new folder for the project (e.g., PIR), and the main sketch file with the same filename (e.g., PIR.ino). We add new files with the same filename but with extensions *.h and *.cpp (e.g., PIR.h and PIR.cpp). As a result, when opening the main sketch with Arduino IDE, all three files automatically open. We start by adding the following code into PIR.h header file:

Source code: [11]

```
1
2 class PIR
3 {
4     public:
5         PIR(int pin);
6         bool detect_motion();
7
8     private:
9         int _pin;
10 };
```

Code descriptions: [11]

This header file contains the definitions of the functions and variables required for creating the library, wrapped in a class named PIR. As a first step, we use a constructor specifying which pin is going to be used as sensor input, to be called when creating class instances. We also define a function detect_motion(), which returns a boolean value depending on whether the motion was detected, or not. Finally, we define a private integer variable, _pin.

Then, we follow up by adding the actual code into `PIR.cpp` source file:

Source code: [12]

```
1 #include "PIR.h"
2 #include <Arduino.h>
3
4
5 PIR::PIR(int pin)
6 {
7     pinMode(pin, INPUT);
8     _pin = pin;
9 }
10
11 bool PIR::detect_motion()
12 {
13     int pinState = digitalRead(_pin);
14     if (pinState == HIGH)
15     {
16         return true;
17     }
18     else
19     {
20         return false;
21     }
22 }
```

Code descriptions: [12]

We first include `PIR.h` header file for the definitions and the library for Arduino functions (installed along with Arduino core). Next, we define the constructor for creating an object of this class, in which we configure the sensor's out pin to act as an input, after which we save it in a private variable for use in other functions. We define the following function (including the name of the class to which it belongs):

- `detect_motion()`: This function first reads the digital value of the sensor's output pin with Arduino function `digitalRead()` and stores the value to an integer variable `pinState`. Then a conditional statement is defined to check whether the motion is detected (the read value is `HIGH`), in which case this function returns `true`, or `false`, otherwise.

Once the functions are created, we add the following code into the main sketch file `PIR.ino`:

Source code: [13]

```
1 #include "PIR.h"
2
3 const int pirSensor = 7; // PIR's pin is
   connected to board's digital pin 7
4 PIR pir(pirSensor);
5
6 void setup()
7 {
8     Serial.begin(9600);
9 }
10
11 void loop()
12 {
13     bool is_motion = pir.detect_motion();
14     if (is_motion == true)
15     {
16         Serial.println("Motion detected!");
17     }
18     else
19     {
20         Serial.println("Motion absent!");
21     }
22     delay(1000);
23 }
```

Code descriptions: [13]

The first step is to include `PIR.h` header file which includes the source code `PIR.cpp` as well. We then define PIR's out pin (`pirSensor`), depending on the Arduino board used. In this example, the code shown matches digital pin 7 of Arduino Uno WiFi Rev2 board. Next, in order to use the functions defined in the `PIR` class, we create an object of this class, here named `pir`, with the sensor's pin variable as its argument, which is going to be passed to the constructor. We use the following functions:

- `setup()`: This function opens serial communication at baud rate of 9600 *bps*.

- `loop()`: This function triggers `detect_motion()` function and saves the returned value into a boolean variable `is_motion`. We then use a conditional statement to check if the motion is detected or not, based on the values returned by `detect_motion()` function. Depending on this value, now stored in variable `is_motion`, the corresponding message is printed using `Serial.print()` function. After that, we add a delay of 1 second using `delay()` function, before reading the new values again.

Now, proceed to *Verify* to check the code is without errors and then *Upload* the code in order to flash it on the Arduino. For more details about this process, please, go back to Lab 0, Sect. 3.4.1.

We can now use *Serial Monitor* to check the output. If everything goes ok, the output should now show the motion detection values in the following form:

```
PIR test!  
Motion absent!  
Motion detected!  
Motion detected!  
...
```

Assignment 10

Complete the following assignment:

Connect red LED and HC-SR501 sensor to Arduino. Combine the code from this task with the code implemented in the first part of this lab to turn the LED on, only if motion is detected; turn it off otherwise.

Add the code to the lab report.

References

Libraries

1. Source code LiquidCrystal_I2C library: https://github.com/johnrickman/LiquidCrystal_I2C
2. Source code DHT11 library: <https://github.com/adafruit/DHT-sensor-library>

Tools

3. OpenSSH Server: <https://ubuntu.com/server/docs/service-openssh>
4. How to install Arduino IDE: <https://www.arduino.cc/en/Guide/Linux>

Other Resources

5. LCD Arduino sketch: <https://github.com/tubskns/notelab-code/tree/master/arduino/actuators/LCD>
6. DHT11 Arduino sketch: <https://github.com/tubskns/notelab-code/blob/master/arduino/sensors/temperature/DHT11/DHT11.ino>
7. External LED Arduino sketch: https://github.com/tubskns/notelab-code/blob/master/arduino/actuators/ext_led/ext_led.ino
8. HCSR04 Arduino header file: <https://github.com/tubskns/notelab-code/blob/master/arduino/sensors/distance/HCSR04/HCSR04.h>
9. HCSR04 Arduino source file: <https://github.com/tubskns/notelab-code/blob/master/arduino/sensors/distance/HCSR04/HCSR04.cpp>
10. HCSR04 Arduino sketch: <https://github.com/tubskns/notelab-code/blob/master/arduino/sensors/distance/HCSR04/HCSR04.ino>
11. PIR Arduino header file: <https://github.com/tubskns/notelab-code/blob/master/arduino/sensors/motion/PIR/PIR.h>
12. PIR Arduino source file: <https://github.com/tubskns/notelab-code/blob/master/arduino/sensors/motion/PIR/PIR.cpp>
13. PIR Arduino sketch: <https://github.com/tubskns/notelab-code/blob/master/arduino/sensors/motion/PIR/PIR.ino>

Chapter 5

Lab 2—Connecting IoT and Edge with MQTT



5.1 Prerequisites

Hardware

- Sensors
 - (1x) - DHT11 temperature humidity sensor
 - (1x) - HC-SR04 Ultrasonic sensor
 - (1x) - HC-SR501 PIR motion sensor
- Actuators
 - (1x) - 5 mm LED (red color)
 - (1x) - 16×2 LCD module
- Auxiliary components
 - (1x) - resistor (330 ohm)
 - (1x) - resistor (10000 ohm)
 - (2x) - breadboards
 - (10x) - wire jumpers (min. 15 cm)
- (2x) - Arduino Uno WiFi Rev2
- Raspberry Pi 4 Model B
- Workstation
- Router

Software

- Raspberry Pi 4 Model B
 - Docker v20.10.8 [8]
 - Python v3.9 [9]
 - OpenSSH v8.4 [10]

- Workstation
 - Arduino IDE v1.8.13 [11]
 - OpenSSH v8.4 [10]
 - mosquitto-clients v1.6.9-1 [12]

5.2 Setup

In *Lab 2*, we connect Arduinos and Raspberry Pi within a wireless local area network (WLAN) and use MQTT protocol for their communication. This lab module includes a series of tasks that include programming in Arduino IDE and Python. The lab setup is shown in Fig. 5.1, and it includes Arduinos, sensors, actuators, auxiliary components, Raspberry Pi, WiFi router, and workstation machine. This lab also implements a new tool, which is a Mosquitto broker on Raspberry Pi. On Arduinos, MQTT publishers and subscribers are implemented using `arduino-mqtt` library.

The main lab components are shown in Fig. 5.2. In Part 1 of *Lab 2*, we deploy and test Mosquitto, which implements an MQTT broker, on Raspberry Pi. For testing purposes, we use `mosquitto-clients`, a tool which allows us to publish and subscribe to the MQTT broker. In Part 2, an Arduino sketch of MQTT client is implemented in Arduino IDE, used publisher and subscriber to MQTT broker. In Part 3, sensor data is sent to the broker using the publisher functionality of the MQTT client. At the same time, the actuators are activated by actuator interfaces on the Arduino using the subscriber functionality of the MQTT client. A detailed overview of the libraries, which are used in this lab, is listed as follows:

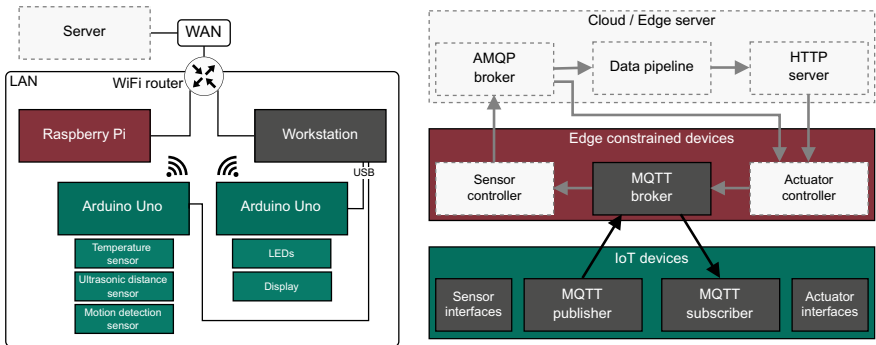


Fig. 5.1 Lab 2 setup

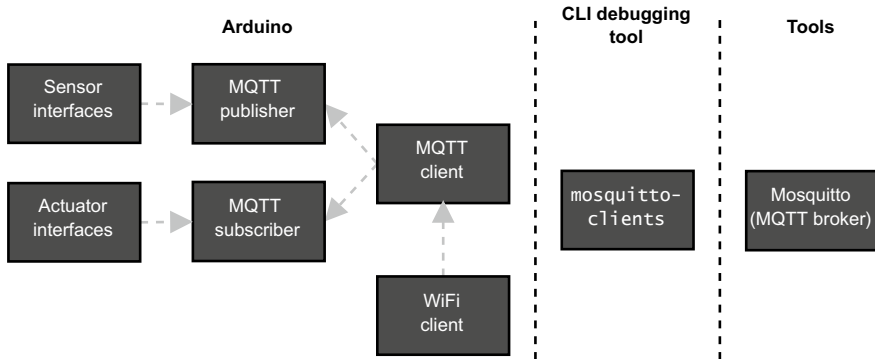


Fig. 5.2 Lab 2 components

- Sensors interfaces
 - DHT11 library v1.4.3 [7]
- Actuator interfaces
 - Liquid Crystal_I2C library v1.1.2 [6]
- MQTT publisher and subscriber
 - WiFiNINA library v1.8.13 [2]
 - WiFi client Arduino header [4] and source file [5] (from Lab 0)
 - Arduino MQTT library v2.5.0 [3]
 - Arduino JSON library v6.19.4 [1]
- MQTT broker
 - Eclipse Mosquitto v1.6.15 [13]

Recall that it is important to make sure that the required external libraries are installed in the system. In Arduino IDE under *Tools*, find the option *Manage Libraries*, from where we can search for the required libraries and have them automatically downloaded and installed by Arduino IDE. WiFiNINA library can be found under the full name *WiFiNINA* (source code available in [2]), DHT11 Library can be found under the full name *DHT sensor library* (source code available in [7]), Liquid Crystal I2C library can be found under the full name *Liquid Crystal I2C* (source code available in [6]), MQTT library can be found under the full name *MQTT library for Arduino* (source code available at [3]), and finally, Arduino JSON library can be found under the full name *Arduino_JSON Library for Arduino* (source code available at [1]).

5.3 Part 1: Deploying and Testing an MQTT Broker

The first task is to install MQTT broker on Raspberry Pi using Mosquitto broker. Then, we test the publish/subscribe communication using the mosquitto-clients tool (See Fig. 5.3).

5.3.1 Task 1: Running Mosquitto

We first check the official docker image of Mosquitto broker, i.e., [13]. As docker images are frequently updated, it is important to always download a stable version, e.g., current version is *1.6.15*. From there, we can find out that ARM64 architecture is supported which is a necessary requirement in NoTE Lab to run the broker on Raspberry Pi. Let us connect remotely to Raspberry Pi using SSH and check if the docker is up and running (use `Ctrl + C` to terminate afterward):

```
ubuntu@raspberrypi:~$ sudo systemctl status docker
docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset
   : enabled)
   Active: active (running) since Mon 2021-08-09 16:24:00 CEST; 1min 14s ago
   ...
```

We now run the container using the default configuration as follows:

```
ubuntu@raspberrypi:~$ docker run -it -p 1883:1883 eclipse-mosquitto:1.6.15
1634745887: mosquitto version 1.6.15 starting
1634745887: Config loaded from /mosquitto/config/mosquitto.conf.
1634745887: Opening ipv4 listen socket on port 1883.
1634745887: Opening ipv6 listen socket on port 1883.
1634745887: mosquitto version 1.6.15 running
```

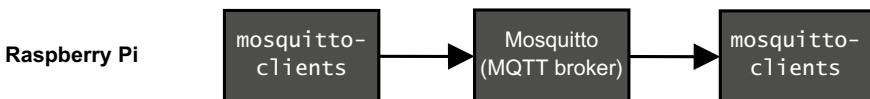


Fig. 5.3 Lab 2, part 1 workflow

It should be noted that once the container is up and running, the terminal can be used only to show the logs from the container, and not anymore used for running other commands.

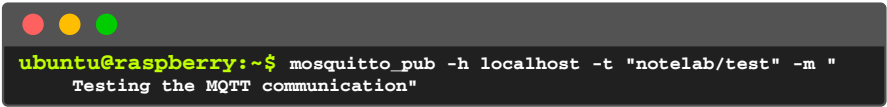
To ensure that MQTT broker is working as expected, we test the publish/subscribe communications and message exchange by using one local publisher and one local subscriber on Raspberry Pi.

Let us open a new terminal and run



```
ubuntu@raspberrypi:~$ mosquitto_sub -h localhost -t "notelab/test"
```

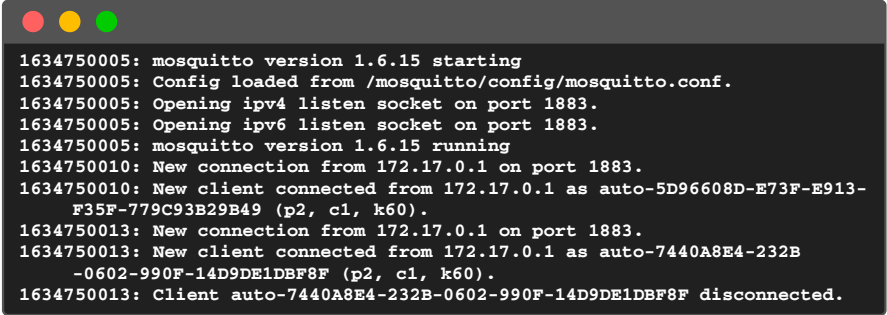
At this time, a subscriber can join the broker using the topic *notelab_test*. Let us now open a new terminal to run a publisher so we can publish a message on that topic:



```
ubuntu@raspberrypi:~$ mosquitto_pub -h localhost -t "notelab/test" -m "
Testing the MQTT communication"
```

In the subscriber terminal, we should now see the message received from the publisher. Otherwise, we need to check again that MQTT broker is up and running properly. Note that we are using *localhost* instead of broker IP address, since the publisher and subscriber are running on the same device!

In case you want to check the output on the terminal for the broker, run the same example but making sure that the Mosquitto broker container is not deployed in a detached mode. It indicates that two MQTT clients are connected: in the output, their randomly generated IDs are different from the ID of the broker, for instance



```
1634750005: mosquitto version 1.6.15 starting
1634750005: Config loaded from /mosquitto/config/mosquitto.conf.
1634750005: Opening ipv4 listen socket on port 1883.
1634750005: Opening ipv6 listen socket on port 1883.
1634750005: mosquitto version 1.6.15 running
1634750010: New connection from 172.17.0.1 on port 1883.
1634750010: New client connected from 172.17.0.1 as auto-5D96608D-E73F-E913-
F35F-779C93B29B49 (p2, c1, k60).
1634750013: New connection from 172.17.0.1 on port 1883.
1634750013: New client connected from 172.17.0.1 as auto-7440A8E4-232B
-0602-990F-14D9DE1DBF8F (p2, c1, k60).
1634750013: Client auto-7440A8E4-232B-0602-990F-14D9DE1DBF8F disconnected.
```

When the task is completed, we terminate all connections [Ctrl+C] and stop and remove the Mosquitto container.

Assignment 1

Answer the following questions:

- (a) What is the difference between publish-subscribe and client-server communication models?
- (b) What is the purpose of *topics* in the publish-subscribe model?
- (c) Name an application that should choose to use publish-subscribe model in favor of client-server model, and explain why?

Add the answers to the lab report.

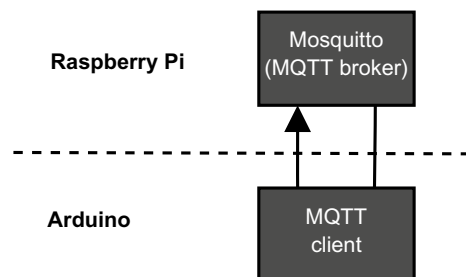
5.4 Part 2: Implementing an MQTT Client in Arduino

In this part of the lab, we only have one task which is to learn how to implement MQTT clients in Arduinos and make them communicate with MQTT broker already running on Raspberry Pi (See Fig. 5.4).

5.4.1 Task 2: Implementing an MQTT Client

Let us first implement a programming sketch of an MQTT client on Arduinos. As previously explained, we divide the code into a few separate files: sketch file `*.ino`, header file `*.h`, and source file `*.cpp`. We start by creating a new folder for the project, for instance, `MqttClient`. In this folder, we create the main sketch file with the same filename, e.g., `MqttClient.ino`. After that, we add new files with the same filename, but with extensions `*.h` and `*.cpp`, e.g., `MqttClient.h` and `MqttClient.cpp`. Let us now copy a library previously developed in Lab 0, Sect. 3.4.2: `WifiClient` library. Recall that this library contains `.cpp` and `.h` files,

Fig. 5.4 Lab 2, part 2 workflow



and we use them here without any modifications. Finally, when opening the main sketch with Arduino IDE, all five files open automatically.

First, we add the following code into `MqtttClient.h` header file:

Source code: [14]

```
1 #include <Arduino.h>
2 #include <MQTT.h>
3
4 class MqtttClient
5 {
6     public:
7         MqtttClient(const char* mqtt_broker_ip, const int
            mqtt_broker_port);
8         MqtttClient(const char* mqtt_broker_ip, const int
            mqtt_broker_port, String *subscribe_topics, const
            int num_subscribe_topics);
9         void connect(const char* client_id);
10        void publish_message(const char *topic, const char
            *msg);
11        void check_connection(const char* client_id);
12        String get_msg();
13        String get_topic();
14        void reset_msg();
15    private:
16        MQTTClient _mqtt_client;
17        String *_subscribe_topics;
18        int _num_subscribe_topics;
19 };
```

Code descriptions: [14]

This header file contains the definitions of the private and public functions and variables required for creating the library, wrapped in a class named `MQTTClient`. First, we include `Arduino.h` header file in order to access standard types, functions, and constants of Arduino language. We also include `MQTT.h` header file for MQTT communication support. We now define two constructors. In the first constructor, we specify IP address of MQTT broker as well as the port number `const char*` variable `mqtt_broker_ip` and `const int` variable `mqtt_broker_port`, respectively. They are called when creating class instance of an MQTT publisher client. For the second constructor, which is MQTT subscriber client, we specify in

addition to the IP address and port number also the related MQTT topics and the number of topics (string array variable `subscribe_topics` and `const int` variable `num_subscribe_topics`, respectively). Next, we specify three functions, which return nothing, `connect()`, `publish_message()`, and `check_connection()`. Functions `connect()` and `check_connection()` both receive `client_id` instance of type `const char *`. Function `publish_message()` receives `topic` and `msg` array instances of type `const char`. The following two defined functions `get_msg()` and `get_topic()` return string value. Function `reset_msg()` also returns nothing. Finally, we define three private variables, `_mqtt_client`, `_subscribe_topics`, and `_num_subscribe_topics`.

In the next step, we focus on the source file `MqtttClient.cpp`. To this end, we use the following source code:

Source code: [15]

```

1 #include "MqtttClient.h"
2 #include <MQTT.h>
3 #include <WiFiNINA.h>
4
5 WiFiClient _wifi_client;
6 String _msg;
7 String _topic;
8 int _num_subscribe_topics;
9
10 MqtttClient::MqtttClient(const char *mqtt_broker_ip, const
    int mqtt_broker_port)
11 {
12     _mqtt_client.begin(mqtt_broker_ip, _wifi_client);
13 }
14
15 MqtttClient::MqtttClient(const char *mqtt_broker_ip, const
    int mqtt_broker_port, String *subscribe_topics, const
    int num_subscribe_topics)
16 {
17     _mqtt_client.begin(mqtt_broker_ip, _wifi_client);
18     _subscribe_topics = subscribe_topics;
19     _num_subscribe_topics = num_subscribe_topics;
20 }
21
22 void callback(String &topic, String &payload)
23 {

```

```
24   Serial.println("Message arrived [" + topic + "]: " +
      payload);
25   _msg = payload;
26   _topic = topic;
27 }
28
29 void MqttClient::connect(const char *client_id)
30 {
31   Serial.print("Connecting to MQTT broker... ");
32   while (!_mqtt_client.connect(client_id, "public",
      "public"))
33   {
34     Serial.print(".");
35     delay(1000);
36   }
37   Serial.println("done!");
38   for (int i = 0; i < _num_subscribe_topics; i++)
39   {
40     _mqtt_client.onMessage(callback);
41     _mqtt_client.subscribe(_subscribe_topics[i]);
42     Serial.println("Subscribed to topic: " +
      _subscribe_topics[i]);
43   }
44 }
45
46 void MqttClient::publish_message(const char *topic, const
      char *msg)
47 {
48   _mqtt_client.publish(topic, msg);
49   Serial.println("Message published [ " + String(topic) +
      " ]: " + String(msg));
50 }
51
52 void MqttClient::check_connection(const char *client_id)
53 {
54   _mqtt_client.loop();
55   delay(10);
56   if (!_mqtt_client.connected())
57     connect(client_id);
58 }
59
60 String MqttClient::get_msg() { return _msg; }
61
62 String MqttClient::get_topic() { return _topic; }
63
64 void MqttClient::reset_msg()
65 {
66   _msg = "";
67   _topic = "";
68 }
```

Code descriptions: [15]

We first include three libraries: `MqttClient.h` header file (definitions), `WiFiNINA.h` library (WiFi support for Arduino Uno WiFi Rev2), and `MQTT.h` library (MQTT communication support). After that, we define instances of class `WiFiClient` (included in `WiFiNINA.h` library). We also define instances `_msg` and `_topic` of Arduino class `String` and an integer instance `_num_subscribe_topics`, where message payload, topic, and number of topics are saved. We define two constructor codes which are executed each time an object of this class is created in the main sketch. For the first constructor, we use function `begin()` from the original `MQTT.h` library to initialize the object using the IP address of MQTT broker (`mqtt_broker_ip`) and `WiFiClient` instances. For the second constructor, we use again the same function `begin()` to initialize the object, and here we save values of the subscribed topics and a number of prospective topics (`subscribe_topics` and `num_subscribe_topics`) in private variables for use in other functions.

After this, we define the following functions:

- `callback()`: This function receives two parameters of type `String`, MQTT topic, and message payload. It prints out the message received and stores the payload and the topic in variables `_msg` and `_topic`, respectively.
- `connect()` (including the name of the class to which it belongs): This function receives the client identifier as a char array. It tries to connect to MQTT broker in a `while` loop statement. Within the loop, we add a delay of 10ms before retrying again. After the client is connected, we add a `for` loop statement in which we call functions from the original `MQTT.h` library, `onMessage`, and `subscribe` for the connected client. In this way, it is going to subscribe to each of the topics and retrieve their corresponding payloads. Finally, it prints the topics to which it subscribes.
- `publish()` (including the name of the class to which it belongs): This function receives topic and a message as a char array. It is used to publish a message to a topic, and it prints out both the message and the topic.
- `check_connection()` (including the name of the class to which it belongs): This function is used to check whether the connection to MQTT broker is up and running. The loop function from the `MQTT` library is used with a small delay of 10 ms. The `if` statement makes sure that in case the client is not connected, new connection is attempted.
- `get_msg` (including the name of the class to which it belongs): This function returns a string with the last message received.

- `get_topic` (including the name of the class to which it belongs): This function returns a string with the topic.
- `reset_msg`: This function resets the message by storing empty strings as the payload and the topic in variables `_msg` and `_topic`, respectively.

We now add the following code to the main sketch file `MqtttClient.ino`:

Source code: [16]

```
1 #include "MqtttClient.h"
2 #include "WifiClient.h"
3
4 char *ssid_wifi = "netw0";
5 char *pass_wifi = "password1";
6
7 const char *mqtt_broker_ip = "RPI_IP";
8 const int mqtt_broker_port = 1883;
9 const char *client_id = "client_id";
10 const int num_subscribe_topics = 1;
11 String subscribe_topics[num_subscribe_topics] =
12     {"topic_test"};
13 const char *publish_topic = "topic_test";
14 const char *publish_message = "message_test";
15 WifiClient wifi_client(ssid_wifi, pass_wifi);
16 MqtttClient mqtt_client(mqtt_broker_ip, mqtt_broker_port,
17     subscribe_topics, num_subscribe_topics);
18
19 void setup()
20 {
21     Serial.begin(9600);
22     wifi_client.connect();
23     mqtt_client.connect(client_id);
24     mqtt_client.publish_message(publish_topic,
25         publish_message);
26 }
27
28 void loop()
29 {
30     mqtt_client.check_connection(client_id);
31 }
```

Code descriptions: [16]

The first line of code includes `MqttClient.h` header file, which also includes the source code of `MqttClient.cpp`. Similarly, `WifiClient.h` header file includes the source code of `WifiClient.cpp`. After this, we define two type char array variables, `ssid_wifi` and `pass_wifi`, and assign them values which correspond to the wireless network configuration in *Lab 0* (recall that we use `ssid_netw1` and `password1` values we define ourselves). We define two const char array variables `mqtt_broker_ip` and `client_id`, to which we assign values to IP address of Raspberry Pi device (e.g., `192.168.1.3`) and random value for client identification (e.g., `"client_id"`). We also define two const int variables `mqtt_broker_port` and `num_subscribe_topics` to which we assign value of MQTT port - 1883 and a number of topics (e.g., 1), respectively. Then we define a string array variable for storing topics to which the client can subscribe to. For instance, a client can subscribe to the topic with the name `"topic_test"`, with the stored number limited to the number defined in variable `num_subscribe_topics`. Next, we define two char array variables `publish_topic` and `publish_message` to which we assign values of test MQTT topic to which the client publishes to (e.g., `"topic_test"`) and the published message (e.g., `"message_test"`), respectively.

Finally, in order to use the functions defined in `MQTTClient` and `WifiClient` classes, we need to create their class objects, `mqtt_client` and `wifi_client`, respectively. To `wifi_client` we pass the network name and password variables as its arguments, which are passed to the constructor. In the same way, to `mqtt_client` we pass MQTT broker IP address and port number, prospective subscribing topics, and their specified number variables as arguments (this constructor corresponds to using the MQTT client as a subscriber). The sketch functions play the following roles:

- `setup()`: In this function, we open serial communication at a baud rate of 9600 *bps*, and we trigger `connect()` functions for both `mqtt_client` and `wifi_client` defined class objects. We also trigger `publish_message()` function.
- `loop()`: In this function, we trigger `check_connection()` function for `mqtt_client` class object to check if the connection to the broker is established.

Now, proceed to *Verify* to check the code is without errors and then *Upload* the code in order to flash it on the Arduino. For more details about this process, please, go back to Lab 0, Sect. 3.4.1.

We can now use *Serial Monitor* to check the output. Make sure that Mosquitto is up and running on Raspberry Pi. If everything goes ok, the output should show MQTT client connecting to the broker, publishing a message to the topic and receiving the published message:

```
Connecting to WPA SSID [netw0]... done, IP: 192.168.1.5  
Connecting to MQTT broker... done!  
Subscribed to topic: topic_test  
Message published [ topic_test ]: message_test  
Message arrived [topic_test]: message_test
```

Assignment 2

Complete the following assignment:

Create two different sketches, one that works only as publisher and one only as subscriber. The publisher should be sending messages continuously to a certain topic with a rate of one message per second. Every five consecutive messages, the content of the message should be modified. The subscriber only prints out the message, when the content of the received message was modified. Save the outputs and the code to the lab report.

Save the outputs and the code to the lab report.

5.5 Part 3: Sensing and Actuating over MQTT

In the following two tasks, the objective is to learn how to use MQTT to sending and receiving sensing data on various devices. To this end, we now connect two Arduinos to Raspberry Pi. One Arduino interfaces a sensor and sends data using an MQTT publisher client. The other one interfaces an actuator and receives data using an MQTT subscriber client. As shown previously, MQTT broker runs on a Raspberry Pi. The testing workflow is illustrated in Fig. 5.5.

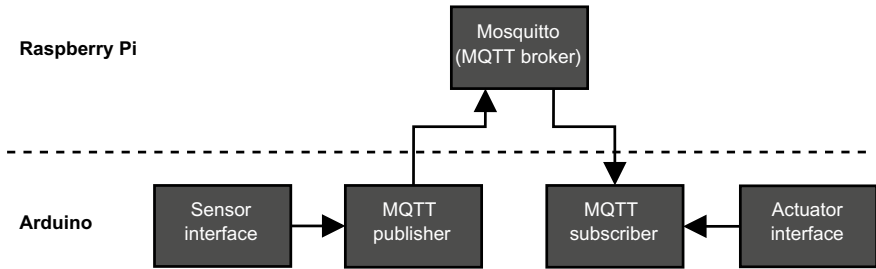


Fig. 5.5 Lab 2, part 3 workflow

5.5.1 Task 3: Sending DHT11 Sensor Data over MQTT

In this task, we send DHT11 sensor data values using the MQTT publisher. Revisit *Lab 1* to be reminded of details about DHT11 sensor. DHT11 sensor is connected to Arduino Uno WiFi Rev2 board as shown in Fig. 5.6.

Start by creating a new folder for the project (e.g., `publisher_DHT11`), and within create a main sketch file with the same filename (e.g., `publisher_DHT11.ino`). All other libraries necessary for this sketch code have been already developed, and we just copy them to this working folder. This includes `WiFiClient` library (.cpp and .h files) and `MqttClient` library (.cpp and .h files). By copying these libraries, we can reuse the functions without any modifications. As a result, when opening the main sketch with Arduino IDE, all five files automatically open.

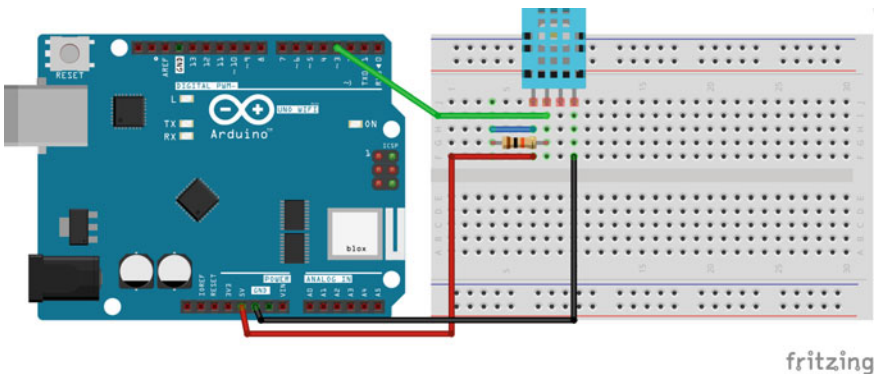


Fig. 5.6 Connecting DHT11 sensor to the Arduino

Add the following code into the main sketch file `publisher_DHT11.ino`:

Source code: [17]

```
1 #include "MqttClient.h"
2 #include "WifiClient.h"
3 #include "DHT.h" // DHT module library
4 #include <ArduinoJson.h>
5
6 char *ssid_wifi = "netw0";
7 char *pass_wifi = "password1";
8
9 const char *mqtt_broker_ip = "RPi_IP";
10 const int mqtt_broker_port = 1883;
11 const char *client_id = "publisher_DHT11";
12 const char *publish_topic = "temperature_topic";
13 WifiClient wifi_client(ssid_wifi, pass_wifi);
14 MqttClient mqtt_client(mqtt_broker_ip, mqtt_broker_port);
15
16 #define DHTPIN 3 // DHT11's data pin is connected to
    board's digital pin 3
17
18 #define DHTTYPE DHT11 // DHT type
19 DHT dht(DHTPIN, DHT11); // instantiate DHT object
20
21 void setup() {
22     Serial.begin(9600);
23     wifi_client.connect();
24     mqtt_client.connect(client_id);
25     dht.begin(); // initialize the sensor
26 }
27
28 void loop(){
29     mqtt_client.check_connection(client_id);
30     float humidity = dht.readHumidity();
31     if (!isnan(humidity)) {
32         String humidity_str = String(humidity); //convert
            float to string
33         Serial.print("Humidity: " + humidity_str + " %\n");
34     } else {
35         Serial.println("Humidity reading from DHT11 sensor
            failed!");
36     }
37     float temperature = dht.readTemperature();
38     if (!isnan(temperature))
39     {
40         String temperature_str = String(temperature);
            //convert float to string
```

```

41     Serial.print("Temperature: " + temperature_str + "
        *C\n");
42 } else {
43     Serial.println("Temperature reading from DHT11 sensor
        failed!");
44 }
45 delay(1000);
46
47 DynamicJsonDocument doc(1024);
48
49 doc["id"] = "publisher_DHT11";
50 doc["temperature"] = temperature;
51 doc["humidity"] = humidity;
52 // Generate the prettified JSON and send it to the
    Serial port
53 // serializeJsonPretty(doc, Serial);
54
55 String json_doc;
56 serializeJson(doc, json_doc);
57 char buf[500];
58 json_doc.toCharArray(buf, 500);
59 mqtt_client.publish_message(publish_topic, buf);
60 delay(5000);
61 }

```

Code descriptions: [17]

The lines of code include `MqttClient.h` and `WifiClient.h` header files, which also include the corresponding source codes `MqttClient.cpp` and `WifiClient.cpp`, respectively. Then we include `DHT.h` header file for definitions and the library for DHT sensor. When publishing messages, we use in this task JSON structures instead of the plain text. To this end, we import `ArduinoJson.h` library for processing JSON objects. After this, we define two type char array variables, `ssid` and `pass`, and assign them values which correspond to our LAN (e.g., `ssid netw1` and password `password1`). We define two const char variables `mqtt_broker_ip` and `client_id`, to which we assign values to actual IP address of Raspberry Pi device (e.g., `192.168.1.3`) and random value for client identification (e.g., `"publisher_DHT11"`). As students work with two Arduinos, they need to assign identifiers. We also define `mqtt_broker_port` to which we assign value of MQTT port, 1883. For MQTT communication, we also define a char array variable `mqtt_topic` to which we assign a value of MQTT topic to which the client is going to publish the messages to (`"temperature_topic"`). In order to use the functions defined in `MQTTClient` and

WifiClient classes, we create objects of these classes, `mqtt_client` and `wifi_client`, respectively. To `wifi_client` we pass the network name and password variables as its arguments, which are going to be passed to the constructor. In the same way, to `mqtt_client`, we pass MQTT broker IP address and port number (since this constructor is used for a publisher client). Next, we define the type of DHT sensor and the chosen data pin on the sensor (e.g., digital pin 3). Finally, we initialize a DHT object passing its DHT type and data pin as arguments. We then perform the following operations in various functions:

- `setup()`: In this function, we open serial communication at a baud rate of 9600 *bps*, and we trigger `connect()` functions for both `mqtt_client` and `wifi_client` defined class objects. Here we also initialize the sensor by calling function `begin()` from the imported DHT sensor library (for setting up DHT11 sensor).
- `loop()`: In this function, we trigger `check_connection()` function for `mqtt_client` class object to check if connection to the broker is established. Then trigger `readHumidity()` and `readTemperature()` functions from the imported DHT sensor library and save the returned values into float variables. We then convert these float values into a string type and print using `Serial.print()` function. As we want to publish temperature and humidity data in JSON format, we create a `DynamicJsonDocument` and allocate 1024 bytes of memory for the messages. Then, we prepare the fields with `doc["id"]`, `doc["temperature"]`, and `doc["humidity"]`. We then serialize the document into a string value `json_doc`, define a buffer of length 500, and use Arduino function `toCharArray()` to convert `json_doc` string value to a char array. Finally, we publish the message to the previously defined topic, and we add a delay of 5 s prior to sending the subsequent message.

Now, proceed to *Verify* to check the code is without errors and then *Upload* the code in order to flash it on the Arduino. For more details about this process, please, go back to Lab 0, Sect. 3.4.1.

We can now use *Serial Monitor* to check the output. Make sure that Mosquitto is up and running on Raspberry Pi. If everything goes without errors, the output should look like this:

```
Connecting to WPA SSID [netw1]... done, IP: 192.168.1.5
Connecting to MQTT broker... done!
Humidity: 40.00 %
Temperature: 29.00 *C
Message published [ temperature_topic ]: {"id": "
    publisher_DHT11", "temperature":29, "humidity":40}
Humidity: 40.00 %
Temperature: 29.00 *C
Message published [ temperature_topic ]: {"id": "
    publisher_DHT11", "temperature":29, "humidity":40}
```

Assignment 3

Complete the following assignment:

Modify the main sketch to send the temperature and humidity values only when there is a change in detected values (hint: use a global variable to store the previous values).

Save the code to the lab report.

5.5.2 Task 4: Activating an External LED and LCD Display over MQTT

In this task, the objective is to connect the previously implemented MQTT publisher to MQTT client which activates an external LED and display temperature values on an LCD display. As in the previous case, MQTT broker runs on a Raspberry Pi. Connect the second Arduino Uno WiFi Rev2 board to the red LED and LCD as shown in Fig. 5.7.

Revisit *Lab 1* to recall the information about the pin connection specifics to external LEDs and LCD display.

Let us start by creating a new folder for the project (e.g., `subscriber_temperature`), and in this folder, let us create the main sketch file with the same filename (e.g., `subscriber_temperature.ino`). All other libraries necessary for this sketch code to work have been already developed, and all we need to do now is just to copy all of them to the folder. This includes `WifiClient` library (.cpp and .h files) and `MqttClient` library (.cpp and .h files). By copying these libraries, we can here reuse all of their functions,

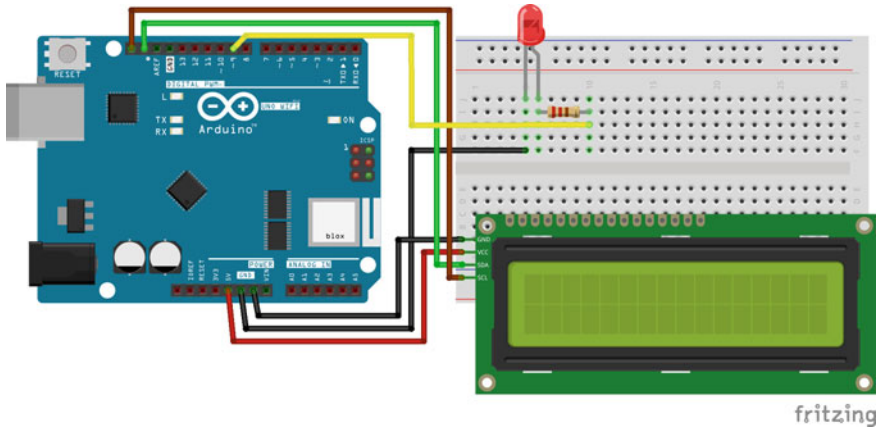


Fig. 5.7 Connecting external LEDs and LCD display to Arduino Uno WiFi Rev2

without any modifications. As a result, when opening the main sketch with Arduino IDE, all five files automatically open.

Let us now add the following code to the main sketch file `subscriber_temperature.ino`:

Source code: [18]

```

1 #include "MqttClient.h"
2 #include "WifiClient.h"
3 #include <ArduinoJson.h>
4 #include <Wire.h>
5 #include <LiquidCrystal_I2C.h>
6
7 #define LED 9 // external LED is connected to board's
   digital pin 9
8
9 char *ssid_wifi = "netw0";
10 char *pass_wifi = "password1";
11
12 const char *mqtt_broker_ip = "RPi_IP";
13 const int mqtt_broker_port = 1883;
14 const char *client_id = "subscriber_DHT11";
15 const int num_subscribe_topics = 1;
16 String subscribe_topics[num_subscribe_topics] =
   {"temperature_topic"};
17 WifiClient wifi_client(ssid_wifi, pass_wifi);
18 MqttClient mqtt_client(mqtt_broker_ip, mqtt_broker_port,
   subscribe_topics, num_subscribe_topics);

```

```
19
20 float temperature = 0;
21 float humidity = 0;
22 float blink_delay = 100;
23 LiquidCrystal_I2C lcd(0x27,16,2);
24
25 void setup(){
26     Serial.begin(9600);
27     wifi_client.connect();
28     mqtt_client.connect(client_id);
29     pinMode(LED, OUTPUT); // initialize LED as an output
30     lcd.init();
31     lcd.backlight();
32 }
33
34 void loop(){
35     mqtt_client.check_connection(client_id);
36     String msg = mqtt_client.get_msg();
37     DynamicJsonDocument doc(1024);
38     deserializeJson(doc, msg);
39     temperature = doc["temperature"];
40     if (temperature >= 10.0 && temperature < 20.0){
41         blink_delay = 2000;
42     }
43     else if (temperature >= 20.0 && temperature < 30.0 ){
44         blink_delay = 500;
45     }
46     else{
47         blink_delay = 1000;
48     }
49     digitalWrite(LED, HIGH); // turn the LED on (HIGH is the
        voltage level)
50     delay(blink_delay); // wait
51     digitalWrite(LED, LOW); // turn the LED off by making
        the voltage LOW
52     delay(blink_delay); // wait
53     lcd.setCursor(0, 0);
54     lcd.print("Temp. ");
55     String temperature_str = String(temperature, 1);
56     lcd.print(temperature_str);
57     lcd.print((char) 223); // celsius char
58 }
```

Code descriptions: [18]

We first include `MqttClient.h` and `WifiClient.h` header files, which also include their source codes, `MqttClient.cpp` and `WifiClient.cpp`, respectively. We also include `ArduinoJson.h` header file for the definitions, the library for processing JSON objects, and `LiquidCrystal.h` header file to allow an Arduino board to control liquid crystal displays. We start by initializing the output of the Arduino that activates LED, in this case digital pin 9. After this, we define variables `ssid`, `pass`, `mqtt_broker_ip`, and `mqtt_broker_port` similarly to the previous task Sect. 5.5.1. We assign a unique identifier to char array variable `client_id` (e.g., `"subscriber_DHT11"`). The identifier has to be unique so we can distinguish between two Arduinos. We also define a const int variable `num_subscribe_topics` to which we assign as a value the number of prospective topics (e.g., 1). Then we define a string array variable `subscribe_topics` for storing topics to which the client subscribes (limited to the number defined in the variable `num_subscribe_topics`). Make sure that the value assigned to variable `subscribe_topics` is the same as for DHT11 publisher (e.g., `"temperature_topic"`).

In order to use functions defined in `MQTTClient` and `WifiClient` classes, we need to create their class objects, `mqtt_client` and `wifi_client`, respectively. To `wifi_client`, we pass the network name and password variables as its arguments, which are passed to the constructor. In the same way, to `mqtt_client` we pass MQTT broker IP address and port number, subscribing topics, and their specified number variables as arguments (since this constructor is used for a subscriber client). Next, define three float variables `temperature`, `humidity`, and `blinking_delay`, initializing all of them with value 0. Finally, we define LCD variables, the number of columns and rows (`lcd_cols` and `lcd_line`), contrast value (`lcd_contrast`), contrast control pin `lcd_pin`, and use function `lcd()` to initialize the library with the used interface pins (digital pins 12, 11, 5, 4, 3, and 2).

The following sketch functions are used:

- `setup()`: This function opens serial communication at baud rate of 9600 *bps* and triggers `connect()` functions for both class objects defined: `mqtt_client` and `wifi_client`. MQTT client's `connect()` function includes invoking of `onMessage` and `subscribe` functions after the client is connected, which in turn subscribes to the specified topics and retrieves their corresponding payloads. Next, we use `analogWrite()` function to simulate an analog signal arriving at the contrast control pin, with the defined contrast. Here also we initialize LCD display with function `begin()`, as well as the output of the Arduino that activates the LED.
- `loop()`: This function triggers `check_connection()` function for `mqtt_client` class object to check if the connection to the broker is

established. Then it triggers `get_msg()` to initialize a string variable `message`. We create `DynamicJsonDocument` object allocating 1024 bytes of memory and then deserialize the buffer into the json object `doc`. Now, we access the temperature value from the dictionary and save and print it into float variable `temperature`. Next, the function adds an `if-else` statement to make the LED blink with different intervals depending on the temperature measured. Finally, using LCD's functions `setCursor()` and `print()`, it starts displaying the temperature values from the first row on.

Now, proceed to *Verify* to check the code is without errors and then *Upload* the code in order to flash it on the Arduino. For more details about this process, please, go back to Lab 0, Sect. 3.4.1.

We can now use *Serial Monitor* to check the output. Make sure that Mosquitto is up and running on Raspberry Pi and that the Arduino interfacing DHT11 sensor is publishing messages. If everything goes without errors, the output should show MQTT client connecting to the broker, subscribing to a topic, and receiving a message published on the said topic:

```
Connecting to WPA SSID [netw1]... done, IP: 192.168.1.12
Connecting to MQTT broker... done!
Subscribed to topic: temperature_topic
Message arrived [ temperature_topic ]: {"id":"
    publisher_DHT11","temperature":29,"humidity":40}
Message arrived [ temperature_topic ]: {"id":"
    publisher_DHT11","temperature":29,"humidity":40}
```

Assignment 4

Complete the following assignment:

Modify the main sketch instead of using the last received temperature value, the sketch uses the average temperature value calculated based on the last 50 previous values to make LED blink accordingly (hint: use a global variable to store the previous values).

Save the code to the lab report.

References

Libraries

1. Arduino JSON library: <https://github.com/bblanchon/ArduinoJson>
2. Arduino Wi-FiNINA library source: <https://github.com/arduino-libraries/WiFiNINA>
3. Arduino MQTT library: <https://github.com/256dpi/arduino-mqtt>
4. Wi-Fi client Arduino header file: <https://github.com/tubskns/notelab-code/blob/master/arduino/WifiClient/WifiClient.h>
5. Wi-Fi client source file: <https://github.com/tubskns/notelab-code/blob/master/arduino/WifiClient/WifiClient.cpp>
6. Source code LiquidCrystal_I2C library https://github.com/johnrickman/LiquidCrystal_I2C
7. Source code DHT11 library: <https://github.com/adafruit/DHT-sensor-library>

Tools

8. Docker Engine installation on Ubuntu: <https://docs.docker.com/engine/install/ubuntu/>
9. Python: <https://www.python.org/downloads/>
10. OpenSSH Server: <https://ubuntu.com/server/docs/service-openssh>
11. How to install Arduino IDE: <https://www.arduino.cc/en/Guide/Linux>
12. mosquito-clients (v1.6.9-1): <https://packages.ubuntu.com/focal/mosquitto-clients>
13. Eclipse Mosquitto (Docker): https://hub.docker.com/_/eclipse-mosquitto

Other Resources

14. MQTT client Arduino header file: <https://github.com/tubskns/notelab-code/blob/master/arduino/MqttClient/MqttClient.h>
15. MQTT client Arduino source file: <https://github.com/tubskns/notelab-code/blob/master/arduino/MqttClient/MqttClient.cpp>
16. MQTT client Arduino sketch: <https://github.com/tubskns/notelab-code/blob/master/arduino/MqttClient/MqttClient.ino>
17. MQTT publisher for DHT11 Arduino sketch: https://github.com/tubskns/notelab-code/blob/master/arduino/sensors/temperature/publisher_DHT11/publisher_DHT11.ino
18. MQTT publisher for DHT11 Arduino sketch: https://github.com/tubskns/notelab-code/blob/master/arduino/actuators/subscriber_temperature/subscriber_temperature.ino

Chapter 6

Lab 3—Connecting Edge and Cloud Tools with AMQP



6.1 Prerequisites

Hardware

- Raspberry Pi 4 Model B
- Server
- Workstation
- Router

Software

- Raspberry Pi 4 Model B
 - Docker v20.10.8 [2]
 - Python v3.9 [3]
 - OpenSSH v8.4 [4]
- Server
 - curl v7.81.0 [5]
 - K3s v1.20.9+k3s1 [6]
 - OpenSSH v8.4 [4]
- Workstation
 - OpenSSH v8.4 [4]
 - amqp-tools v0.10.0-1 [7].

6.2 Setup

The main objective of *Lab 3* is to learn how typical edge computing devices and cloud computing tools and system can connect with AMQP protocol. We learn how to establish communication between Raspberry Pi and the server in NoTE Lab. The setup for this lab is shown in Fig. 6.1 and includes server, Raspberry Pi, WiFi router, and workstation. In *Lab 3*, we learn how to deploy RabbitMQ tool, which includes an AMQP broker, on the server. We also learn how to implement an AMQP client on Raspberry Pi in Python using *pika* library.

This short lab is divided into two parts. The first part of *Lab 3* is dedicated to deploying and testing RabbitMQ which implements an AMQP broker (as shown in Fig. 6.2). For testing purposes, we use *amqp-tools*, a tool that allows us to publish and subscribe to an AMQP broker using CLI. In the second part, an AMQP client is implemented in Python and used to publish and subscribe to AMQP broker. AMQP client, which normally runs on Raspberry Pi, will be extensively used in the subsequent labs by various modules. For instance, Actuator controller uses AMQP client as publisher, and Sensor data collector uses it as subscriber. A detailed overview of the libraries used in this lab is listed as follows:

- AMQP broker
 - RabbitMQ helm chart v10.1.14 [9]

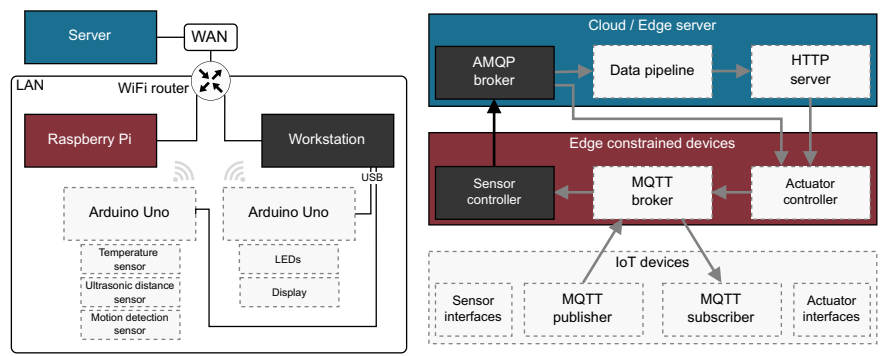
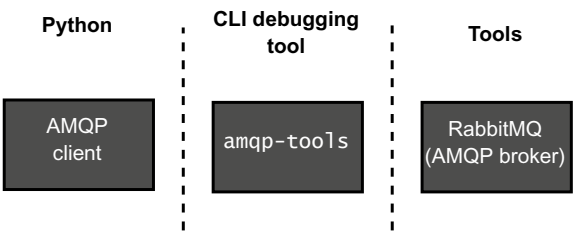


Fig. 6.1 Lab 3 setup

Fig. 6.2 Lab 3 components



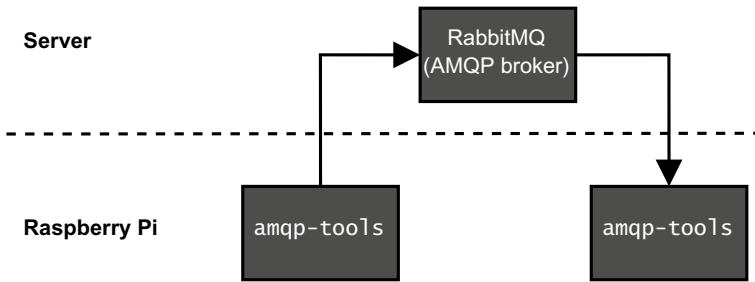


Fig. 6.3 Lab 3, part 1 workflow

- Sensor data collector and Actuator controller
 - Pika library v1.2.1 [1].

6.3 Part 1: Deploy and Test RabbitMQ

We define two tasks here, one to deploy and one to test RabbitMQ ([8]). Specifically, an instance of RabbitMQ ([8]) is installed on the Kubernetes cluster (our “cloud”). We learn how to install an AMQP broker on the server and to test it from the workstation using the `amqp` client tool (See Fig. 6.3).

6.3.1 Task 1: Deploying RabbitMQ

Let us first ensure that the server is reachable through its public IP address or domain name from Raspberry Pi used in this setup, as well as from the workstation machine using `ping` command. Recall that both Raspberry Pi and workstation are connected to the router. Then, we use `ssh` to remotely access the server.

Tips

In case we are unable to access the server, see Appendix B.3 for checking the status of SSH service.

Then we check whether `k3s` is properly installed:

```
ubuntu@server:~$ kubectl cluster-info

Kubernetes control plane is running at https://127.0.0.1:6443
CoreDNS is running at https://127.0.0.1:6443/api/v1/namespaces/
  kube-system/services/kube-dns:dns/proxy
Metrics-server is running at https://127.0.0.1:6443/api/v1/
  namespaces/kube-system/services/https:metrics-server:/proxy
```

It should be noted here that we are running `kubectl` command without `sudo`, which results in an error by default. Let us hence add `sudo` to all `kubectl` commands or alternatively find out in Appendix C.1 the instructions on how to give permissions to the non-root user account. As the next step, we learn how to deploy RabbitMQ. We first add the repository to `helm` and then run the installation command as follows:

```
ubuntu@server:~$ helm repo add bitnami https://charts.bitnami.com/bitnami
ubuntu@server:~$ helm repo update

Hang tight while we grab the latest from your chart repositories
...
...Successfully got an update from the "crowdsec" chart repository
...Successfully got an update from the "elastic" chart repository
...Successfully got an update from the "openfaas" chart repository
...Successfully got an update from the "groundhog2k" chart
  repository
...Successfully got an update from the "bitnami" chart repository
Update Complete. *Happy Helming!*
```

```
ubuntu@server:~$ helm install rabbitmq bitnami/rabbitmq \
  --version 10.1.14 \
  --set replicaCount=1 \
  --set auth.username=user,auth.password=password \
  --set service.type=NodePort,service.nodePorts.amqp=30672,
  service.nodePorts.manager=31672

NAME: rabbitmq
LAST DEPLOYED: Sat Jul  2 11:38:29 2022
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
```

Tips

In case this installation method fails because of either the container image or the repository are not available anymore, please refer to Appendix C.6 for an alternative installation method.

It should be noted that the above command manually specifies the authentication credentials (i.e., username as `user` and password as `password`), which is not recommended in production environments due to security concerns. If the installation was successful, we wait until `rabbitmq` is ready. We can check the pod status with

```
ubuntu@server:~$ kubectl get pods --all-namespaces
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	metrics-server-...	1/1	Running	0	2m51s
kube-system	local-path-prov...	1/1	Running	0	2m51s
kube-system	coredns-64888...	1/1	Running	0	2m51s
kube-system	helm-install-tra...	0/1	Completed	0	2m51s
kube-system	svclb-traefik-4...	2/2	Running	0	2m34s
kube-system	traefik-6f9cbd...	1/1	Running	0	2m34s
default	rabbitmq-0	1/1	Running	0	2m31s

With this command, we do not only see `rabbitmq` status, but also the status for the rest of the required `k3s` containers. Once `rabbitmq` pod status is *Running*, we use port 30672 for AMQP communication. Port 31672 is used to access RabbitMQ management user interface (UI) or dashboard. After that, we can check from any browser installed on the workstation machine that `rabbitmq` management UI is available at `http://SERVER_IP:31672/` (use previously specified authentication credentials to login).

RabbitMQ dashboard provides an initial tab *Overview* as it can be seen in Fig. 6.4. In this tab, it is possible to analyze two charts (currently empty), *Queued messages* and *Message rates*. Chart *Queued messages* represents the total number of queued messages for all declared queues, differentiating between the number of messages available to be delivered (*Ready*) and unacknowledged messages (*Unacked*). Chart *Message rates* represents the rate at which the messages are handled, differentiating between multiple options, such as the rate at which messages are entering RabbitMQ (*Publish*) or the rate that the messages are being confirmed (*Publish confirm*). Information about all potential message statuses and different rate metrics can be found by pressing "?" option next to the charts. For more details on RabbitMQ dashboard, the best is to refer to the official documentation in [10].

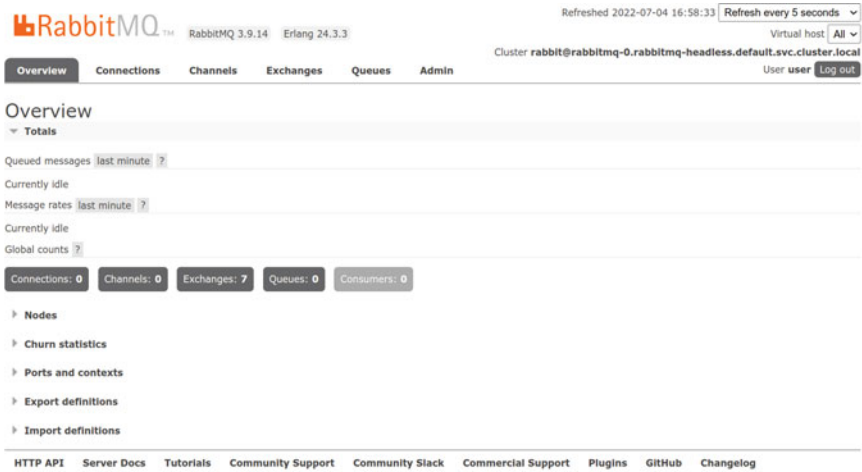


Fig. 6.4 RabbitMQ dashboard

Assignment 1

Answer the following questions:

- Why would we consider installing RabbitMQ over K3s deployment?
- Do we need K3s to deploy RabbitMQ? If not, what are the alternative installation options for RabbitMQ?
- Search for other available open-source messaging system tools based on AMQP protocol. Discuss the pros and cons as compared to RabbitMQ.

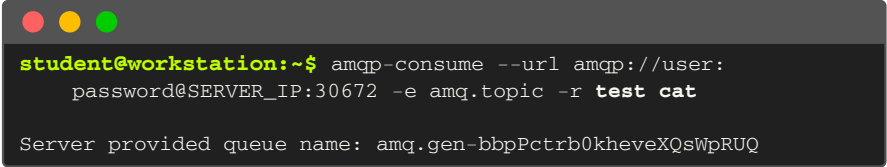
Add the answers to the lab report.

6.3.2 Task 2: Testing RabbitMQ

In this task, we use amqp client tool to test AMQP broker we already installed in the previous task. Since RabbitMQ uses AMQP 0-9-1 version, the messages are never published directly to the queue. Instead, AMQP producer publishes a message to an exchange (AMQP intermediate entity). Exchanges then decide how to further route the message to one or multiple queues, using bindings which are associations

between an exchange and a queue. Routing keys are then used by the exchange to decide how to route an AMQP message to specific queues. The manner in which messages are routed depends on the exchange type. For more details, please refer to [11].

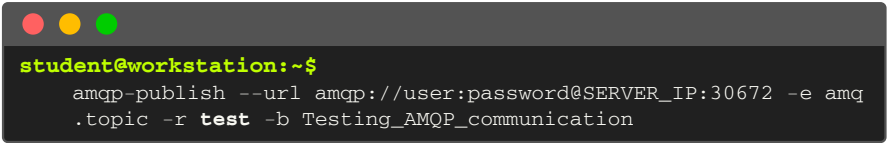
In NoTE Lab, we use *topic* exchange type (`amq.topic`). This option, along with other exchange topics can be found in RabbitMQ dashboard under *Exchanges* tab. Queues can also be found under *Exchanges* tab. To test AMQP message exchange, we use `amqp-tools` from workstation, which is a CLI-based tool used to create AMQP publisher and consumers. First, we run a AMQP subscriber on a workstation terminal by specifying server IP address, port, user credentials, exchange type, and routing key:



```
student@workstation:~$ amqp-consume --url amqp://user:
password@SERVER_IP:30672 -e amq.topic -r test cat

Server provided queue name: amq.gen-bbpPctrb0kheveXQsWpRUQ
```

Since the queue name was not declared, RabbitMQ assigns a unique name to the queue instead (e.g., `amq.gen-bbpPctrb0kheveXQsWpRUQ`). This queue can now also be found in RabbitMQ dashboard, under *Queues* tab. Next, we proceed to publish a message from another terminal in the workstation, by specifying the same parameters as before:



```
student@workstation:~$ amqp-publish --url amqp://user:password@SERVER_IP:30672 -e amq
.topic -r test -b Testing_AMQP_communication
```

Another way we can add queues is to create them in *Queues* tab of RabbitMQ dashboard, using the option *Add a new queue*. Once a queue is created, open its tab in the dashboard and find the option *Bindings*. There simply choose *Add binding to this queue*, specifying `amq.topic` as the exchange and an arbitrary name for the routing key, before pressing the option *Bind*.

Check that the subscriber successfully receives the published message. Otherwise, check that AMQP broker is up and running properly.

Assignment 2

Answer the questions and complete the following assignment:

- (a) What is the difference between exchanges and queues in AMQP?
- (b) What are routing keys and bindings in AMQP?
- (c) Name and shortly describe the different exchange types supported in AMQP version 0-9-1.

Add the answers and outputs to the lab report.

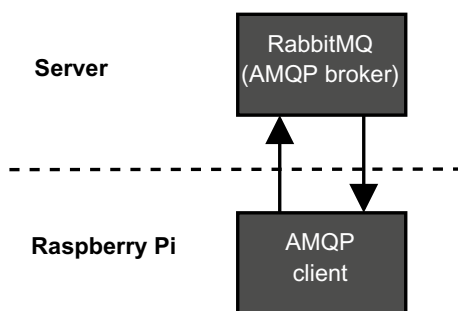
6.4 Part 2: Implementing an AMQP Client in Python

In this part of Lab 3, we learn how to develop and use an AMQP client in Python so it can work as either publisher or subscriber. The client is tested using RabbitMQ instance running on the server (See Fig. 6.5). To start with this task, we need to make sure that RabbitMQ is up and running, as explained previously.

6.4.1 Task 3: Implementing an AMQP Client

Recall that when programming in Python in NoTE Lab, we do not go into details of programming itself; instead, we create a Python script file using a text editor and use it for the complete tasks. In this task, we first create a folder (*amqp_client_rpi*) on Raspberry Pi where the script should be saved. Let us open the text editor (use command `nano`) and save a Python script under *amqp_client.py*. Add the following code, or Python script, to the created file:

Fig. 6.5 Lab 3, part 2 workflow



Source code: [12]

```
1 import pika
2 import logging
3
4
5 def connect_to_broker(ip, port, user, passwd):
6     credentials = pika.PlainCredentials(user, passwd)
7     parameters = pika.ConnectionParameters(ip, port, "/",
8         credentials)
9     connection = pika.BlockingConnection(parameters)
10    channel = connection.channel()
11    logging.info("AMQP - Connected to broker: " + str(ip) +
12        ":" + str(port))
13    return channel
14
15 def create_queue(channel, exchange, routing_key, queue,
16     durable=False):
17     channel.queue_declare(queue=queue, durable=durable)
18     channel.queue_bind(exchange=exchange, queue=queue,
19         routing_key=routing_key)
20     logging.debug("AMQP - Queue declare and bind: " + queue)
21
22
23 def on_message(channel, method, properties, msg):
24     logging.debug("AMQP - Message received: " +
25         msg.decode("utf-8"))
26
27
28 def subscribe(channel, queue, on_message_callback):
29     channel.basic_consume(queue,
30         on_message_callback=on_message_callback)
31     logging.info("AMQP - Subscribed to: " + queue)
32
33
34 if __name__ == "__main__":
35     logging.basicConfig(
36         format="%(asctime)s | %(name)s | %(levelname)s |
37             %(message)s",
38         level=logging.DEBUG,
39     )
40     ip = "SERVER_IP"
41     port = 30672
42     user = "user"
43     passwd = "password"
44     queue = "queue_test"
45     exchange = "amq.topic"
46     routing_key = "routing_key_test"
47     message = "message_test"
```

```
42     channel = connect_to_broker(ip, port, user, passwd)
43     create_queue(channel, exchange, routing_key, queue)
44     channel.basic_publish(exchange=exchange,
45                           routing_key=routing_key, body=message)
45     subscribe(channel, queue, on_message)
46     channel.start_consuming()
```

Code descriptions: [12]

The script starts by importing `pika` library, a pure-Python implementation of AMQP 0-9-1 protocol, which has been previously installed as one of the software prerequisites. It also imports `logging` library, to be able to easily define the format of the logging messages. Then, we define the following functions:

- `connect_to_broker()`: This function is used to connect to AMQP broker. It accepts four arguments: `ip` address of the broker, `port` of the broker, username `user`, and password `passwd` for client authentication. Next, we use a `pika` method, called `PlainCredentials()`, to properly create credentials from the specified username and password. Credentials are then passed as parameter to `ConnectionParameters` along with broker IP and port. Next, we create a new connection instance of type `BlockingConnection()` passing the previously created parameters. Then a channel is created by specifying the type of connection. With `logging.info()` function, we provide a debugging output of connection status. Finally, we return the channel instance.
- `create_queue()`: This function defines the queue properties. It accepts four arguments: `channel` to be used to publish the message, `exchange` to publish to, `routing_key` to bind on, and the name of the queue. This is done with a method `queue_declare()`, which declares a queue with a specific name and specifies if the type of queue is `durable`, so whether it is persistent after broker server restart, or not. This is followed by binding the queue to an exchange, through method `queue_bind()`, which binds these specified queues and the exchange, with a specific routing key name. Lastly, we provide a more detailed debugging output on queue properties with `logging.debug()` function.
- `on_message()`: This function is triggered each time the consumer receives a message using four arguments: `channel`, `method`, `properties`, and `msg` (AMQP message body). In this function, we sim-

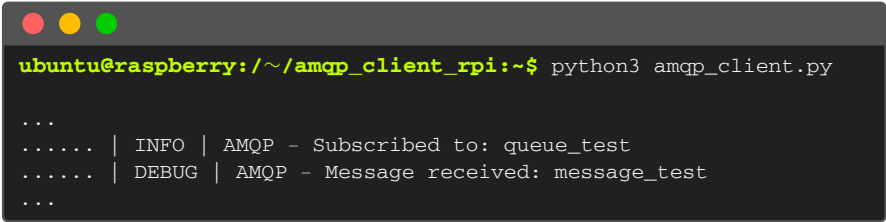
ply print the received payload to the terminal, using `logging.debug()` function, prior to conversion of the payload bytes into a string.

- `subscribe()`: This function is used to consume messages from an AMQP queue. It accepts three arguments: `channel` to subscribe, `queue` from where to consume messages, and `on_message_callback` to access the messages published to the consumer. Next, from `channel` we use `basic_consume()` to actually subscribe to `queue`. Here, we specify the function to be triggered when a new message is received with `on_message_callback` function. With `logging.info()` function, we provide a debugging output for the consumer of the specified queue.

In order to test the previously defined functions, we add `if __name__ == "__main__":` statement block, which is only executed if the script runs as a main program. Here, we define variables for connecting to RabbitMQ server: `ip` and `port` of the broker (30672), credentials for client authentication `user` and `password` ("user" and "password"), `queue`, `exchange`, `routing_key` of type string, and `message` to be sent. Then, we connect to AMQP broker with `connect_to_broker()`, create a queue with `create_queue()`, publish a message to an exchange with `basic_publish()` function from the original `pika` library, and subscribe to the broker with `subscribe()` function. Finally, we trigger `start_consuming()` messages. If everything goes as expected, the message is shown in the output terminal.

In order to test AMQP communication, the queue declared in AMQP publisher (e.g., `test_queue`) has to be bound to our exchange type `amq.topic` with a specific message routing key. This is used by the exchange to decide how to route the incoming messages to corresponding queues. To check this, we open created queue in RabbitMQ dashboard and check *Binding* tab, as illustrated in Fig. 6.6. Note that in case we have not used `queue_bind()` method when publishing messages, we can add it afterward in RabbitMQ dashboard.

Let us now run the script from Raspberry Pi terminal as follows (the output shows only relevant messages, while the script generates many more of them):



```
ubuntu@raspberrypi: ~/amqp_client_rpi:~$ python3 amqp_client.py
...
..... | INFO | AMQP - Subscribed to: queue_test
..... | DEBUG | AMQP - Message received: message_test
...
```

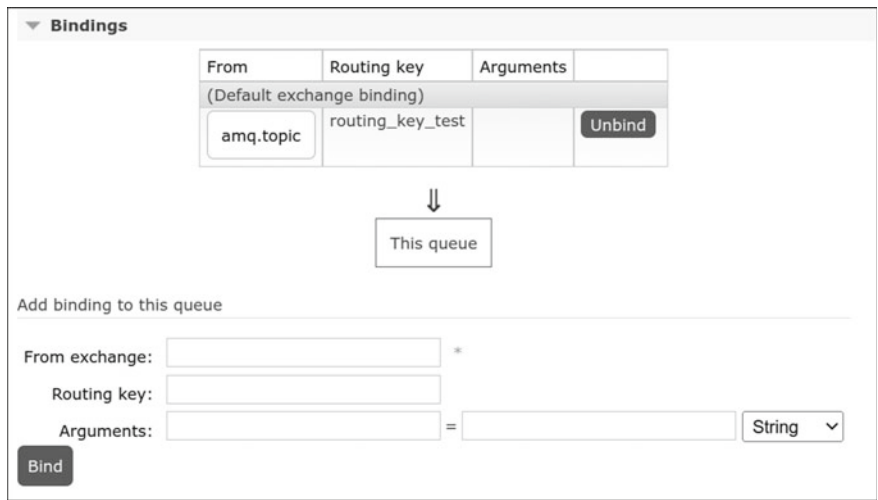


Fig. 6.6 RabbitMQ dashboard: Add binding to the notelab queue

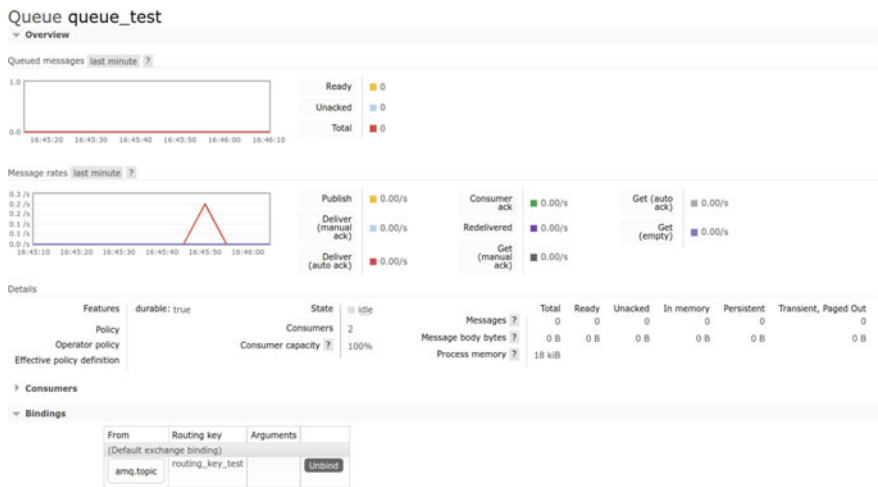


Fig. 6.7 RabbitMQ dashboard: Message from the AMQP client

Check again RabbitMQ dashboard, under chart *Queues* for AMQP message queue named *queue_test*, as shown in the above terminal. In chart *Message rates*, as illustrated in Fig. 6.7, we can observe the activity after running AMQP publisher script. This activity points out that the published message has arrived at the server. If we run the publisher script multiple times the graphs will show the increased message activity.

Now, in order to check if the client is also consuming messages, run a publisher from a workstation terminal using the previously introduced tool in the first part of

this lab. This publisher sends a message to a queue and routing key corresponding to values assigned in the client (e.g., `queue_test` and `test`). The other information includes AMQP broker url, port, credentials (username and password), exchange type, and routing key:

```
student@workstation:~$  
amqp-publish --url amqp://user:password@SERVER_IP:30672 -e amq  
.topic -r routing_key_test -b Testing_AMQP_consumer
```

Check the output on the terminal on Raspberry Pi terminal where the client script is running:

```
ubuntu@raspberrypi:~/amqp_client_rpi:~$ python3 amqp_client.py  
  
...  
..... | INFO | AMQP - Subscribed to: queue_test  
..... | DEBUG | AMQP - Message received: message_test  
...  
..... | DEBUG | AMQP - Message received: Testing_AMQP_consumer  
...
```

Check that AMQP client runs as expected.

Assignment 3

Complete the following assignment:

Create two separate scripts, one that works only as publisher and one as subscriber. The publisher should be sending messages continuously to a certain exchange with a rate of 1 message per second. Every five consecutive messages, the content of the message should be modified. The subscriber only prints out when the content of the received message was modified.

Add the code to the lab report.

References

Libraries

1. Pika library: <https://pika.readthedocs.io/en/stable/>

Tools

2. Docker Engine installation on Ubuntu: <https://docs.docker.com/engine/install/ubuntu/>
3. Python: <https://www.python.org/downloads/>
4. OpenSSH Server: <https://ubuntu.com/server/docs/service-openssh>
5. CURL: <https://curl.se/download.html>
6. K3s container orchestrator: <https://rancher.com/docs/k3s/latest/en/installation/install-options/>
7. Package: amqp-tools (0.10.0-1): <https://packages.ubuntu.com/focal/amqp-tools>
8. RabbitMQ: <https://www.rabbitmq.com/>
9. RabbitMQ helm chart: <https://github.com/bitnami/charts/tree/master/bitnami/rabbitmq>

Other Resources

10. RabbitMQ plugin for management and monitoring: <https://www.rabbitmq.com/management.html>
11. RabbitMQ AMQP 0-9-1 protocol model <https://www.rabbitmq.com/tutorials/amqp-concepts.html>
12. AMQP client in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/amqp_client.py

Chapter 7

Lab 4—Connecting Edge and Cloud Tools with REST HTTP



7.1 Prerequisites

Hardware

- Raspberry Pi 4 Model B
- Server
- Workstation
- Router

Software

- Raspberry Pi 4 Model B
 - Docker v20.10.8 [2]
 - Python v3.9 [3]
 - OpenSSH v8.4 [4]
- Server
 - curl v7.81.0 [5]
 - K3s v1.20.9+k3s1 [6]
 - OpenSSH v8.4 [4]
- Workstation
 - curl v7.81.0 [5]
 - OpenSSH v8.4 [4].

7.2 Setup

Just like *Lab 3*, also *Lab 4* establishes communication between Raspberry Pi and the server, this time however with HTTP protocol. The setup is illustrated in Fig. 7.1. In *Lab 4*, we learn how to install and deploy Elasticsearch tools and how to master REST API using HTTP protocol. We also learn how to implement an HTTP client on Raspberry Pi in Python using `requests` library tools.

This lab is divided into two different parts, using the components shown in Fig. 7.2. The first part of *Lab 4* is dedicated to deploying and testing Elasticsearch which implements an HTTP server. For testing purposes, we use `curl`, a tool that allows us for querying to HTTP servers using the CLI. In the second part, an HTTP client is implemented in Python which will be used to interact with the HTTP server. The HTTP client is intended to be run from the Raspberry Pi and will be used in the next labs by the Actuator controller to retrieve data from HTTP server. A detailed overview of the libraries used in this lab is listed as follows:

- HTTP server
 - Elasticsearch helm chart v7.17.3 [7]
- Actuator controller
 - Requests library v2.25.1 [1]

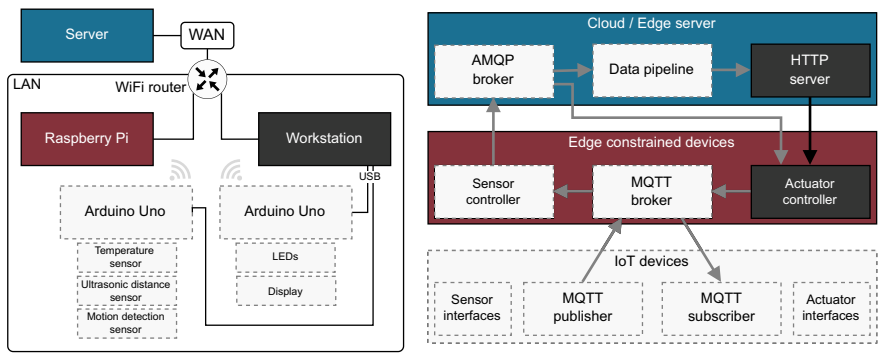
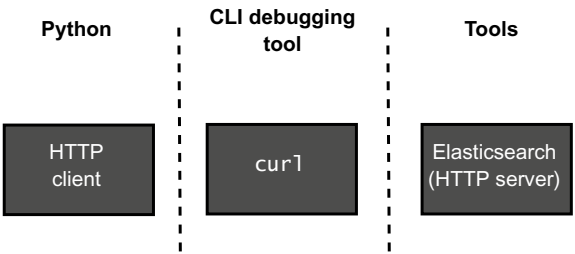


Fig. 7.1 Lab 4 setup

Fig. 7.2 Lab 4 components



7.3 Part 1: Deploy and Test Elasticsearch

In this part of the lab, we define two tasks, one to deploy and one to test an instance of Elasticsearch [8] tool on the server, along with Representational State Transfer (REST) API using HTTP protocol. For testing we make use of curl tool from the workstation (See Fig. 7.3). It should be noted that we install Elasticsearch on the Kubernetes cluster which we already created in *Lab 3*. In some cases, a reinstallation of Kubernetes may be necessary if Elasticsearch installation fails.

7.3.1 Task 1: Deploying Elasticsearch

Let us use ping command from the workstation to check whether the server is reachable through its public IP address, or domain name from the Raspberry Pi. We recall also here that Raspberry Pi and workstation are both connected to the router. By using ssh to remotely access the server, we check that k3s is properly installed. As explained in *Lab 3*, Sect. 6.3, we use helm tool for installing Elasticsearch:

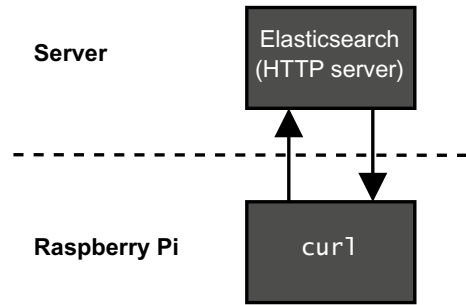
```
ubuntu@server:~$ helm repo add elastic https://Helm.elastic.co
ubuntu@server:~$
    helm install elasticsearch elastic/elasticsearch \
    --version 7.17.3 \
    --set replicas=1 \
    --set service.type=NodePort,service.nodePort=32200

NAME: elasticsearch
LAST DEPLOYED: Fri Oct  1 15:37:47 2021
NAMESPACE: default
STATUS: deployed
REVISION: 1
NOTES:
1. Watch all cluster members come up.
  \ $ kubectl get pods --namespace=default -l app=elasticsearch-
    master -w
2. Test cluster health using Helm test.
  \ $ helm test elasticsearch
```

Tips

In case this installation method fails because of either the container image or the repository are not available anymore, please refer to Appendix C.7 for an alternative installation method.

Fig. 7.3 Lab 4, part 1 workflow



Once the pod is labeled with status `Running`, we can use port 32200 for communication with Elasticsearch. We now can check from any browser installed on the workstation that Elasticsearch API is available at `http://SERVER_IP:32200/`.

Before going any further, we recommend to review the literature about the reasoning behind REST (Representational State Transfer) architectural style and the typical HTTP methods used when designing REST APIs in [9]. These HTTP methods are used in the following tasks to interact with Elasticsearch API.

Assignment 1

Answer the following questions:

- Elasticsearch is a schema-free-based search engine. What does it mean and how is this different from the traditional SQL databases?
- What tools similar to Elasticsearch are available in the free and open - source community? Please identify these tools and discuss their pros and cons.
- Describe HTTP methods when defining REST APIs.

Add the answers to the lab report.

7.3.2 Task 2: Testing Elasticsearch

In this task, we use `curl` tool from the workstation to test Elasticsearch on the server. Recall that this tool was previously installed as one of the workstation software prerequisites. With this tool, we can test Elasticsearch REST API and different HTTP methods (GET, POST, PUT, and DELETE). While `curl` supports many options (see with `curl -help all`), the syntax we use to define HTTP requests consists of

```
student@workstation:~$ curl --help all

Usage: curl [options...] <url>
-d, --data <data>           HTTP POST data
-X, --request <command>     Specify request command to use
-H, --header <header/@file> Pass custom header(s) to server
...
```

Let us first check the version of Elasticsearch installed, by executing the following command:

```
student@workstation:~$ curl -XGET "http://SERVER_IP:32200"

{
  "name" : "elasticsearch-master-0",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "kFfP8S10QSyGRRaImyusPg",
  "version" : {
    "number" : "7.17.1",
    "build_flavor" : "default",
    "build_type" : "docker",
    "build_hash" : "79d65f6e357953a5b3cbcc5e2c7c21073d89aa29",
    "build_date" : "2021-09-16T03:05:29.143308416Z",
    "build_snapshot" : false,
    "lucene_version" : "8.9.0",
    "minimum_wire_compatibility_version" : "6.8.0",
    "minimum_index_compatibility_version" : "6.0.0-beta1"
  },
  "tagline" : "You Know, for Search"
}
```

The output for HTTP GET method request matches the Elasticsearch API output on the browser. Next, we use `curl` to send data with HTTP POST method request to the Elasticsearch. The data is represented as a test JSON message containing two fields, `id` and `temperature`, as follows:

```
student@workstation:~$ curl -H "Content-Type: application/json" \
  -XPOST "http://SERVER_IP:32200/notelab/temperature_topic/1" \
  -d '{"id":"tester", "temperature":22}'

{"_index":"notelab", "_type":"temperature_topic", "_id":"1", "_version":1, "result":"created", "_shards":{"total":2, "successful":1, "failed":0}, "_seq_no":0, "_primary_term":1}
```

By performing this request, Elasticsearch creates a specific index *notelab*. This is possible because Elasticsearch is schema-free, so no previous schema definitions are required such as index or field types before indexing new data. Now, check that the data was actually stored by making use of HTTP GET request:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The prompt is 'student@workstation:~\$'. The command is 'curl -XGET "http://SERVER_IP:32200/notelab/_search?pretty" -H "Content-Type: application/json" -d '{"query":{"match_all":{"}}}}''. The output is a pretty-printed JSON object.

```
student@workstation:~$ curl -XGET "http://SERVER_IP:32200/
notelab/_search?pretty" -H 'Content-Type: application/json
' -d '{"query":{"match_all":{"}}}}'

{
  "took" : 205,
  "timed_out" : false,
  "_shards" : {
    "total" : 1,
    "successful" : 1,
    "skipped" : 0,
    "failed" : 0
  },
  "hits" : {
    "total" : {
      "value" : 1,
      "relation" : "eq"
    },
    "max_score" : 1.0,
    "hits" : [
      {
        "_index" : "notelab",
        "_type" : "temperature_topic",
        "_id" : "1",
        "_score" : 1.0,
        "_source" : {
          "id" : "tester",
          "temperature" : 22
        }
      }
    ]
  }
}
```

As we can see, Elasticsearch returns all the data contained in index *notelab*. The total number of entries in that specific index is represented with *hits*. Then, on every *hit*, we can find the data source in *_source*. Next, we update the data already inserted, using the PUT method:

```
student@workstation:~$ curl -H "Content-Type: application/json" \
  -XPUT "http://SERVER_IP:32200/notelab/temperature_topic/1" \
  -d '{"id":"tester", "temperature":10}'

{"_index":"notelab", "_type":"temperature_topic", "_id":"1", "
 _version":2, "result":"updated", "_shards":{"total":2, "
 successful":1, "failed":0}, "_seq_no":1, "_primary_term":1}
```

Retrieve the same entry again, to check whether the data was updated correctly. We can also delete the entry by using the DELETE method:

```
student@workstation:~$ curl -XDELETE http://{SERVER_IP}:32200/
notelab/temperature_topic/1
```

Retrieve again all the entries from *notelab* index to verify that the entry was actually deleted and removed from the database. To try out other options, refer to other basic Elasticsearch operations in the official quick start guide [10].

Assignment 2

Complete the following assignment:

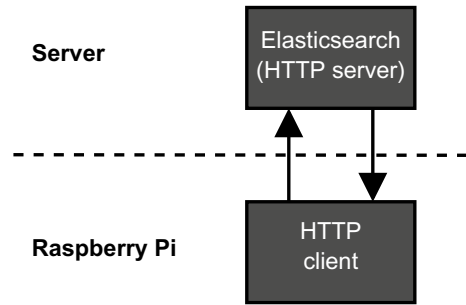
- In the previous example, add multiple entries to Elasticsearch, but with changed values of *id* and *temperature*.
- Create a query to only retrieve data matching a specific *id*.
- Create a query to retrieve only the field *temperatures* from all entries instead of the entire *_source* of each one of the entries.

Add the queries and outputs to the lab report.

7.4 Part 2: Implementing an HTTP Client in Python

We now develop an HTTP client in Python on Raspberry Pi. The client needs to be tested using Elasticsearch previously deployed on the server (See Fig. 7.4). As a baseline, we hence need to make sure that Elasticsearch is up and running, as explained previously.

Fig. 7.4 Lab 4, part 2 workflow



7.4.1 Task 3: Implementing an HTTP Client

Whenever programming in Python, in NoTE Lab, we always create a Python script file using a text editor. First, let us create a folder for this task (*http_client_rpi*) on the Raspberry Pi. Open the text editor (use command `nano`) and save a Python script under *http_client.py*. Add the following code to the file:

Source code: [11]

```
1 import requests, json, time
2 import logging
3
4
5 def query(url, method, payload=None):
6     try:
7         res = requests.Response()
8         if method == "GET":
9             res = requests.get(url)
10        if method == "POST":
11            res = requests.post(
12                url,
13                data=json.dumps(payload),
14                headers={"content-type": "application/json"},
15            )
16        if method == "PUT":
17            res = requests.put(
18                url,
19                data=json.dumps(payload),
20                headers={"content-type": "application/json"},
21            )
22        if method == "DELETE":
23            res = requests.delete(url)
24        if res.content is not None:
25            logging.debug(
```

```

26         "HTTP - [" + method + "]" + str(res.status_code) +
27         "]" + url + "]"
28     )
29     return json.loads(res.content.decode("utf8"))
30 except:
31     logging.error("HTTP - " + method + " - Connection error!")
32
33 if __name__ == "__main__":
34     logging.basicConfig(
35         format="%asctime)s | %(name)s | %(levelname)s |
36         %(message)s",
37         level=logging.DEBUG,
38     )
39     root_url = "http://134.169.115.91:32200/"
40     index_es = "testindex"
41     query(
42         url=root_url + index_es + "/test_sensor/1",
43         method="POST",
44         payload={"tester": "workstation", "sensor_data": 23},
45     )
46     query(
47         url=root_url + index_es + "/test_sensor/1",
48         method="PUT",
49         payload={"tester": "workstation", "sensor_data": 15},
50     )
51     time.sleep(1)
52     query(url=root_url + index_es + "/" + "_search", method="GET")
53     query(url=root_url + index_es, method="DELETE")

```

Code descriptions: [11]

The script starts with importing libraries: `requests`, HTTP client library, `json`, for dealing with JSON specification format, `time` library, and `logging` library, to easily define the format of the logging messages. The script only has a single function named `query()` which accepts three parameters: `url` to where to perform the query, `method` to specify which HTTP method to use, and `payload` to optionally specify the payload to send. Next, in `try-catch` statement is where the actual queries are going to be performed. Within this statement, we first declare `request`. `Response()` type as the response that will be used to store the response from the server. Then, using `if` statements, we define utilization of different methods from `requests` library based on the `method` parameter specified. Depending on the method used, there are four options:

- GET: We make use of `get()` from the `request` library to send a GET request to the specified `url`. The response is then saved in `res`, containing all the data requested from the server including standard HTTP headers, payload, and response status code among other parameters.
- POST: `post()` method sends a POST request also to the specified `url`. Here we also pass `data` as additional required parameter, in our case always in JSON format, and `headers` which actually advertises to the server that the data to accept is in content type JSON format. The response object for this request is also stored in `res`.
- PUT: PUT method requests specifies `url`, along with `data` and `headers`. The response is also saved in `res`.
- DELETE: This method sends a DELETE request to the specified `url`, passed as a parameter. With this request, we delete specified resources on the server. The response object for this request is saved in `res`.

Once we get the response from the server, we first declare a new variable `res_content` and set it to `None` value, and then, we add a condition to check if the response contains a payload in the content with `res.content` (returns content in bytes). If it is not empty, we deserialize it from the string of JSON data into `res_content` which now is a dictionary. Finally, we provide a more detailed debugging output on the invoked HTTP method, the status code, and the response with `logging.debug()` function. The block `try` returns content stored in `res_content`, while in case of an error in `except` block, we print a debug error message with `logging.error()` function.

In order to test the previously defined functions, we add `if __name__ == "__main__":` statement block, and we first define a variable `root_url`, which includes the IP address and the port for querying connecting to Elasticsearch server. We also define variables `index_es` as the name of the index that we want to create in Elasticsearch to store the data. Then, we follow up by triggering the previously defined HTTP methods using `query` function. First, we create a test sensor resource with a sample JSON payload for the index named in `index_es`. This is achieved by using the method POST, corresponding endpoint (e.g., containing index with its type and id fields), and sample payload (in JSON format) to the function. Second, we update the data stored under the specified index. This is done again by triggering the same function, but this time specifying the method PUT, the corresponding endpoint, and the updated payload value. As Elasticsearch needs some time before requesting the data newly stored, we suspend the code execution for one second with `time.sleep()`. To test data retrieving from the specified index from Elasticsearch, we again trigger the same function, this time passing the method GET and the corresponding index endpoint (appending `"_search"`). Lastly, we delete the created resource, by passing the method DELETE and corresponding index endpoint.

Let us now run the script from the Raspberry Pi's terminal and check the results:

```
ubuntu@raspberrypi:~/http_client_rpi:~$ python3 http_client.py
.....| DEBUG | Starting new HTTP connection (1): SERVER_IP
:32200
.....| DEBUG | http://SERVER_IP:32200 "POST /testindex/
test_sensor/1 HTTP/1.1" 201 142
.....| DEBUG | HTTP - [POST][201][http://SERVER_IP:32200/
testindex/test_sensor/1]
.....| DEBUG | Starting new HTTP connection (1): SERVER_IP
:32200
.....| DEBUG | http://SERVER_IP:32200 "PUT /testindex/
test_sensor/1 HTTP/1.1" 200 141
.....| DEBUG | HTTP - [PUT][200][http://SERVER_IP:32200/
testindex/test_sensor/1]
.....| DEBUG | Starting new HTTP connection (1): SERVER_IP
:32200
.....| DEBUG | http://SERVER_IP:32200 "GET /testindex/
_search HTTP/1.1" 200 205
.....| DEBUG | HTTP - [GET][200][http://SERVER_IP:32200/
testindex/_search]
.....| DEBUG | Starting new HTTP connection (1): SERVER_IP
:32200
.....| DEBUG | http://SERVER_IP:32200 "DELETE /testindex
HTTP/1.1" 200 47
.....| DEBUG | HTTP - [DELETE][200][http://SERVER_IP:32200/
testindex]
```

If everything goes as expected, the response from the server for each one of the four queries is shown in the output terminal. This includes all successful response statuses, such as 200 or 201.

Assignment 3

Complete the following assignment:

Create a separate Python script from where to import the previously created query function. Create a loop to index five times the same data structure as in previous tasks, by changing *id* each time. In the same script, add the required code to update *temperature* values of each entry previously created. Add the code to query the new index in order to verify that all entries have been modified. Add the code to delete all entries from the index you created. Add the queries and outputs to the lab report.

References

Libraries

1. Requests library: <https://requests.readthedocs.io/en/latest/user/install/>

Tools

2. Docker Engine installation on Ubuntu: <https://docs.docker.com/engine/install/ubuntu/>
3. Python: <https://www.python.org/downloads/>
4. OpenSSH Server: <https://ubuntu.com/server/docs/service-openssh>
5. CURL: <https://curl.se/download.html>
6. K3s container orchestrator: <https://rancher.com/docs/k3s/latest/en/installation/install-options/>
7. Elasticsearch helm chart: <https://github.com/elastic/helm-charts/tree/7.17/elasticsearch>
8. Elasticsearch: <https://www.elastic.co/elasticsearch/>

Other Resources

9. RESTful API tutorial: <https://restfulapi.net/http-methods/>
10. Elasticsearch tutorial: <https://www.elastic.co/guide/en/elasticsearch/reference/current/getting-started.html>
11. HTTP client in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/http_client.py

Chapter 8

Lab 5—Prototyping Data Connectors



8.1 Prerequisites

Hardware

- Raspberry Pi 4 Model B
- Server
- Workstation
- Router

Software

- Raspberry Pi 4 Model B
 - Docker v20.10.8 [4]
 - Python v3.9 [5]
 - OpenSSH v8.4 [6]
- Server
 - curl v7.81.0 [7]
 - K3s v1.20.9+k3s1 [8]
 - OpenSSH v8.4 [6]
- Workstation
 - curl v7.81.0 [7]
 - OpenSSH v8.4 [6]
 - mosquitto-clients v1.6.9-1 [9]
 - amqp-tools v0.10.0-1 [10].

xss

8.2 Setup

In this lab, we implement *data connectors* between various parts of the system, such that data can be forwarded end-to-end, from IoT devices to cloud tools. The setup for this lab is shown in Fig. 8.1, and it includes Raspberry Pi, WiFi router, workstation, and server. To enable data exchange between the different components, we make use of three different connectors: (1) *sensor controller* (MQTT-to-AMQP brokers), to forward the sensor data coming from MQTT broker running on Raspberry Pi to RabbitMQ running on the server, (2) *Data pipeline* (AMQP broker-to-HTTP server), to index the data received from AMQP broker into the Elasticsearch database, both running on the server, and (3) *actuator controller* (HTTP server/AMQP broker-to-MQTT broker), to forward data from RabbitMQ and Elasticsearch to the MQTT broker.

This lab is divided into four different parts, using a variety of tools and pieces of code, as shown in Fig. 8.2. In Part 1, an MQTT client is implemented in Python on Raspberry Pi and used to publish and subscribe to the MQTT broker. In Part 2, the *sensor controller* implements the subscriber functionality from MQTT client, so it can collect data arriving from MQTT broker and then forward these data to AMQP broker, by using AMQP client. For testing purposes of this part, we use `mosquitto-clients` to publish test MQTT messages on the workstation. In Part 3, *actuator controller* implements a subscriber from AMQP client to collect data from AMQP broker, and it also uses HTTP client to retrieve data from HTTP server. All data collected is forwarded to MQTT broker using MQTT client. For testing purposes, we use `amqp-tools` to publish test AMQP messages and `curl` to send data to HTTP server from the workstation. In Part 4, we prototype the third connector, this time by deploying a data pipeline between AMQP broker and HTTP server. This data pipeline uses Logstash tool to automatically forward the data received in AMQP broker into the HTTP server. For testing purposes of this part, we use again `amqp-tools` to publish test AMQP messages to AMQP broker, followed by the

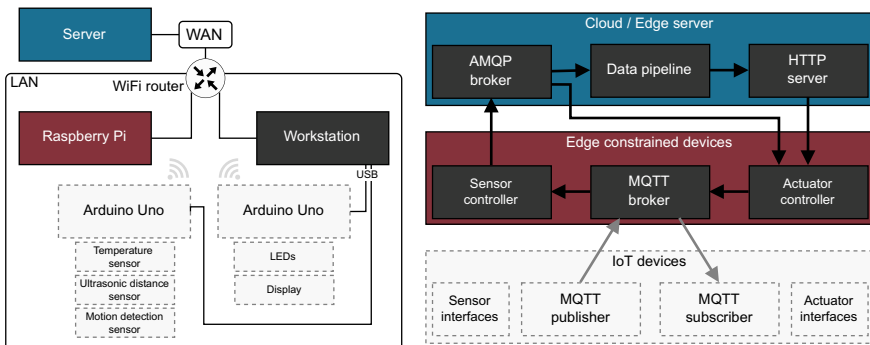


Fig. 8.1 Lab 5 setup

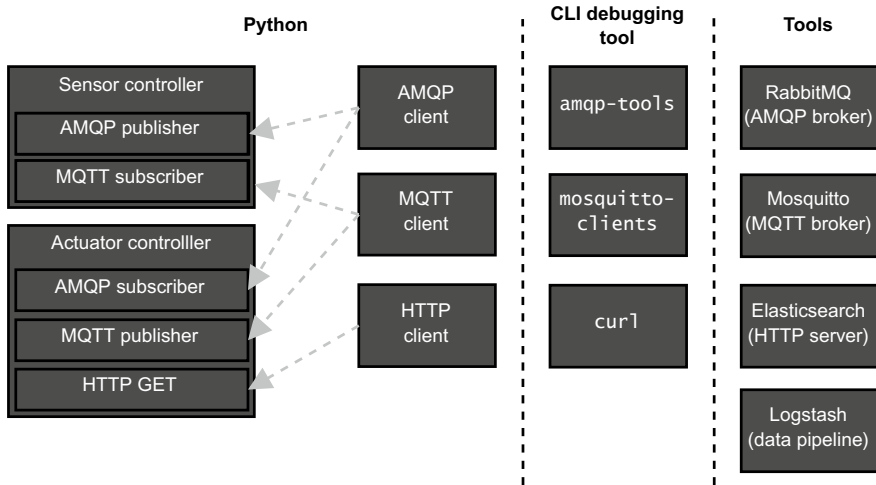


Fig. 8.2 Lab 5 components

utilization of `curl` to retrieve the data sent in these messages from the HTTP server. In our final Lab 7, we then replace the testing tool `amqp-tools` with real sensors and actuators.

A detailed overview of the libraries that were previously developed and also used in this lab is listed as follows:

- MQTT broker
 - Eclipse Mosquitto v1.6.15 [11]
- AMQP broker
 - RabbitMQ helm chart v10.1.14 [12]
- HTTP server
 - Elasticsearch helm chart v7.17.3 [14]
- Data pipeline
 - Logstash [13]
- *sensor controller and actuator controller*
 - Paho Python Client v1.6.1 [1]
 - Pika library v1.2.1 [2]
 - Requests library v2.25.1 [3]
 - AMQP client [16] (from Lab 3)
 - HTTP client [15] (from Lab 4)

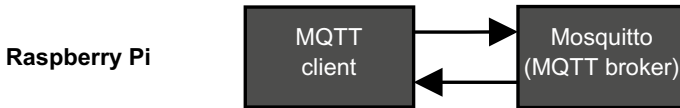


Fig. 8.3 Lab 5, part 1 workflow

8.3 Part 1: Implementing MQTT Client in Python

In this part, we learn how to develop an MQTT client in Python on Raspberry Pi. The client to be developed is expected to work either as publisher or subscriber. We also test the communication to MQTT broker, also running on Raspberry Pi, and previously implemented (See Fig. 8.3). Before starting with Task 1, let us first make sure that Mosquitto is up and running.

8.3.1 Task 1: Implementing an MQTT Client

In this task, we first create a folder on Raspberry Pi, where we will save the Python script, for instance, folder *mqtt_client_rpi*. Let us open text editor nano and save the Python script under *mqtt_client.py*. We now add the following code to that Python script:

Source code: [16]

```
1 import paho.mqtt.client as mqtt
2 import logging
3
4
5 def on_connect(client, userdata, flags, rc):
6     logging.info("MQTT - connected to broker, code " + str(rc))
7
8
9 def on_message(client, userdata, msg):
10     global message
11     message = msg
12     topic = msg.topic
13     payload = str(msg.payload.decode("utf-8"))
14     logging.debug("MQTT - msg received [" + topic + "]: " +
15                 payload)
16
17 def on_subscribe(client, userdata, mid, granted_qos):
```

```
18     logging.info("MQTT - subscribed: " + str(mid) + " " +
19                  str(granted_qos))
20
21 def on_publish(client, userdata, mid):
22     logging.debug("MQTT - published: " + str(mid))
23
24
25 def on_log(client, userdata, level, string):
26     logging.debug(string)
27
28
29 def connect(ip, port=1883):
30     mqttc = mqtt.Client()
31     mqttc.on_connect = on_connect
32     mqttc.on_message = on_message
33     mqttc.on_publish = on_publish
34     mqttc.on_subscribe = on_subscribe
35     # mqttc.on_log = on_log
36     try:
37         mqttc.connect(ip, port, 60)
38     except:
39         logging.error(
40             "MQTT - [" + ip + ":" + str(port) + "] connection
41                 failed!, exiting..."
42         )
43         exit(1)
44     return mqttc
45
46 if __name__ == "__main__":
47     logging.basicConfig(
48         format="%asctime)s | %(name)s | %(levelname)s |
49             %(message)s",
50         level=logging.DEBUG,
51     )
52     mqtt_broker_ip = "RPI_IP"
53     mqtt_broker_port = 1883
54     mqtt_topic = "testMQTT"
55     mqttc = connect(mqtt_broker_ip, mqtt_broker_port)
56     mqttc.subscribe(mqtt_topic)
57     mqttc.loop_forever()
```


Code descriptions: [17]

The script starts by importing the Paho MQTT client library, `import paho-mqtt`, previously installed as one of the software prerequisites on the workstation, and logging library, to easily define the format of the logging messages. Then, the script is split into five functions: `on_connect()`, `on_messages()`, `on_subscribe()`, `on_publish()`, and `on_log()`. These functions override the original functions from the Paho MQTT library, and they are going to be triggered when some specific event occurs. We now describe these functions in more detail.

- `on_connect()`: This function is triggered each time MQTT broker responds to the request for connection. It uses four arguments: `client` (corresponding to the client instance), `userdata` (corresponding to user data of any type which can be defined when creating a new client instance), `flags` (which are response flags from MQTT broker), and `rc` (meaning *result code* of type integer, indicating connection status). In our `on_connect()` function, we only make use of `rc` argument to provide debugging output of connection status using `logging.info()` function, considering that `rc` is first converted to a string type using function `str()`.
- `on_message()`: This function is triggered each time the client receives a message. It uses three arguments: `client`, `userdata`, and `msg` (corresponding to a specific MQTT message). In this function, we define a new global variable `message`. This message is important to remember, as it is used in other functions to access the message stored in the client. For now, let us just assign it to variable `msg`. Variable `msg` is a dictionary that includes the topic (string value with the topic) and payload (array of bytes with the message); there are other values but we do not use them here. We provide a more detailed debugging output with `logging.debug()` function for topic and payload arguments, prior to conversion of the payload bytes into a string, as shown in the code.
- `on_subscribe()`: This function is triggered when the client subscribes to a topic. It uses four arguments: `client`, `userdata`, `mid` (corresponding to an integer as a unique message identifier), and `granted_qos` (corresponding to an integer specifying the service classes, or what is often referred to as Quality of Service QoS level). In this function, we make use of `mid` and `granted_qos` with `logging.info()` function, to print a debug message when a client is subscribed with corresponding message identifier and QoS level, prior to conversion to string type.
- `on_publish()`: This function is triggered when a message is successfully sent to the publisher. It uses three arguments: `client`, `userdata`,

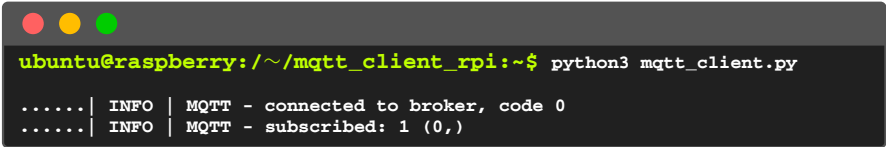
and `mid`. In this function, we make use of only `mid` and print out a debug message for the corresponding message identifier, prior to conversion to a string type.

- `on_log()`: This function returns MQTT protocol events as client log information when needed. It uses four arguments: `client`, `userdata`, `level`, and `buff`. Here, we just print out the string data of the event in a debug message.

After overriding original functions, we define our own `connect()` function used to connect the client to the broker. This function takes as argument `ip` and `port` of MQTT broker that the clients connect to. This function is used by other tasks and scripts that use this client. Within this function, we first create a new client instance of the MQTT client from paho library `mqtt.Client()`. We override the original paho functions with the newly defined ones, managing in this way the functionality of the client functions. Then, we trigger the original `connect()` function from the MQTT library by passing the `ip`, `port`, and `keepalive` (timeout value 60s) values. This function is triggered within a `try/except` block to catch the exception in case the client cannot connect, which in this case we print a debug error message with `logging.error()` function and exit the process. Otherwise, the function returns the client instance.

Finally, we add the code to test the client in `if __name__ == "__main__"` statement block. This statement is only executed when the script is run as the main program, so it will not be executed when this script is imported by other script files. Within, we define `mqtt_broker_ip`, `mqtt_broker_port`, and `mqtt_broker_topic`. In this case, as we only need to use this script currently as MQTT subscriber, for the client instance we only trigger functions `connect()` and `subscribe()` specifying the corresponding parameters as previously explained. The last step is to trigger the `loop_forever()` function to process network traffic running the program indefinitely and handling reconnection automatically as required.

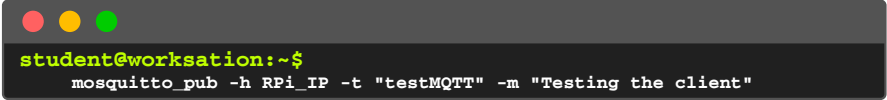
Before running the script, make sure that Mosquitto is up and running on Raspberry Pi. Run the script on Raspberry Pi as follows:



```
ubuntu@raspberrypi:~/mqtt_client_rpi~$ python3 mqtt_client.py
.....| INFO | MQTT - connected to broker, code 0
.....| INFO | MQTT - subscribed: 1 (0,)
```

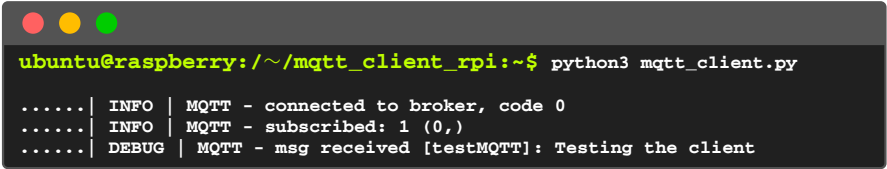
As we can see, MQTT client is connected and subscribed to the broker. The status of the connection equals to result code 0, corresponding to the value *Connection*

successful (see original library documentation [1] to see further result code values). The output for subscribing indicates subscribe message identifier 1 with QoS level 0. You can also check the output on from Mosquitto, and it should show that a new client with the workstation IP address was connected. Let us open also a terminal on the workstation and run a test Mosquitto publisher, which publishes a test message to the MQTT topic specified in the script:



```
student@workstation:~$  
mosquitto_pub -h RPi_IP -t "testMQTT" -m "Testing the client"
```

After publishing the message, the logs from *mqtt_client* should show that the message was received:



```
ubuntu@raspberrypi:~/mqtt_client_rpi:~$ python3 mqtt_client.py  
.....| INFO | MQTT - connected to broker, code 0  
.....| INFO | MQTT - subscribed: 1 (0,)  
.....| DEBUG | MQTT - msg received [testMQTT]: Testing the client
```

Assignment 1

Complete the following assignment:

Implement an MQTT publisher in a separate Python script by importing the MQTT client previously described. This new publisher needs to send a message to a certain topic every 5 seconds. Check that the subscriber receives the messages and save the output. Save the code and outputs to the lab report.

8.4 Part 2: Data Connector—MQTT and AMQP

The objective of this part is to prototype *sensor controller* on Raspberry Pi. *sensor controller* forwards the data received from MQTT broker to AMQP broker running on the server. To this end, we use MQTT and AMQP clients previously implemented in Task 1 of this lab and in Lab 4, respectively. Also, here, first make sure that Mosquitto and RabbitMQ brokers are up and running on Raspberry Pi and server, correspondingly. The workflow to test is illustrated in Fig. 8.4.

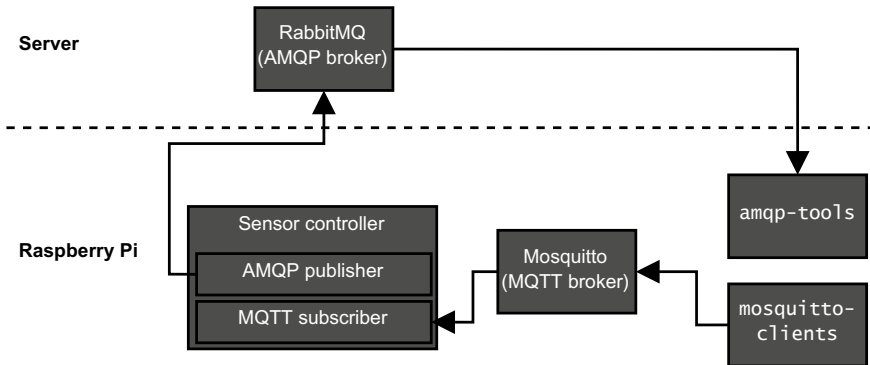


Fig. 8.4 Lab 5, part 2 workflow

8.4.1 Task 2: Prototyping the Sensor Controller

We first establish remote access to Raspberry Pi and create a folder for this part of the lab (*sensor_controller_proto*), which is a folder where the script is to be saved. Copy into this folder AMQP client code (*amqp_client.py*), as implemented in Sect. 6.4.1 in Lab 3. Also, copy MQTT client (*mqtt_client.py*), implemented in Sect. 8.3 in Task 1 of this lab. Next, create a new script under *sensor_controller_proto.py* name and add the following code:

Source code: [20]

```

1 import mqtt_client
2 import amqp_client
3
4 # rabbit
5 ip_rabbit = "SERVER_IP"
6 port_rabbit = 30672
7 user_rabbit = "user"
8 pass_rabbit = "password"
9 exchange_rabbit = "amq.topic"
10 r_key_rabbit = "control_sensor_key"
11 queue_rabbit = "control_sensor_queue"
12 # mosquitto
13 ip_mosquitto = "RPi_IP"
14 topic_mosquitto = "test_topic"
15
16 # connect to rabbitmq
17 amqp_ch = amqp_client.connect_to_broker(ip_rabbit,
    port=port_rabbit, user=user_rabbit, passw=pass_rabbit)

```

```

18 amqp_client.create_queue(amqp_ch, exchange_rabbit,
    r_key_rabbit, queue_rabbit)
19
20 # connect to mosquitto and subscribe to topics
21 mqtt_client.message = None
22 mqttc = mqtt_client.connect(ip_mosquitto)
23 mqttc.subscribe(topic_mosquitto)
24
25 # keep checking if new data arrived
26 mqttc.loop_start()
27 while True:
28     if mqtt_client.message is not None:
29         # retrieve topic and payload
30         topic = mqtt_client.message
31         payload =
            str(mqtt_client.message.payload.decode("utf-8"))
32         print("sensor-controller - MQTT subscriber - Message
            received: " + payload)
33         # forward payload to amqp server
34         print("sensor-controller - AMQP publisher - Forwarding
            data from Mosquitto to Rabbitmq: " + payload)
35         amqp_ch.basic_publish(exchange_rabbit, r_key_rabbit,
            payload)
36         # reset message
37         mqtt_client.message = None

```

Code descriptions: [20]

The script starts by importing MQTT and AMQP client scripts, `import mqtt_client, amqp_client`. It then defines the required parameters to connect to AMQP and MQTT brokers. AMQP parameters are `ip_rabbit`, `exchange_rabbit`, `r_key_rabbit`, and `queue_rabbit` corresponding to server IP address, exchange, routing key, and queue, respectively. MQTT parameters used include `ip_mosquitto` and `topics_mosquitto` corresponding to Raspberry Pi IP address and MQTT topic, respectively. After defining parameters, we connect to RabbitMQ by passing on server IP address, port, and authentication credentials. Next, we create a queue, passing the channel previously opened, exchange, routing key, and queue name. Then, with MQTT client, we first initialize the global message, and then we connect to the Mosquitto broker specifying the IP address. From the created client, we then subscribe to the topic. Once both clients MQTT and AMQP are initialized, we trigger MQTT loop to check for new messages, and then we start a loop to check if the value of

`mqtt.client.message` is not empty. In case a new message is received, we retrieve the topic and the payload, considering that payload needs to be decoded first to a string. The last step is to publish the payload received from MQTT broker to AMQP broker, with `basic_publish()` function from the original `pika` library, by also adding the parameters previously defined.

Let us now run the `sensor_controller_proto.py` script from the terminal on Raspberry Pi, as follows:

```
ubuntu@raspberrypi: /~/sensor_controller_proto:~$
python3 sensor_controller_proto.py
```

Check in RabbitMQ dashboard, under chart *Queues* that all the queues defined in the script have been created and are binded to the corresponding routing keys, as shown here in Fig. 8.5.

In order to check the functionality of `sensor_controller_proto` apart from the logs, run a test AMQP subscriber from a separate terminal on your workstation, specifying the server IP, port, user credentials, exchange type, and three routing keys, as coded in the script.

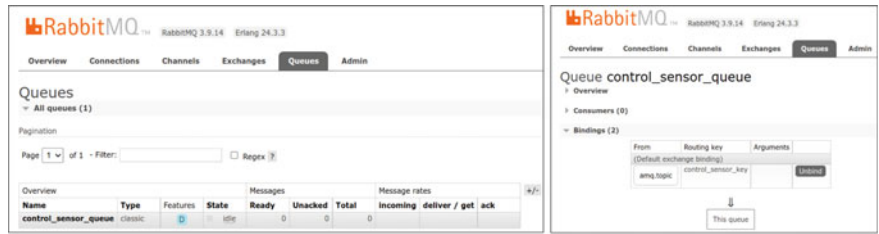


Fig. 8.5 RabbitMQ: `control_sensor_queue`

```
student@workstation:~$ amqp-consume --url amqp://user:
password@SERVER_IP:30672 -e amq.topic -r control_sensor_key cat
```

Let us open also the second terminal on the workstation and run a test Mosquitto publisher, sending the following set of messages to the MQTT topic specified in the script:

```
student@workstation:~$ mosquitto pub -h RPi_IP -t "test_topic" -m "
Testing the sensor controller - message 1"
student@workstation:~$ mosquitto pub -h RPi_IP -t "test_topic" -m "
Testing the sensor controller - message 2"
```

After publishing the message, the logs from *sensor_controller_proto* script should show that the message was received from MQTT broker and forwarded to AMQP server:

```
ubuntu@raspberrypi:~/sensor_controller_proto:~$
python3 sensor_controller_proto.py

sensor-controller - MQTT subscriber - Message received: Testing the sensor
controller - message 1
sensor-controller - AMQP publisher - Forwarding data from Mosquitto to
Rabbitmq: Testing the sensor controller - message 1
sensor-controller - MQTT subscriber - Message received: Testing the sensor
controller - message 2
sensor-controller - AMQP publisher - Forwarding data from Mosquitto to
Rabbitmq: Testing the sensor controller - message 2
```

We can also check that the output of AMQP consumer is as expected, for instance,

```
student@workstation:~$ amqp-consume --url amqp://user:
password@SERVER_IP:30672 -e amq.topic -r collect_sensor_key cat
student@workstation:~$
student@workstation:~$ Testing the sensor controller - message 1Testing
the sensor controller - message 2
```

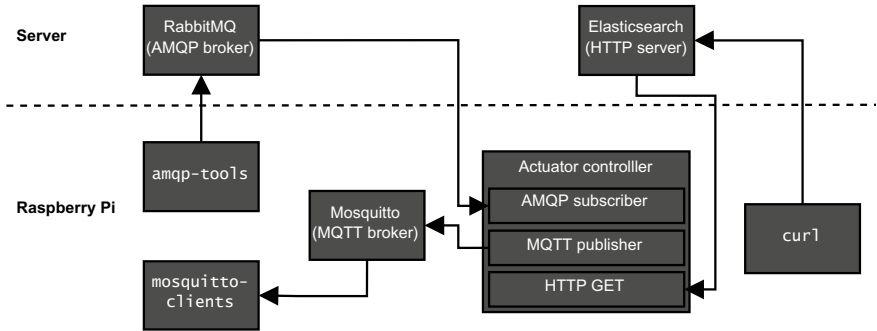


Fig. 8.6 Lab 5, part 3 workflow

Assignment 2

Complete the following assignment:

Modify the connector to be able to convert the payload received from the MQTT in JSON format into a Python dictionary (hint: use `json.loads()` function). Assuming that JSON message contains *id* and *temperature* values, create a condition in the connector so that the message is only forwarded if the temperature value is different from the previous one (hint: use a global variable to store the previous value). Test these modifications by sending the data using the mosquitto publisher tool.

Save the code and outputs to the lab report.

8.5 Part 3: Data Connector—HTTP, AMQP and MQTT

In this part of the lab, we prototype *actuator controller* on Raspberry Pi. This connector forwards data from HTTP server and AMQP broker to MQTT broker and hence uses previously developed AMQP, HTTP, and MQTT clients. First, AMQP client is used to subscribe to a topic in RabbitMQ; every time a new data arrives HTTP client also retrieves data from HTTP server. All data are then forwarded to MQTT broker using different topics. Before starting with Task 3, let us once more make sure that Mosquitto is up and running on Raspberry Pi, and RabbitMQ and Elasticsearch are running on the server. The workflow to test is illustrated in Fig. 8.6.

8.5.1 Task 3: Prototyping the Actuator Controller

As always, we first create a folder for this lab (*actuator_controller_proto*) where we can save the Python script. For implementation, we need to use functions from MQTT client (implemented previously in Sect. 8.3), AMQP client (implemented in Sect. 6.4.1 in *Lab 3*), and HTTP client (implemented in Sect. 7.4.1 in *Lab 4*). We copy their corresponding scripts, `mqtt_client.py`, `amqp_client.py`, and `http_client.py` to the newly created folder. Add the following code to the created script file:

Source code: [21]

```

1 import mqtt_client
2 import amqp_client
3 import http_client
4 import json
5
6
7 def on_amqp_message(channel, method, properties, msg):
8     print("actuator-controller - AMQP consumer - Message
9         received: " + msg.decode("utf-8"))
10    payload_rabbit = str(msg.decode("utf-8"))
11    print("actuator-controller - HTTP GET - Retrieving message
12        from Elasticsearch...")
13    res_elastic = http_client.query(url_elastic + "/" +
14        index_elastic + "/_search",method="GET")
15    payload_elastic = None
16    if res_elastic is not None:
17        if res_elastic["hits"]["total"]["value"] > 0:
18            hits = res_elastic["hits"]["hits"]
19            last_hit = hits[len(hits) - 1]
20            payload_elastic = json.dumps(last_hit["_source"])
21    if payload_elastic is not None:
22        print("actuator-controller - MQTT publisher -
23            Forwarding data from Elasticsearch to Mosquitto: "
24            + payload_elastic)
25        mqttc.publish(actuator1_topic_mosquitto,
26            payload_elastic)
27    if payload_rabbit is not None:
28        print("actuator-controller - MQTT publisher -
29            Forwarding data from Rabbitmq to Mosquitto: " +
30            payload_rabbit)
31        mqttc.publish(actuator2_topic_mosquitto,
32            payload_rabbit)
33
34
35 url_elastic = "http://SERVER_IP:32200"

```

```

27 index_elastic = "control_actuator_index"
28 ip_rabbit = "SERVER_IP"
29 port_rabbit = 30672
30 user_rabbit = "user"
31 pass_rabbit = "password"
32 exchange_rabbit = "amq.topic"
33 r_key_rabbit = "control_actuator_key"
34 queue_rabbit = "control_actuator_queue"
35 ip_mosquitto = "RPi_IP"
36 actuator1_topic_mosquitto = "actuator_topic_1"
37 actuator2_topic_mosquitto = "actuator_topic_2"
38
39 mqttc = mqtt_client.connect(ip_mosquitto)
40
41 amqp_client.message = None
42 amqp_channel = amqp_client.connect_to_broker(ip_rabbit,
43                                             port=port_rabbit, user=user_rabbit, passw=pass_rabbit)
44 amqp_client.subscribe(amqp_channel, queue_rabbit,
45                       on_amqp_message)
46
47 try:
48     amqp_channel.start_consuming()
49 except KeyboardInterrupt:
50     amqp_channel.stop_consuming()
51 amqp_channel.close()

```

Code descriptions: [21]

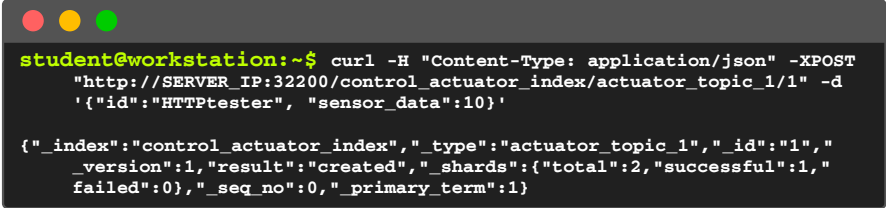
The script starts with importing the previously developed clients: HTTP (`http_client`), AMQP (`amqp_client`), and MQTT (`mqtt_client`). We also import the `json` library in order to handle JSON messages. The script contains one function that is triggered when a new AMQP message is received:

- `on_amqp_message()`: This function is triggered each time AMQP consumer receives a message using four arguments: `channel`, `method`, `properties`, and `msg`, from where only `msg` (AMQP message body) is used (for now.) The first step is to decode `msg` and convert it to a string type named `payload_rabbit`. The following step is to call `query()` function from `http_client` to retrieve data stored under a specified index from Elasticsearch, using the GET method. The data is then stored into `res_elastic`. Afterward, we check whether the response from Elasticsearch is not empty, otherwise the code would fail when trying to extract the values. We also check if the total number of hits in that specific index

returns a positive integer, otherwise we do not need to try to extract any data. Only when some hits exist, we retrieve the last one, and by using JSON library, we encode it into `payload_elastic` variable. Finally, in case that the payloads from Elasticsearch and RabbitMQ are not empty, we use `mqttc` client instance to publish data to different MQTT topics with the corresponding payloads.

Next, we define the parameters for the connector to connect to RabbitMQ, Mosquitto, and Elasticsearch. For connecting to Elasticsearch, this includes assigning `url` and the name of the index that we want to create in Elasticsearch to store the data. For connecting to RabbitMQ, parameters include `ip` of the broker and three strings assigned to queue `queue_rabbit`, exchange `exchange_rabbit`, and routing key `r_key_rabbit`. Finally, for connecting to Mosquitto, parameters need to include `ip` of the broker and two topic variables for testing the connector (e.g., `actuator1_topic_mosquitto` and `actuator2_topic_mosquitto`). After defining these parameters, we connect to MQTT and AMQP brokers. Finally, we subscribe the client to RabbitMQ with `subscribe()` function while specifying the parameters.

Before testing the *actuator controller*, we are going to add a corresponding queue in the RabbitMQ dashboard, as explained in Sect. 6.3.1 of Lab 3. Add a queue with the same name as defined in the script (here: `control_actuator_queue`) and bind specified exchange type (*amq.topic*) with the specified message routing key, i.e., `control_actuator_key` under *Binding*. Then, from the workstation, we use `curl` to send data with HTTP POST method to Elasticsearch. For endpoints, we use one of the topics defined in the script (i.e., `actuator_topic_1`). The data is now represented as a test JSON message containing two fields, *id* and *sensor_data*:



```
student@workstation:~$ curl -H "Content-Type: application/json" -XPOST
"http://SERVER_IP:32200/control_actuator_index/actuator_topic_1/1" -d
'{"id": "HTTPtester", "sensor_data": 10}'

{"_index": "control_actuator_index", "_type": "actuator_topic_1", "_id": "1",
  "_version": 1, "result": "created", "_shards": {"total": 2, "successful": 1,
  "failed": 0}, "_seq_no": 0, "_primary_term": 1}
```

Open an additional terminal in the workstation to subscribe to one of the MQTT topics as follows:

```
student@workstation:~$ mosquitto_sub -h RPi_IP -t "actuator_topic_1"
```

Open yet another new terminal, now on Raspberry Pi, and run *actuator_controller_proto.py* as follows:

```
ubuntu@raspberrypi:~/actuator_controller_proto:~$  
python3 actuator_controller_proto.py
```

Now open one more terminal in the workstation to publish messages to RabbitMQ, specifying the routing key as follows:

```
student@workstation:~$  
amqp-publish --url amqp://user:password@SERVER_IP:30672 -e amq.topic -r  
control_actuator_key -b '{"id":"AMQPtester", "sensor_data":25}'
```

Finally, check the logs from *actuator_controller_proto.py* to make sure that the messages are forwarded correctly:

```
ubuntu@raspberrypi:~/actuator_controller_proto:~$  
python3 actuator_controller_proto.py  
  
actuator-controller - AMQP consumer - Message received: {"id":"AMQPtester",  
"sensor_data":25}  
actuator-controller - HTTP GET - Retrieving message from Elasticsearch...  
actuator-controller - MQTT publisher - Forwarding data from Elasticsearch to  
Mosquitto: {"id": "HTTPtester", "sensor_data": 10}  
actuator-controller - MQTT publisher - Forwarding data from Rabbitmq to  
Mosquitto: {"id":"AMQPtester", "sensor_data":25}
```

If everything goes as expected, the responses from RabbitMQ and Elasticsearch appear in the output terminal. In addition, the output shows payload extracted from these responses and forwarded to Mosquitto broker. We should also see that our Mosquitto subscriber received the corresponding message.

Assignment 3

Complete the following assignment:

Instead of only considering the last hit when retrieving data from Elasticsearch, modify the code to consider all the hits. Convert the source of hits to a Python dictionary (hint: use JSON library), calculate their average value and send the resulting value within the JSON structure to the MQTT broker. Save the code and outputs to the lab report.

8.6 Part 4: Data Connector—AMQP and HTTP

In this part, we implement a new connector, by deploying the so-called *data pipeline* in Logstash tool, connecting RabbitMQ to Elasticsearch by indexing data into the database automatically. Before starting Task 4, make sure that RabbitMQ and Elasticsearch are both up and running on the server. The workflow to test is illustrated in Fig. 8.7.

8.6.1 Task 4: Deploying and Testing Logstash

Before installing Logstash, we need to first create a new folder (e.g., `logstash_config`). In this folder, create a configuration file in YAML format, `logstash_conf.yml`, as follows:

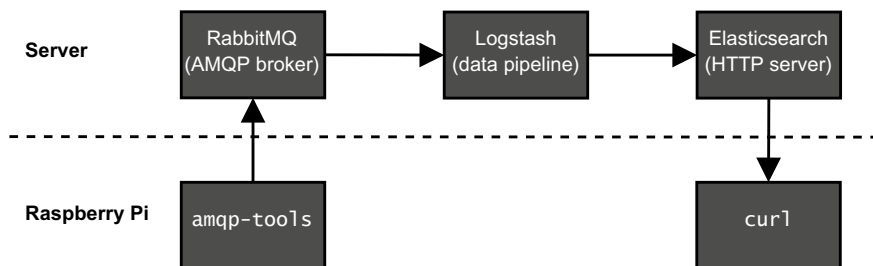


Fig. 8.7 Lab 5, part 4 workflow

Source code: [22]

```
1 logstashConfig:
2   logstash.yml: |
3     xpack.monitoring.enabled: false
4     http.host: 0.0.0.0
5
6 logstashPipeline:
7   logstash.conf: |
8     input {
9       rabbitmq {
10         host => "rabbitmq"
11         user => "user"
12         password => "password"
13         exchange => "amq.topic"
14         durable => false
15         queue => "temperature"
16       }
17     }
18     output {
19       elasticsearch {
20         hosts => "elasticsearch-master"
21         data_stream => "false"
22         index => "temperature"
23       }
24     }
```

Code descriptions: [22]

The important part of this file is in `logstash.conf` section where we specify the input and output of the logstash pipeline. As input, we specify RabbitMQ, where `host` is the name of the service, in this case `rabbitmq` and `user/password` as previously defined during the deployment of RabbitMQ. As `exchange`, we use in this case `amq.topic` which is going to route the messages to the queues based on specific matching between certain routing key and the pattern used to bind a queue to an exchange. In this case, the queue name is specified as `notelab` and we mark it as non-*durable*, so the messages will not persist after restarting the broker. To learn more about the different RabbitMQ input plugin parameters, please check the official Logstash documentation [19]. As output, we specify as `hosts` the name of the Elasticsearch service in K3s, `elasticsearch-master`, and the index where to store the data `notelab`. To learn more about the different Elasticsearch input plugin parameters, please check the official Logstash documentation [18].

Assuming we save Logstash configuration YAML file under the name `logstash_conf.yml`, we can install Logstash on the server with the following command:

```
ubuntu@server:~$ helm install logstash elastic/logstash \
--version 7.17.3 \
-f logstash_conf.yml \
--set replicas=1

NAME: logstash
LAST DEPLOYED: Fri Oct 1 17:57:41 2021
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
...
```

Tips

In case this installation method fails because of either the container image or the repository are not available anymore, please refer to Appendix C.8 for an alternative installation method.

Wait until the logstash pod has the status `Running` (check with `kubectl get pods -all-namespaces -o wide`). To verify that Logstash deployment was successful, we can check in RabbitMQ dashboard if a new consumer has been added to the topic we specified as illustrated in Fig. 8.8.

To test that Logstash is working as expected, we send a JSON message to RabbitMQ and check if this data is indexed into Elasticsearch by Logstash as expected. For this to work, we need to make sure that RabbitMQ's parameters (user credentials, exchange type, and queue name) in AMQP publish message correspond to Logstash configuration file. We send AMQP messages from workstation, where we specify the routing key, which is required for binding a queue to the exchange, with value `logstash`, as follows:

```
student@workstation:~$
amqp-publish --url amqp://user:password@{SERVER_IP}:30672 -e amq.topic
-r logstash -b "{\"id\":\"AMQPtester\", \"sensor_data\":5}"
```

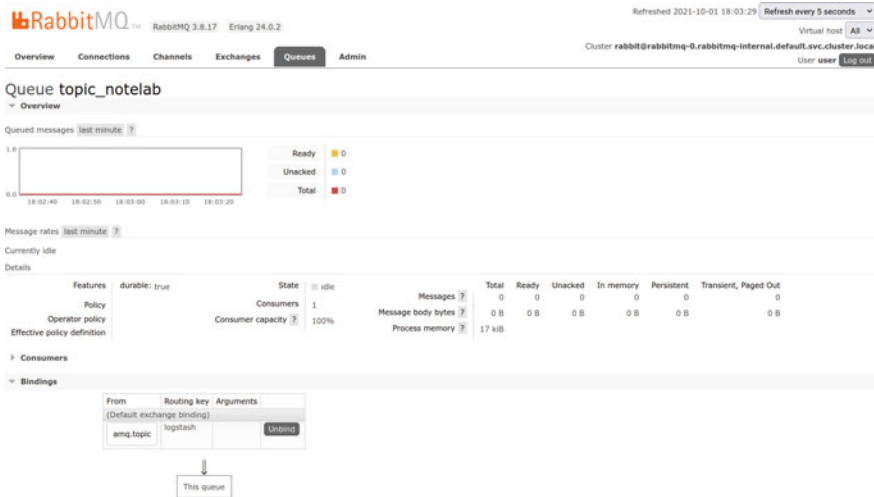


Fig. 8.8 Logstash subscribed to RabbitMQ

After publishing the message, we can check in RabbitMQ dashboard that the message arrived correctly, under *Queues* tab. Check also that in *Bindings* tab, the specified Logstash routing key is there. The next step is to check if the data sent was successfully indexed into Elasticsearch. To do that, we run `curl` from the workstation to retrieve the data with HTTP GET method request. The data should be stored under the index corresponding to the matching parameter of Logstash configuration file, in our case under *temperature*:

```
student@workstation:~$
curl -XGET http://{SERVER_IP}:32200/temperature/_search?pretty

....
{"max_score" : 1.0,
 "hits" : [
   {
     "_index" : "notelab",
     "_type" : "doc",
     "_id" : "pZVvA4AB3D5RJ7rdovj4",
     "_score" : 1.0,
     "_source" : {
       "sensor_data" : 5,
       "@version" : "1",
       "@timestamp" : "2022-04-07T09:51:29.278Z",
       "id" : "AMQPtester"
     }
   }
 ]
....
```


If everything goes ok, Elasticsearch should return all the *hits* containing data that the publisher has sent to RabbitMQ.

Assignment 4

Complete the following assignment:

Create a new connector in Python that can replace Logstash tool. The connector should first subscribe to the temperature topic in RabbitMQ and then index the data into Elasticsearch (hint: use the HTTP POST method when indexing to Elasticsearch).

Save the code and outputs to the lab report.

References

Libraries

1. Paho Python Client: <https://www.eclipse.org/paho/index.php?page=clients/python/index.php>
2. Pika library: <https://pika.readthedocs.io/en/stable/>
3. Requests library: <https://requests.readthedocs.io/en/latest/user/install/>
4. Docker Engine installation on Ubuntu: <https://docs.docker.com/engine/install/ubuntu/>
5. Python: <https://www.python.org/downloads/>

Tools

6. OpenSSH Server: <https://ubuntu.com/server/docs/service-openssh>
7. CURL: <https://curl.se/download.html>
8. K3s container orchestrator: <https://rancher.com/docs/k3s/latest/en/installation/install-options/>
9. mosquito-clients (v1.6.9-1): <https://packages.ubuntu.com/focal/mosquitto-clients>
10. Package: amqp-tools (0.10.0-1): <https://packages.ubuntu.com/focal/amqp-tools>
11. Eclipse Mosquitto (Docker): https://hub.docker.com/_/eclipse-mosquitto
12. RabbitMQ helm chart: <https://github.com/bitnami/charts/tree/master/bitnami/rabbitmq>
13. Logstash helm chart: <https://github.com/elastic/helm-charts/tree/7.17/logstash>
14. Elasticsearch helm chart: <https://github.com/elastic/helm-charts/tree/7.17/elasticsearch>
15. HTTP client in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/http_client.py
16. AMQP client in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/amqp_client.py

Other Resources

17. MQTT client in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/mqtt_client.py
18. Logstash output plugins: <https://www.elastic.co/guide/en/logstash/current/plugins-outputs-elasticsearch.html>
19. Logstash input plugins: <https://www.elastic.co/guide/en/logstash/current/plugins-inputs-rabbitmq.html>
20. Sensor controller prototype in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/sensor_controller_proto.py
21. Actuator controller prototype in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/actuator_controller_proto.py
22. Logstash configuration file: https://github.com/tubskns/notelab-code/blob/master/server/logstash_conf.yml

Chapter 9

Lab 6—Integrating Sensors and Actuators with Edge and Cloud Tools



9.1 Prerequisites

Hardware

- Raspberry Pi 4 Model B
- Server
- Workstation
- Router

Software

- Raspberry Pi 4 Model B
 - Docker v20.10.8 [4]
 - Python v3.9 [5]
 - OpenSSH v8.4 [6]
- Server
 - curl v7.81.0 [7]
 - K3s v1.20.9+k3s1 [8]
 - OpenSSH v8.4 [6]
- Workstation
 - curl v7.81.0 [7]
 - OpenSSH v8.4 [6]
 - mosquitto-clients v1.6.9-1 [9]
 - amqp-tools v0.10.0-1 [10]

9.2 Setup

In *Lab 6*, we extend the previously implemented data connectors, by adding more advanced functionalities, such as data processing. When sensor data gets forwarded from edge to cloud, it can be interpreted, correlated, or optimized. For instance, we may forward an average temperature based on many measurements, as opposed to every sensing temperature value. Another example is processing with data correlation, where we send the distance values only if we detect movements. Similarly, actuators can be triggered based on processed data. In this lab, we learn how to implement and integrate such functionalities by defining different options on how to interpret sensor data and trigger the actuators. The setup for this lab is shown in Fig. 9.1, and it includes Raspberry Pi, WiFi router, workstation, and server.

This lab includes two parts, which use the components shown in Fig. 9.2. In the first part, the *sensor controller* component is going to be extended with *data processing* component. Compared to the code previously implemented in Lab 5, in this lab, *sensor controller* processes the data before sending it out. In the second part, it is the actuator controller which is extended by processing the data received from the cloud, before forwarding it to the actuators. Since this lab focuses on extending the functionalities of both *sensor controller* and *actuator controller*, the workflows and tools to be used remain the same as in Lab 5, with the addition of data processing functionality. Similar to *Lab 5*, for testing purposes, we use `amqp-tools` to publish test AMQP messages to AMQP broker, followed with the utilization of `curl` to retrieve the data sent in these messages from the HTTP server. In our final Lab 7, we then replace the testing tool `amqp-tools` with real sensors and actuators.

A detailed overview of the libraries that were previously developed and are used in this lab is listed as follows:

- MQTT broker
 - Docker v1.6.15 [11]

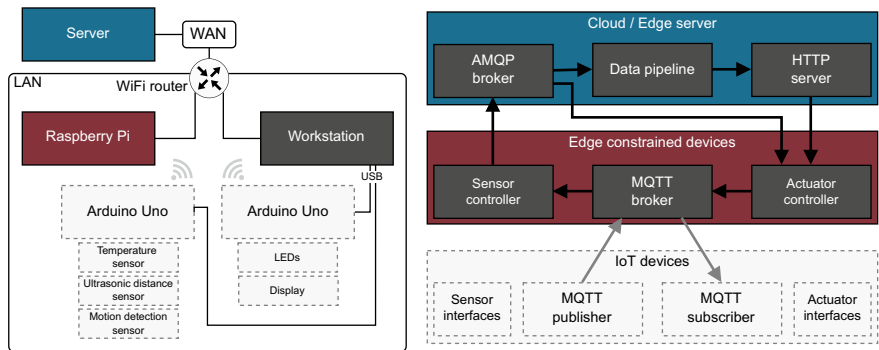


Fig. 9.1 Lab 6 setup

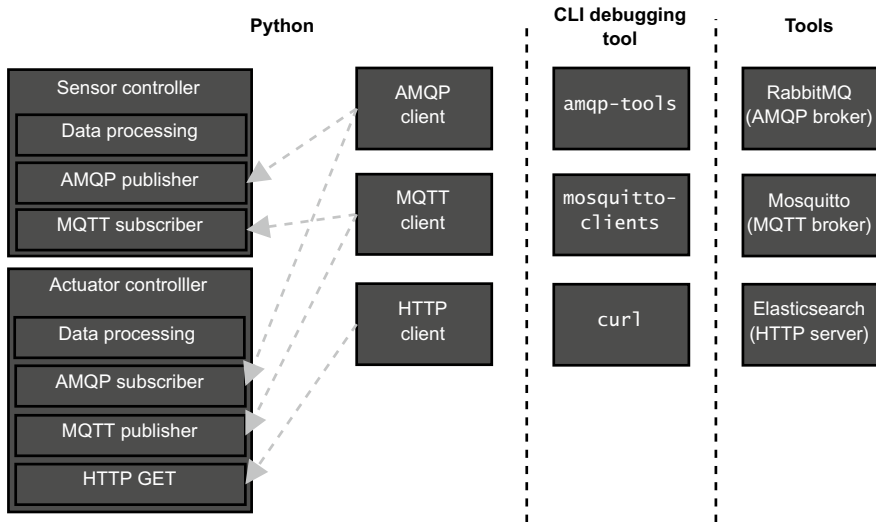


Fig. 9.2 Lab 6 components

- AMQP broker
 - RabbitMQ helm chart v10.1.14 [12]
- HTTP server
 - Elasticsearch helm chart v7.17.3 [13]
- *sensor controller* and *actuator controller*
 - Paho library v1.6.1 [1]
 - Pika library v1.2.1 [2]
 - Requests library v2.25.1 [3]
 - AMQP client [15] (from Lab 3)
 - HTTP client [14] (from Lab 4)
 - MQTT client [16] (from Lab 5).

9.3 Part 1: Sending Data from Sensors to Edge and Cloud Tools

The objective of this part is to extend the *sensor_controller* module by adding data processing, instead of simply forwarding the data. We again make use of the MQTT and AMQP clients previously implemented in *Lab 5* and in *Lab 3*, respectively. With *mosquitto-clients*, we are sending data to three different topics one per type of data from each sensor type. The *sensor_controller*, which subscribes to this broker,

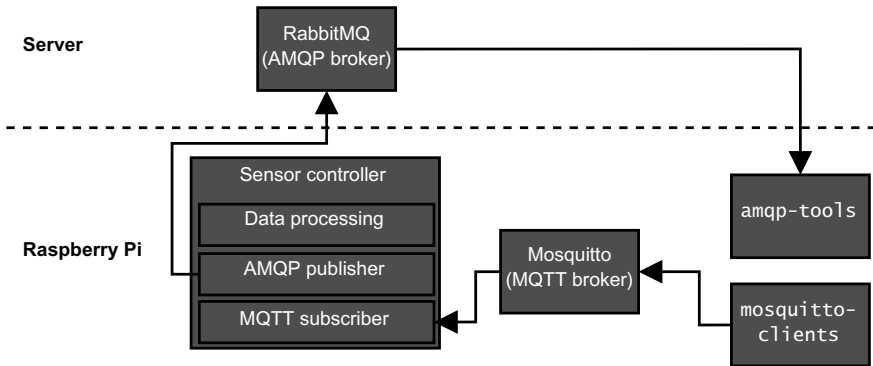


Fig. 9.3 Lab 6, part 1 components

is then going to process the sensor data before forwarding it to the AMQP broker. We use `amqp-tool` to check that the data is properly sent to the AMQP broker. Before starting with Task 1, we make sure that Mosquitto and RabbitMQ brokers are up and running on Raspberry Pi and server, respectively. The workflow to test is represented in Fig. 9.3.

9.3.1 Task 1: Implementing and Testing the Sensor Controller

We now create a new folder for this lab (`sensor_controller`) on the Raspberry Pi and we add there the AMQP client (`amqp_client.py`), implemented in Sect. 6.4.1 of Lab 3, and MQTT client (`mqtt_client.py`), implemented in Sect. 8.3 of Lab 5. Next, create a new script with name `sensor_controller.py`, and add the following code:

Source code: [17]

```
1 import mqtt_client
2 import amqp_client
3 import json, time
4 import logging
5
6
7 ip_rabbit = "SERVER_IP"
8 port_rabbit = 30672
9 user_rabbit = "user"
10 pass_rabbit = "password"
11 ex_rabbit = "amq.topic"
```

```

12 rkey_rabbit_temp = "temperature"
13 queue_rabbit_temp = "temperature"
14 rkey_rabbit_dist = "dist_rabbit"
15 queue_rabbit_dist = "dist_rabbit"
16 rkey_rabbit_motion = "motion_rabbit"
17 queue_rabbit_motion = "motion_rabbit"
18
19 ip_mosquitto = "Rpi_IP"
20 topic_temp = "temp_topic"
21 topic_dist = "dist_topic"
22 topic_motion = "motion_topic"
23 topics_mosquitto = [
24     (topic_temp, 1),
25     (topic_dist, 1),
26     (topic_motion, 1),
27 ]
28
29 IS_ACTIVATED = False
30 activation_interval = 5
31 start_time = 1000000000
32 temp_values = []
33 logging.basicConfig(format="%(asctime)s | %(name)s | %(levelname)s |
    %(message)s", level=logging.INFO)
34
35 amqp_ch = amqp_client.connect_to_broker(ip_rabbit, port=port_rabbit,
    user=user_rabbit, passw=pass_rabbit)
36 amqp_client.create_queue(amqp_ch, ex_rabbit, rkey_rabbit_temp,
    queue_rabbit_temp)
37 amqp_client.create_queue(amqp_ch, ex_rabbit, rkey_rabbit_dist,
    queue_rabbit_dist)
38 amqp_client.create_queue(amqp_ch, ex_rabbit, rkey_rabbit_motion,
    queue_rabbit_motion)
39
40 mqtt_client.message = None
41 mqttc = mqtt_client.connect(ip_mosquitto)
42 mqttc.subscribe(topics_mosquitto)
43 mqttc.loop_start()
44 while True:
45     if mqtt_client.message is not None:
46         topic = mqtt_client.message.topic
47         msg =
            json.loads(str(mqtt_client.message.payload.decode("utf-8")))
48     if IS_ACTIVATED:
49         elapsed_time = time.time() - start_time
50         if elapsed_time > activation_interval:
51             logging.debug("Activation interval is over")
52             IS_ACTIVATED = False
53             logging.debug("STOPPING collection of distance...")
54             msg_motion = {}
55             msg_motion["motion"] = False
56             amqp_ch.basic_publish(ex_rabbit, rkey_rabbit_motion,
                json.dumps(msg_motion))
57             logging.debug("RESETING distance to 0... ")
58             msg_dist = {}
59             msg_dist["distance"] = 0.0

```

```

60         amqp_ch.basic_publish(ex_rabbit, rkey_rabbit_dist,
                               json.dumps(msg_dist))
61     if topic == topic_motion:
62         if msg["motion"] and IS_ACTIVATED == False:
63             logging.debug("STARTING collection of distance...")
64             IS_ACTIVATED = True
65             start_time = time.time()
66             amqp_ch.basic_publish(ex_rabbit, rkey_rabbit_motion,
                                   json.dumps(msg))
67     elif topic == topic_temp:
68         if len(temp_values) < 24:
69             temp_values.append(msg["temperature"])
70         else:
71             logging.debug("Sending average temperature... ")
72             avg_temp = sum(temp_values) / len(temp_values)
73             msg["temperature"] = avg_temp
74             amqp_ch.basic_publish(ex_rabbit, rkey_rabbit_temp,
                                   json.dumps(msg))
75             temp_values = []
76     elif topic == topic_dist:
77         if IS_ACTIVATED:
78             logging.debug("SENDING distance... ")
79             amqp_ch.basic_publish(ex_rabbit, rkey_rabbit_dist,
                                   json.dumps(msg))
80     mqtt_client.message = None

```

Code descriptions: [17]

The script starts by importing MQTT and AMQP clients implemented in previous labs `mqtt_client`, `amqp_client`. Then we also need `json` library, for dealing with JSON format, `time` library, to calculate elapsed time, and `logging` library, to easily define the format of the logging messages. Next, we define the required parameters to connect to AMQP and MQTT brokers. The AMQP parameters include the IP of the server where RabbitMQ is running `ip_rabbit`, the port `port_rabbit`, the user `user_rabbit`, password `pass_rabbit`, and the exchange `ex_rabbit`. Then, we need to add three routing keys and three queues, where data from different sensors is going forwarded. `rkey_rabbit_temp` and `queue_rabbit_temp` for DHT11 temperature data; `rkey_rabbit_dist` and `queue_rabbit_dist` for HCSR04 distance data; and `rkey_rabbit_motion` and `queue_rabbit_motion` for PIR motion data. As MQTT parameters, we define `ip_mosquitto` and three topics in `topics_mosquitto`, one per type of sensor data. Additionally, we define four global variables,

`IS_ACTIVATED` which is a boolean which will be used for storing the status of whether motion is detected or not, `activation_interval` to set the interval time after which the detection of motion will be checked again, `start_time` to store the starting time when the motion is detected, and an empty array `temp_values` for temporally storing temperature values.

Next, from logging library, we use `basicConfig()` method to configure the format of logging messages and keep the log level to `INFO` to see information messages. For full logging details, change the level to `DEBUG`. Then, we first connect to RabbitMQ passing server IP address, port, and authentication credentials. Next, we create three RabbitMQ queues, corresponding to each of the sensors. For each queue, we pass the created AMQP channel, exchange, and corresponding routing key and queue name. Then, using MQTT client, we first initialize the global message and then connect to the Mosquitto broker by specifying the IP address which returns an instance of the MQTT client, and then we subscribe to all topics.

Data processing: From this point on, the main loop starts where we check for new messages. In case a new message is received, we retrieve the topic and the payload, considering that payload needs to be decoded first to a string. We add here a condition to check if variable `IS_ACTIVATED` is true, in which case we check the elapsed time. If the elapsed time exceeds the activation interval, then we make `IS_ACTIVATED` false to terminate the activation interval. However, in order to make the actuator controller know that the activation interval is over, we still need to notify them. So, we create a new message for motion with value set to false, and distance to value set to zero, and we forward it to their specific queues in RabbitMQ.

In all cases, regardless of whether `IS_ACTIVATED` is true or false, we still need to check what is the type of message we received from MQTT. So, in case the topic matches `"motion_topic"`, we check if the motion value is `True` to trigger activation by making the variable `IS_ACTIVATED` true. At this point, we save the current time into `start_time`, since it is at this point when we start measuring time from when the activation interval starts, and then we publish to RabbitMQ.

In case the topic matches `"temp_topic"`, we first check whether the length of `"temp_values"` is shorter than 24 values. If true, we just add the received value into the array. If false, we calculate the average temperature value from all 24 values stored in the array and send it to RabbitMQ. The array is then reset, for the next iteration.

If the topic matches `"dist_topic"`, we first check if the variable `IS_ACTIVATED` is true, which means that the motion activation interval is on, so only in this case, we forward the message values to RabbitMQ.

Change in `sensor_controller.py` the logging level to `DEBUG` value so all debugging messages are printed out. Then, run the script as the following:

```
ubuntu@raspberrypi: ~/sensor_controller:~$
python3 sensor_controller.py
```

Check in RabbitMQ dashboard, under chart *Queues* that all the queues defined in the script have been created and are binded to the corresponding routing keys, as shown here in Fig. 9.4.

Then, now run three test AMQP subscribers from three separate terminals on the workstation, specifying the server IP, port, user credentials, exchange type, and three routing keys, specified in the script as follows:

```
student@workstation:~$ amqp-consume --url amqp://user:
password@SERVER_IP:30672 -e amq.topic -r motion_rabbit cat
```

```
student@workstation:~$ amqp-consume --url amqp://user:
password@SERVER_IP:30672 -e amq.topic -r dist_rabbit cat
```

```
student@workstation:~$ amqp-consume --url amqp://user:
password@SERVER_IP:30672 -e amq.topic -r temperature cat
```

Also in the workstation, open a new terminal to run a test Mosquitto publisher, sending the following set of messages to different topics specified in *sensor_controller* as follows:

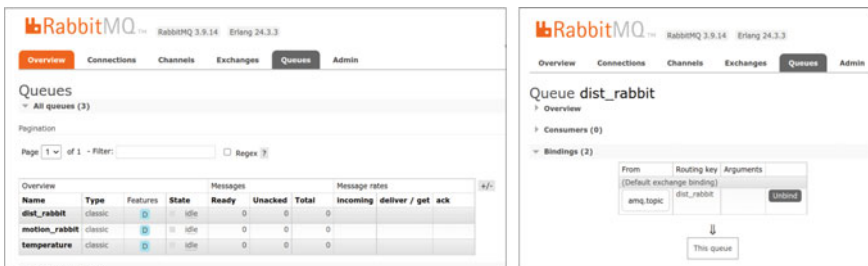


Fig. 9.4 RabbitMQ queues corresponding to the sensors

```

ubuntu@workstation:~$ mosquitto_pub -h RPi_IP -t "motion_topic" -m "{\"motion\":false}"
ubuntu@workstation:~$ mosquitto_pub -h RPi_IP -t "dist_topic" -m "{\"distance\":2}"
ubuntu@workstation:~$ mosquitto_pub -h RPi_IP -t "motion_topic" -m "{\"motion\":true}"
ubuntu@workstation:~$ mosquitto_pub -h RPi_IP -t "dist_topic" -m "{\"distance\":5}"
ubuntu@workstation:~$ mosquitto_pub -h RPi_IP -t "temp_topic" -m "{\"temperature\":2}"
ubuntu@workstation:~$ mosquitto_pub -h RPi_IP -t "dist_topic" -m "{\"distance\":5}"
ubuntu@workstation:~$ mosquitto_pub -h RPi_IP -t "temp_topic" -m "{\"temperature\":22}" --repeat 25

```

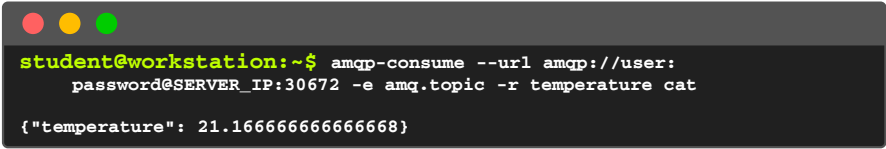
After publishing these messages, the logs from *sensor_controller* should show an output similar to the following one:

```

ubuntu@workstation:~/sensor_controller:~$ python3 sensor_controller.py
...
.....| DEBUG | AMQP - Queue declare and bind: motion_rabbit
.....| INFO | MQTT - connected to broker, code 0
.....| INFO | MQTT - subscribed: 1 (1, 1, 1)
...
.....| DEBUG | MQTT - msg received [motion_topic]: {"motion":false}
.....| DEBUG | MQTT - msg received [dist_topic]: {"distance":2}
.....| DEBUG | MQTT - msg received [motion_topic]: {"motion":true}
.....| DEBUG | STARTING collection of distance...
...
.....| DEBUG | MQTT - msg received [dist_topic]: {"distance":5}
.....| DEBUG | SENDING distance...
...
.....| DEBUG | MQTT - msg received [temp_topic]: {"temperature":2}
.....| DEBUG | Activation interval is over
.....| DEBUG | STOPPING collection of distance...
...
.....| DEBUG | RESETTING distance to 0...
...
.....| DEBUG | MQTT - msg received [dist_topic]: {"distance":5}
.....| DEBUG | MQTT - msg received [temp_topic]: {"temperature":22}
.....| DEBUG | MQTT - msg received [temp_topic]: {"temperature":22}
...
.....| DEBUG | Sending average temperature...

```

We can also check that the output of AMQP consumer is as expected, for instance,



```
student@workstation:~$ amqp-consume --url amqp://user:
password@SERVER_IP:30672 -e amq.topic -r temperature cat
{"temperature": 21.166666666666668}
```

After completing this task, learn how to delete the messages from RabbitMQ. This can be done using the option `Purge Messages` under `Queues` in the RabbitMQ dashboard. The purging of messages is often necessary for testing purposes.

Assignment 1

Answer the following questions:

- (a) Are there any issues in our approach of converting MQTT messages using `json.loads()`? Is there a more robust way of doing the same? Reason your answer.
- (b) Why do we need `json.loads()` and `json.dumps()`? What would happen if the message received does not follow the expected JSON schema? How would we solve that?
- (c) Explain in your own words the main purpose of implementing the *sensor_controller*.

Add your answers to the lab report.

9.4 Part 2: Triggering Actuators from Cloud and Edge Tools

In this part, we extend the *actuator_controller* module by adding data processing, instead of simply forwarding the data. Here, we make use of MQTT, AMQP, and HTTP clients previously implemented in *Lab 5*, *Lab 3*, and *Lab 4*, respectively. We are going to send test messages to Elasticsearch using `curl` and use `amqp-tools` to publish data to AMQP broker. The *actuator_controller*, which subscribes to AMQP broker and retrieves data from Elasticsearch, is going to process the data before forwarding it to MQTT broker. Before starting, make sure that Mosquitto, RabbitMQ, and Elasticsearch are up and running. The workflow to test is illustrated in Fig. 9.5).

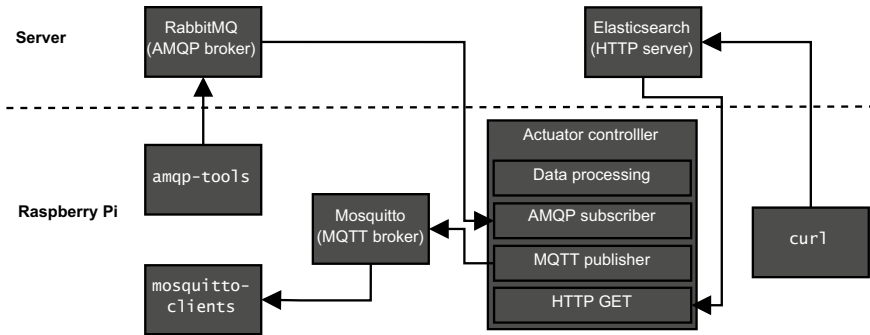


Fig. 9.5 Lab 6, part 2 components

9.4.1 Task 2: Implementing and Testing the Actuator Controller

Create a new folder for this lab (*actuator_controller*), and add the HTTP client (`http_client.py`), implemented in Sect. 7.4.1 of Lab 4, AMQP client (`amqp_client.py`), implemented in Sect. 6.4.1 of Lab 3, as well as MQTT client (`mqtt_client.py`), implemented in Sect. 8.3 of Lab 5. Next, create a new script named `actuator_controller.py` and add the following code:

Source code: [18]

```

1 import mqtt_client
2 import amqp_client
3 import http_client
4 import json
5 import logging
6
7 def on_motion_message(channel, method, properties, msg):
8     global IS_ACTIVATED
9     msg = json.loads(str(msg.decode("utf-8")))
10    if msg is not None:
11        if msg["motion"]:
12            logging.debug("STARTING collection of distance... ")
13            IS_ACTIVATED = True
14        else:
15            logging.debug("STOPPING collection of distance... ")
16            IS_ACTIVATED = False
17        mqttc.publish(motion_led_topic, json.dumps(msg))
18
19
20 def on_dist_message(channel, method, properties, msg):
21     msg = str(msg.decode("utf-8"))

```

```

22     if msg is not None:
23         if IS_ACTIVATED:
24             logging.debug("SENDING distance... ")
25             mqttc.publish(dist_led_topic, msg)
26             res_es = http_client.query(url_elastic + "/" +
27                                     index_elastic + "/_search", method="GET")
28             if res_es is not None:
29                 if res_es["hits"]["total"]["value"] > 0:
30                     hits = res_es["hits"]["hits"]
31                     last_hit = hits[len(hits) - 1]
32                     hit_source = json.dumps(last_hit["_source"])
33                     if hit_source is not None:
34                         logging.debug("SENDING last temperature
35                                     value... ")
36                         mqttc.publish(temp_led_topic, hit_source)
37
38 url_elastic = "http://SERVER_IP:32200"
39 index_elastic = "temperature"
40
41 ip_rabbit = "SERVER_IP"
42 port_rabbit = 30672
43 user_rabbit = "user"
44 pass_rabbit = "password"
45 exchange_rabbit = "amq.topic"
46 dist_r_key_rabbit = "dist_rabbit"
47 dist_queue_rabbit = "dist_rabbit"
48 motion_r_key_rabbit = "motion_rabbit"
49 motion_queue_rabbit = "motion_rabbit"
50
51 ip_mosquitto = "RPI_IP"
52 motion_led_topic = "motion_led_topic"
53 temp_led_topic = "temp_led_topic"
54 dist_led_topic = "dist_led_topic"
55
56 IS_ACTIVATED = False
57 logging.basicConfig(format="%(asctime)s | %(name)s | %(levelname)s |
58                     %(message)s", level=logging.INFO)
59
60 mqttc = mqtt_client.connect(ip_mosquitto)
61
62 amqp_client.message = None
63 amqp_channel = amqp_client.connect_to_broker(ip_rabbit,
64                                             port=port_rabbit, user=user_rabbit, passw=pass_rabbit)
65 amqp_client.subscribe(amqp_channel, dist_queue_rabbit,
66                     on_dist_message)
67 amqp_client.subscribe(amqp_channel, motion_queue_rabbit,
68                     on_motion_message)
69
70 try:
71     amqp_channel.start_consuming()
72 except KeyboardInterrupt:
73     amqp_channel.stop_consuming()
74 amqp_channel.close()

```

Code descriptions: [18]

The script starts by importing the MQTT, AMQP, and HTTP client scripts, `mqtt_client`, `amqp_client`, `http_client` implemented in previous labs. We also import the `json` library for dealing with JSON messages and `logging` for formatting logs. The script contains two functions that are triggered when AMQP messages are received for motion data and distance data.

- `on_motion_message()`: This function is triggered each time the AMQP consumer receives a message from a motion sensor using four arguments: `channel`, `method`, `properties`, and `msg`, from where only the `msg` (AMQP message body) is going to be used for now. *Data processing*: We define a global variable `IS_ACTIVATED` for saving the status of whether motion is detected or not. Next, we deserialize the received `msg` into a Python dictionary. Then, we check the message is not `None`, otherwise we could fail while trying to access the values. Then, we check the value of motion, and if so, we make the variable `IS_ACTIVATED` true or false otherwise. In both cases, we publish to the MQTT broker the motion message.
- `on_dist_message()`: This function continues with data processing and is triggered each time AMQP consumer receives a message from a distance sensor using four arguments: `channel`, `method`, `properties`, and `msg`, from where only the `msg` (AMQP message body) is used for now. The first step is to decode `msg` and convert it to a string type. In this case, we do not need to use the `JSON` library because we are not going to access the specific fields. Then, we check whether the message is not `none`, before the evaluation of the content. By using the previously defined global variable `IS_ACTIVATED`, we then check if the motion sensor is activated, and if so, we publish the message to the MQTT broker by specifying the distance topic. Next, we perform a `query()` using the `http_client` to retrieve the temperature data from Elasticsearch, using the `GET` method. If the response is not empty, we check that the total number of hits in that specific index returns a positive integer, otherwise we do not need to go further. Only when hits exist, so that some temperature data exists on Elasticsearch, we then retrieve the last value by decoding it into `hit_source`. This variable is now our actual temperature payload. Finally, we use `mqttc` client to publish the data to the temperature topic in MQTT with the corresponding payload.

Next, we define the required parameters to connect to AMQP and MQTT brokers and Elasticsearch. For connecting to Elasticsearch, this includes a URL and the name of the index where the data has been indexed, in this case, temperature. Next, for connect-

ing to AMQP parameters, it includes the IP address of the server where RabbitMQ is running `ip_rabbit`, port `port_rabbit`, user `user_rabbit`, password `pass_rabbit`, and exchange `ex_rabbit`. We then define two routing keys and two RabbitMQ queues, from which the data sent from different sensor types is going to be retrieved `dist_r_key_rabbit` and `dist_queue_rabbit` for retrieving distance data and `motion_r_key_rabbit` and `motion_queue_rabbit` for retrieving motion data. For connecting to Mosquitto, we include `ip_mosquitto` of the broker and three topic variables for publishing the data to actuators (`motion_led_topic`, `temp_led_topic`, and `dist_led_actuator`). We also define an additional global variable, `IS_ACTIVATED` where we are going to save the status sent by the motion sensor. Next, we make use of `basicConfig()` method to configure log messages, setting the log level to `INFO`. After defining parameters, we connect to MQTT broker by specifying the IP address of the broker and to RabbitMQ by specifying the IP address of the server, port, and authentication credentials. This is followed by subscribing the client to RabbitMQ's distance and motion queues with `subscribe()` function specifying the parameters. Finally, we start consuming AMQP messages arriving at the channel in a `try-catch` block. In order to close the channel properly, an exemption is triggered if we forcefully stop the program.

In order to test this component, first check that there are two queues with the same name as defined in the scripts `dist_rabbit` and `motion_rabbit` in RabbitMQ dashboard and that they are binded using the specified message routing keys from the script (`dist_rabbit` and `motion_rabbit`) under *Binding* tab. (If we are continuing right after the previous Task 1, this should already be the case.) Make sure that these queues are empty. In case the queues with these names and bindings cannot be found, we can create them in *Queues* tab of the RabbitMQ dashboard, using the option *Add a new queue*. Once they are created, we go to each of these two queues and under the option *Bindings* choose *Add binding to this queue*, specifying `amq.topic` as the exchange and the routing keys from the script, before pressing the option *Bind*.

Then, from the workstation, we use `curl` to send data with HTTP POST method to Elasticsearch. Since based on the script the temperature data is stored in Elasticsearch, for the endpoint, we use the topic `temperature`. The data is now represented as a test JSON message containing two fields, *id* and *temperature*:


```
student@workstation:~$  
curl -H "Content-Type: application/json" -XPOST "http://SERVER_IP  
:32200/temperature/1" -d '{"id":"test_temp_sensor", "temperature":22}'  
  
{ "_index": "temperature", "_type": "1", "_id": "prZVz4EB95vEHVMCgSQw", "_version"  
: 1, "result": "created", "_shards": { "total": 2, "successful": 1, "failed": 0 }, "  
_seq_no": 0, "_primary_term": 1 }
```

Set the logging level to DEBUG mode and run the script by using the following command thereafter:

```
ubuntu@raspberrypi:~/actuator_controller:~$  
python3 actuator_controller.py
```

Now run test MQTT subscribers from three separate terminals on the workstation, specifying the IP address of MQTT broker, port, and the three MQTT topics specified in the script.

```
student@workstation:~$ mosquitto_sub -h RPi_IP -t "motion_led_topic"
```

```
student@workstation:~$ mosquitto_sub -h RPi_IP -t "dist_led_topic"
```

```
student@workstation:~$ mosquitto_sub -h RPi_IP -t "temp_led_topic"
```

Now open a new terminal on the workstation to run a test AMQP publishers, sending a following set of messages to different routing keys specified in *actuator_controller* script:

```

student@workstation:~$
amqp-publish --url amqp://user:password@SERVER_IP:30672 -e amq.topic -r
motion_rabbit -b "{ \"motion\":false}"
student@workstation:~$
amqp-publish --url amqp://user:password@SERVER_IP:30672 -e amq.topic -r
dist_rabbit -b "{ \"distance\":5}"
student@workstation:~$
amqp-publish --url amqp://user:password@SERVER_IP:30672 -e amq.topic -r
dist_rabbit -b "{ \"distance\":2}"
student@workstation:~$
amqp-publish --url amqp://user:password@SERVER_IP:30672 -e amq.topic -r
dist_rabbit -b "{ \"distance\":4}"
student@workstation:~$
amqp-publish --url amqp://user:password@SERVER_IP:30672 -e amq.topic -r
motion_rabbit -b "{ \"motion\":true}"
student@workstation:~$
amqp-publish --url amqp://user:password@SERVER_IP:30672 -e amq.topic -r
dist_rabbit -b "{ \"distance\":5}"
student@workstation:~$
amqp-publish --url amqp://user:password@SERVER_IP:30672 -e amq.topic -r
dist_rabbit -b "{ \"distance\":2}"

```

After publishing these messages, the logs from *actuator_controller* should show an output similar to the following:

```

ubuntu@raspberrypi:~/actuator_controller:~$
python3 actuator_controller.py
...
.....| INFO | AMQP - Subscribed to: motion_rabbit
...
.....| DEBUG | STOPPING collection of distance...
.....| DEBUG | MQTT - published: 1
...
.....| DEBUG | STARTING collection of distance...
.....| DEBUG | MQTT - published: 2
...
.....| DEBUG | SENDING distance...
.....| DEBUG | MQTT - published: 3
.....| DEBUG | Starting new HTTP connection (1): SERVER_IP:32200
.....| DEBUG | \url{http://SERVER_IP:32200} "GET /temperature/_search HTTP
/1.1" 200 226
.....| DEBUG | HTTP - [GET][200][http://SERVER_IP:32200/temperature/_search]
.....| DEBUG | SENDING last temperature value...
.....| DEBUG | MQTT - published: 4
...
.....| DEBUG | SENDING distance...
.....| DEBUG | MQTT - published: 5
.....| DEBUG | Starting new HTTP connection (1): SERVER_IP:32200
.....| DEBUG | \url{http://SERVER_IP:32200} "GET /temperature/_search HTTP
/1.1" 200 226
.....| DEBUG | HTTP - [GET][200][SERVER_IP:32200/temperature/_search]
.....| DEBUG | SENDING last temperature value...
.....| DEBUG | MQTT - published: 6
...

```

As expected, the output shows that each time a message is published to the distance queue in RabbitMQ, *actuator_controller* publishes to Mosquitto broker. When the published motion value is true, temperature data retrieved from Elasticsearch and distance data retrieved from RabbitMQ are also published to Mosquitto. We can also check that these outputs correspond to the expected messages arriving at the MQTT subscribers:

```
student@workstation:~$ mosquitto_sub -h RPi_IP -t "motion_led_topic"
{"motion": false}
{"motion": true}
```

```
student@workstation:~$ mosquitto_sub -h RPi_IP -t "dist_led_topic"
{ "distance":5}
{ "distance":2}
```

```
student@workstation:~$ mosquitto_sub -h RPi_IP -t "temp_led_topic"
{"id": "test_temp_sensor", "temperature": 22}
{"id": "test_temp_sensor", "temperature": 22}
```

Assignment 2

Answer the following questions:

- Why do we need to constantly check that variables are not None in the *actuator_controller* code? What would happen if we removed these lines?
- What would happen if the database was empty by the time we try to retrieve the temperature values in *actuator_controller*? Would the proposed implementation fail? Explain the reasoning behind the answer.
- Explain in your own words the main purpose of implementing *actuator_controller* component.

Add the answers to the lab report.

References

Libraries

1. Paho Python Client: <https://www.eclipse.org/paho/index.php?page=clients/python/index.php>
2. Pika library: <https://pika.readthedocs.io/en/stable/>
3. Requests library: <https://requests.readthedocs.io/en/latest/user/install/>

Tools

4. Docker Engine installation on Ubuntu: <https://docs.docker.com/engine/install/ubuntu/>
5. Python: <https://www.python.org/downloads/>
6. OpenSSH Server: <https://ubuntu.com/server/docs/service-openssh>
7. CURL: <https://curl.se/download.html>
8. K3s container orchestrator: <https://rancher.com/docs/k3s/latest/en/installation/install-options/>
9. mosquitto-clients (v1.6.9-1): <https://packages.ubuntu.com/focal/mosquitto-clients>
10. Package: amqp-tools (0.10.0-1): <https://packages.ubuntu.com/focal/amqp-tools>
11. Eclipse Mosquitto (Docker): https://hub.docker.com/_/eclipse-mosquitto
12. RabbitMQ helm chart: <https://github.com/bitnami/charts/tree/master/bitnami/rabbitmq>
13. Elasticsearch helm chart: <https://github.com/elastic/helm-charts/tree/7.17/elasticsearch>

Other Resources

14. HTTP client in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/http_client.py
15. AMQP client in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/amqp_client.py
16. MQTT client in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/mqtt_client.py
17. Sensor controller in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/sensor_controller.py
18. Actuator controller in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/actuator_controller.py

Chapter 10

Lab 7—Network-of-Things Final Setup



10.1 Prerequisites

Hardware

- Sensors
 - (1x) - DHT11 temperature humidity sensor
 - (1x) - HC-SR04 Ultrasonic sensor
 - (1x) - HC-SR501 PIR motion sensor
- Actuators
 - (3x) - 5 mm LEDs (red, green, and yellow)
 - (1x) - 16 × 2 LCD module
- Auxiliary components
 - (3x) - resistors (330 ohm)
 - (1x) - resistor (10000 ohm)
 - (2x) - breadboards
 - (10x) - wire jumpers (min. 15 cm)
- (2x) - Arduino Uno WiFi Rev2
- Raspberry Pi 4 Model B
- Server
- Workstation
- Router

Software

- Raspberry Pi 4 Model B
 - Docker v20.10.8 [9]

- Python v3.9 [10]
- OpenSSH v8.4 [11]
- Server
 - curl v7.81.0 [12]
 - K3s v1.20.9+k3s1 [17]
 - OpenSSH v8.4 [11]
- Workstation
 - curl v7.81.0 [12]
 - Arduino IDE v1.8.13 [13]
 - OpenSSH v8.4 [11]
 - mosquitto-clients v1.6.9-1 [14]
 - amqp-tools v0.10.0-1 [15].

10.2 Setup

Congratulations! We are at the final step in engineering our IoT-edge-cloud system with all NoTE Lab components! *Lab 7* integrates all software and hardware previously developed and used into a final *Network-of-Things Lab Setup*. The setup for this lab is shown in Fig. 10.1, and it includes two Arduino boards, Raspberry Pi, WiFi router, workstation, and server, along with all previously used sensors and actuators.

This lab is divided into two parts and uses all previously used components as shown in Fig. 10.2. In the first part, we connect not only one sensor as in previous tasks, but also all three sensors together in one Arduino board and we publish data using an MQTT publisher. The second Arduino board integrates four actuators and receives data using an MQTT subscriber. In Part 2, we finally integrate and test all components of the final setup, and we collect and process data in real-world IoT

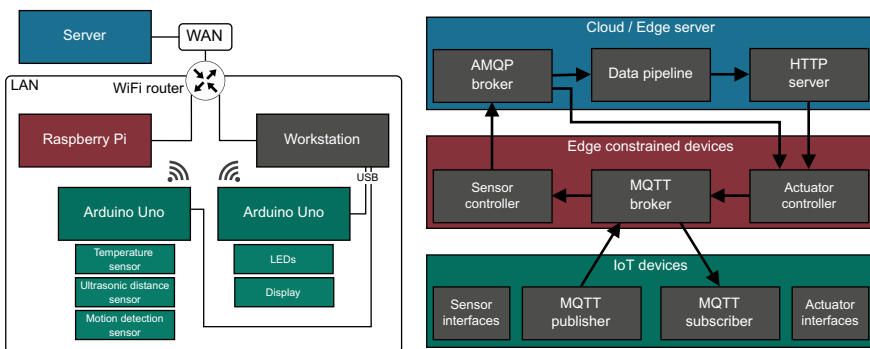


Fig. 10.1 Lab 7 setup

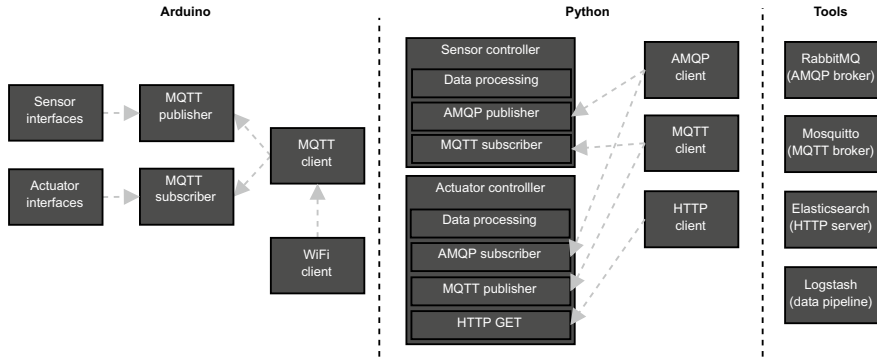


Fig. 10.2 Lab 7 components

and edge devices, over to cloud tools. A detailed overview of the libraries that were previously developed and are also going to be used in this lab is listed as follows:

- Sensors interfaces
 - DHT11 library v1.4.3 [2]
 - HCSR04 Arduino header [21] and source file [22] (from Lab 1)
 - PIR Arduino header [23] and source file [24] (from Lab 1)
- Actuator interfaces
 - Liquid Crystal_I2C library v1.1.2 [1]
- MQTT publisher and subscriber on Arduino
 - Wi-FiNINA library v1.8.13 [3]
 - Wi-Fi client Arduino header [25] and source file [26] (from Lab 0)
 - Arduino MQTT library v2.5.0 [4]
 - MQTT client Arduino header [27] and source file [28] (from Lab 2)
- MQTT broker on Raspberry Pi
 - Eclipse Mosquitto v1.6.15 [16]
- AMQP broker on server
 - RabbitMQ helm chart v10.1.14 [18]
- Data pipeline on server
 - Logstash helm chart v7.17.3 [19]
- HTTP server
 - Elasticsearch helm chart v7.17.3 [20]

- *sensor controller* and *actuator controller* on Raspberry Pi
 - Paho library v1.6.1 [6]
 - Pika library v1.2.1 [7]
 - Requests library v2.25.1 [8]
 - AMQP client in Python [31] (from Lab 3)
 - HTTP client in Python [29] (from Lab 4)
 - MQTT client in Python [30] (from Lab 5) source v1.8.13 [1]
 - Sensor controller [32]
 - Actuator controller [33]

Make sure the required external libraries are installed in the system. In Arduino IDE under *Tools*, find the option *Manage Libraries*, from where we can search for the required libraries and have them automatically download and be installed by Arduino IDE. Wi-Fi-NINA library can be found under the full name *Wi-Fi-NINA* (source code available in [3]), DHT11 Library can be found under the full name *DHT sensor library* (source code available in [2]), Liquid Crystal I2C library can be found under the full name *Liquid Crystal I2C* (source code available in [1]), MQTT library can be found under the full name *MQTT library for Arduino* (source code available at [4]), and finally, Arduino JSON library can be found under the full name *Arduino_JSON Library for Arduino* (source code available at [5]).

10.3 Part 1: Integrating Different Sensors and Actuators

In this part of the lab, we use two Arduino boards. In one board, we integrate the code for reading data from all three sensors. In the other board, we integrate the code for controlling the actuators. The board interfacing the sensors publishes the data to independent topics to Mosquitto broker, which runs on Raspberry Pi. The board interfacing actuators will then subscribe to those topics and activate actuators accordingly. Before starting with Task 1, make sure that Mosquitto is up and running on the Raspberry Pi. The workflow to test is illustrated in Fig. 10.3

10.3.1 Task 1: Integrating all Sensors

In this task, we send sensor data using MQTT publisher. Consider revisiting *Lab 1* for details about how to interface each type of the sensor. Then, connect all three sensors to one Arduino as shown in Fig. 10.4.

We now implement a new sketch in Arduino. This sketch combines the already developed functions implemented in previous labs and tasks. Start by creating a new folder for the project (e.g., `publisher_all`), and within create a main sketch file with the same filename (e.g., `publisher_all.ino`). Copy in this directory also the other libraries implemented in previous labs: HCSR04 library (.cpp and .h

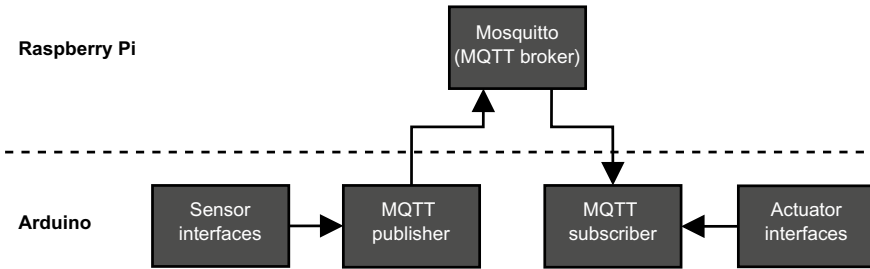


Fig. 10.3 Lab 7, part 1 components

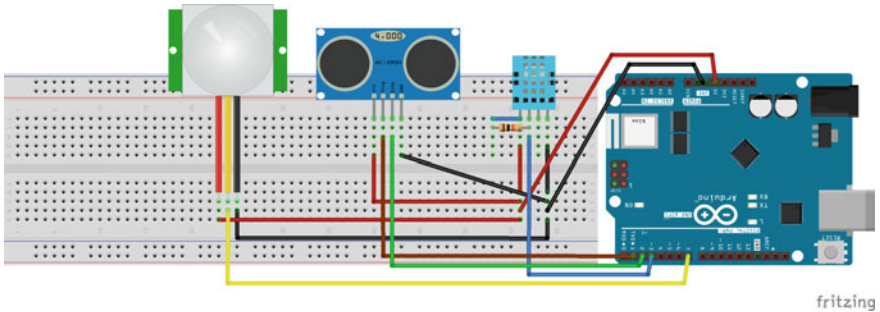


Fig. 10.4 Interfacing and wiring all sensors to Arduino

files), PIR library (.cpp and .h files), WifiClient library (.cpp and .h files), and MQTTClient library (.cpp and .h files). By copying these libraries, we can here reuse all of their functions, without any modifications. As a result, when opening the main sketch with Arduino IDE, all nine files will open.

Add the following code into the main sketch file `publisher_all.ino`:

Source code: [35]

```

1 #include "MqttClient.h"
2 #include "WifiClient.h"
3 #include "HCSR04.h"
4 #include "PIR.h"
5 #include "DHT.h"
6 #include <ArduinoJson.h>
7
8 char *ssid_wifi = "netw0";
9 char *pass_wifi = "password1";
10
11 const char *mqtt_broker_ip = "Rpi_IP";

```

```

12 const int mqtt_broker_port = 1883;
13 const char* client_id = "publisher_sensors";
14 char* temp_topic = "temp_topic";
15 char* dist_topic = "dist_topic";
16 char* motion_topic = "motion_topic";
17 WifiClient wifi_client(ssid_wifi, pass_wifi);
18 MqttClient mqtt_client(mqtt_broker_ip,
    mqtt_broker_port);
19
20 #define DHTPIN 3 //specify DHT11's pin
21 #define DHTTYPE DHT11 // DHT11's type
22 DHT dht(DHTPIN, DHT11);
23 HCSR04 hcsr04(1, 2); // specify HC-SR04 trig and
    echo pins
24 PIR pir(7); // specify PIR's pin
25
26 DynamicJsonDocument temp_doc(1024),
    dist_doc(1024), motion_doc(1024);
27
28 void setup(){
29     Serial.begin(9600);
30     wifi_client.connect();
31     mqtt_client.connect(client_id);
32     dht.begin(); // initialize the sensor
33 }
34
35 void loop(){
36     mqtt_client.check_connection(client_id);
37     float temperature = dht.readTemperature();
38     if (!isnan(temperature)){
39         temp_doc["id"] = client_id;
40         temp_doc["temperature"] = temperature;
41     }
42     float distance = hcsr04.calculate_distance();
43     dist_doc["id"] = client_id,
        dist_doc["distance"] = distance;
44     bool is_motion = pir.detect_motion();
45     motion_doc["id"] = client_id,
        motion_doc["motion"] = is_motion;
46     char temp_out[500], dist_out[500],
        motion_out[500];
47     serializeJson(temp_doc, temp_out);
48     serializeJson(dist_doc, dist_out);
49     serializeJson(motion_doc, motion_out);
50     mqtt_client.publish_message(temp_topic,
        temp_out);
51     mqtt_client.publish_message(dist_topic,
        dist_out);
52     mqtt_client.publish_message(motion_topic,
        motion_out);
53 }

```

Code descriptions: [35]

The first step is to include our libraries: `MqttClient.h`, `WifiClient.h`, `HCSR04.h`, and `PIR.h` libraries. In addition, we also include external libraries `DHT.h` and `ArduinoJson.h`. We then first define the usual variables for connecting to the wireless network (network ssid and password), and for connecting to the MQTT broker (IP address and port). We define `client_id` and MQTT topics to which we assign values of corresponding sensors ("*temp_topic*", "*dist_topic*", and "*motion_topic*"). Then we instantiate `WifiClient` by passing the network SSID and password, and we also instantiate `MqttClient` by passing the IP address of the MQTT broker and port number. Next, we define sensor-specific pin variables and create corresponding objects for each of the sensors. This includes instantiation of `DHT` by specifying its pin and the type of sensor, in our case `DHT11`. `HC-SR04` and `PIR` instances are also initialized by specifying the proper pins. As data is published in JSON format, we define `DynamicJsonDocuments` for storing sensor messages after they are serialized. From this point on, we use the following functions:

- `setup()`: In this function, we open serial communication at baud rate of 9600 *bps*, and we trigger `connect()` functions for both `mqtt_client` and `wifi_client` defined class objects (for connecting the Arduino to the MQTT broker and to WLAN). We also initialize `DHT11` sensor by calling function `begin()` from the imported `DHT` sensor library.
- `loop()`: Here, we first trigger `check_connection()` function for `mqtt_client` class object to check if the connection to the broker is up and running, while passing the arguments of Arduino's client ID. We then trigger functions for reading data from each of the sensors individually (functions `readTemperature()`, `calculate_distance()`, and `detect_motion()`), saving the returned values into float for temperature, integer for distance, and boolean for motion. Since `readTemperature()` may return null values, we have to first check that the returned value is valid. Next, we prepare the message fields (client id and sensor data) and serialize the documents into string values. Finally, we publish the three messages corresponding to the three topics.

Now, proceed to *Verify* to check the code is without errors and then *Upload* the code in order to flash it on the Arduino. For more details about this process, please, go back to Lab 0, Sect. 3.4.1.

We can now use *Serial Monitor* to check the output. If everything goes without errors, the output should look like this:

```

Connecting to WPA SSID [netw1]... done, IP: 192.168.1.5
Connecting to MQTT broker... done!
Message published [ temp_topic ]: {"id":"publisher_sensors"
    ,"temperature":26}
Message published [ dist_topic ]: {"id":"publisher_sensors"
    ,"distance":5}
Message published [ motion_topic ]: {"id":"
    publisher_sensors","motion":true}
....

```

Assignment 1

Answer the following questions:

- (a) When importing libraries, what is the purpose of creating our own HCSR04 library? Is there any other option available to us?
- (b) Do we also create our own DHT library? Why?

Add the answers to the lab report.

10.3.2 Task 2: Integrating all Actuators

We now use MQTT subscriber to trigger the actuators on the other Arduino board. Consider revising *Lab 1* to get reminded about details of the actuators used, i.e., three LEDs and one LCD display. We connect three LEDs and LCD display to Arduino board based on Fig. 10.5. Pay attention to the choice of resistors depending on LED color.

Let us write a sketch to subscribe to the topics previously created by the sensors and then trigger the actuators, accordingly. Start by creating a new folder for the project (e.g., `subscriber_all`), and in that folder, create the main sketch file with the same filename (e.g., `subscriber_all.ino`). Add `WifiClient` (.cpp and .h files) and `MqttClient` (.cpp and .h files). By copying these libraries, we reuse all their functions without any modifications. As a result, when opening the main sketch with Arduino IDE, all files open automatically.

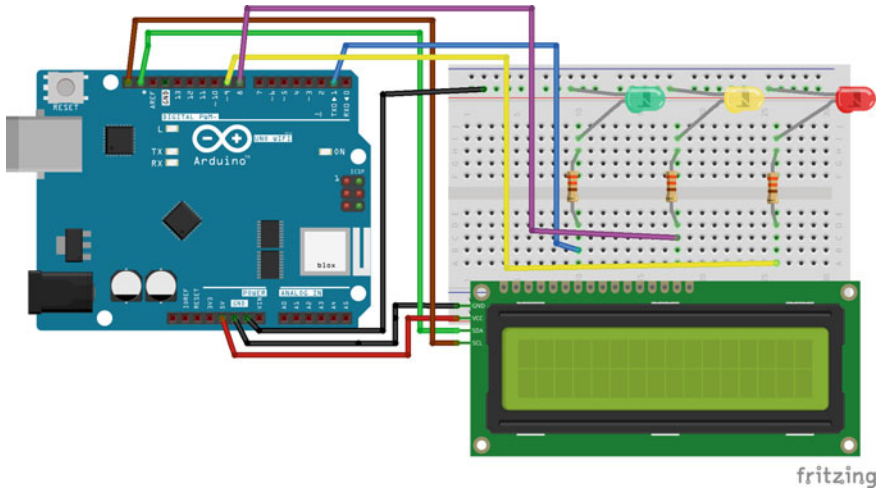


Fig. 10.5 Connecting all actuators to Arduino

Add the following code into the main sketch file `subscriber_all.ino`:

Source code: [36]

```

1 #include "MqttClient.h"
2 #include "WifiClient.h"
3 #include <ArduinoJson.h>
4 #include <Wire.h>
5 #include <LiquidCrystal_I2C.h>
6
7 char *ssid_wifi = "netw0";
8 char *pass_wifi = "password1";
9
10 const char *mqtt_broker_ip = "RPI_IP";
11 const int mqtt_broker_port = 1883;
12 const char *client_id = "subscriber_sensors";
13 const int num_subscribe_topics = 3;
14 String subscribe_topics[num_subscribe_topics] =
15     {"motion_topic", "dist_topic", "temp_topic"};
16 uint8_t leds[3] = {1, 8, 9}; // green, yellow,
    red
17 WifiClient wifi_client(ssid_wifi, pass_wifi);

```

```

17 MqttClient mqtt_client(mqtt_broker_ip,
    mqtt_broker_port, subscribe_topics,
    num_subscribe_topics);
18
19 DynamicJsonDocument msg_doc(1024);
20 float prev_temp = 0;
21 int led2_state = LOW;
22 long led2_interval = 1000000;
23 unsigned long prev_millis = 0;
24 LiquidCrystal_I2C lcd(0x27,16,2);
25
26 void setup()
27 {
28     Serial.begin(9600);
29     wifi_client.connect();
30     mqtt_client.connect(client_id);
31     for (int i = 0; i < (sizeof(leds) /
        sizeof(leds[0])); i++)
32         pinMode(leds[i], OUTPUT);
33     lcd.init();
34     lcd.backlight();
35 }
36
37 void loop()
38 {
39     mqtt_client.check_connection(client_id);
40     String msg = mqtt_client.get_msg();
41     String topic = mqtt_client.get_topic();
42     unsigned long current_millis = millis();
43     deserializeJson(msg_doc, msg);
44     if (topic == subscribe_topics[0])
45     {
46         bool motion = msg_doc["motion"];
47         if (motion == true){
48             digitalWrite(leds[0], HIGH);
49             lcd.setCursor(13, 0);
50             lcd.print("ON ");
51         }
52         else{
53             digitalWrite(leds[0], LOW);
54             lcd.setCursor(13, 0);
55             lcd.print("OFF");
56         }
57     }
58     else if (topic == subscribe_topics[1])
59     {
60         if (current_millis - prev_millis >=
            led2_interval)
61         {
62             prev_millis = current_millis;
63             if (led2_state == LOW)

```

```
64         led2_state = HIGH;
65     else
66         led2_state = LOW;
67     digitalWrite(leds[1], led2_state);
68 }
69 float distance = msg_doc["distance"];
70 if (distance <= 3.0)
71     led2_interval = 500;
72 else if (distance > 3.0 && distance < 15)
73     led2_interval = 2000;
74 else {
75     led2_interval = 1000000;
76     digitalWrite(leds[1], LOW);
77     distance = 0;
78 }
79 lcd.setCursor(0, 1); // second row
80 lcd.print("Dist. ");
81 String distance_str = String(distance, 1);
82 lcd.print(distance_str);
83 lcd.print(" cm");
84 }
85 else if (topic == subscribe_topics[2])
86 {
87     float temperature = msg_doc["temperature"];
88     if (temperature != prev_temp)
89     {
90         digitalWrite(leds[2], HIGH);
91         prev_temp = temperature;
92     }
93     else
94         digitalWrite(leds[2], LOW);
95     lcd.setCursor(0, 0); // first row
96     lcd.print("Temp. ");
97     String temperature_str = String(temperature,
98                                     1);
99     lcd.print(temperature_str);
100    lcd.print((char) 223); // celsius char
101 }
102 else
103 { // ignore
104 }
105 mqtt_client.reset_msg();
106 }
```

Code descriptions: [36]

We first include our own libraries `MqttClient.h` and `WifiClient.h`. We then also include the external libraries `ArduinoJson.h`, `Wire.h`, and `LiquidCrystal_I2C.h`. Then, we define variables for connecting to the wireless network (network ssid and password) and for connecting to MQTT broker (IP address and port). We change `client_id`, for instance, to `"subscriber_sensors"`. Since this Arduino interfaces three LED actuators, we define one array for three topics matching the ones defined on the publisher and another array for the PINs where LEDs are connected. The topics array is limited with the defined variable `num_subscribe_topics` to which we assign as a value the number of topics. Next, we initialize `wifi_client` specifying the SSID and password, and we also initialize `mqtt_client` specifying the IP address of the MQTT broker, the port number, and also the topics to subscribe and the number of topics. Since data is published in JSON format, we define `DynamicJsonDocuments` for later deserializing the incoming messages. In addition, we define a float variable `prev_temp`, initialized to zero, for storing previous temperature values for comparison purposes. An integer variable `led2_state` for controlling the state of the second LED which will be used for distance, initialized to LOW value; a long variable `led2_interval` for setting the frequency of blinking of the second LED for distance; and unsigned long variable `prev_millis`, to store the last time LED was updated, initialized to zero. Next, we initialize an instance of `LiquidCrystal_I2C.h` library, setting the I2C address (0x27) and display's dimensions (16 columns and 2 rows). We then use the following functions:

- `setup()`: In this function, we open serial communication at baud rate of 9600 *bps*, and we trigger `connect()` functions for both `mqtt_client` and `wifi_client` defined class objects (for connecting the Arduino to MQTT broker and to the wireless network). Next, in a `for` loop, we initialize the outputs of the Arduino to activate three LEDs. To trigger LCD display, we use `init()` function to initialize it, followed by `backlight()` function, which turns on the backlight.
- `loop()`: In this function, we first trigger `check_connection()` function for `mqtt_client` class object to check if the connection is up and running, while passing the arguments of Arduino's client ID. Then we trigger `get_msg()` and `get_topic()` to retrieve the message and topic, respectively. Next, we store the current time into `current_millis`. Using `deserializeJson()`, we deserialize the message into an iterable object. *Data processing*: We next add conditions to trigger LEDs behavior, depending on the extracted topic and message values. In case the topic matches `"motion_topic"`, we retrieve the boolean

value and turn on the green LED if the motion is detected, and turn off otherwise. Using the LCD display functions `setCursor()` and `print()`, we display the retrieved motion values starting from the 13th column of the first row. In case the topic matches `"dist_topic"`, we first check if the current interval time is already longer than the configured interval, by calculating the time difference between `current_millis` and `prev_millis`. If true, then we first store the current time as the previous one, and we change the state of the LED to the opposite state. We then retrieve the corresponding message value into `(distance)`. Then, we add `if-else` statement to change the configured interval accordingly to the measured distance value. Here we also use the same functions to display the distance values, but this time from the beginning of the second row. In case the topic matches `"temp_topic"`, we retrieve the corresponding message value from the deserialized dictionary, storing it into a float (`temperature`) variable. We add an `if` statement to activate the red LED only when the newly received temperature value is different from the previous one. Again, the temperature values are displayed using LCD functions, starting from the first column and first row. As the last step, we use function `reset_msg()` from our `mqtt_client` library to clear the message and topic values.

Now, proceed to *Verify* to check the code is without errors and then *Upload* the code in order to flash it on the Arduino. For more details about this process, please, go back to Lab 0, Sect. 3.4.1.

We can now use *Serial Monitor* to check the output. If everything goes without errors, the output should show MQTT client connecting to the broker, subscribing to three different topics, and receiving messages published on that topics. Note that the publisher needs to be running for "Message arrived" output, see below.

```
Connecting to WPA SSID [netw0]... done, IP: 192.168.1.12
Connecting to MQTT broker... done!
Subscribed to topic: motion_topic
Subscribed to topic: dist_topic
Subscribed to topic: temp_topic
Message arrived [ temp_topic ]: {"id":"publisher_sensors",
    "temperature":26}
Message arrived [ dist_topic ]: {"id":"publisher_sensors",
    "distance":5}
Message arrived [ motion_topic ]: {"id":"publisher_sensors",
    "motion":true}
}
...
```

Assignment 2

Answer the following questions:

- (a) What is the benefit, if any, of using the same MQTT library for both publisher and subscriber? Is there any other approach?
- (b) In order to make the yellow LED to blink at a certain frequency, we are using a method that calculates elapsed time. Is there any other way of achieving the same blinking effect? Is there any problem or limitation with your proposed approach?

Add the answers to the lab report.

10.4 Part 2: Final Setup—Integrating IoT, Edge to Cloud

In this final setup, we test and benchmark the complete system, including all the devices, tools, and components. The workflow to test is illustrated in Fig. 10.6.

In this workflow, Arduino interfacing the sensors with MQTT publisher first sends the sensor values to MQTT broker running on Raspberry Pi. In Raspberry Pi, *sensor_controller* collects and processes sensor data and forwards it to AMQP broker running on the server. Logstash pipeline indexes the temperature data into Elasticsearch, and motion and distance values are collected by *actuator_controller* also

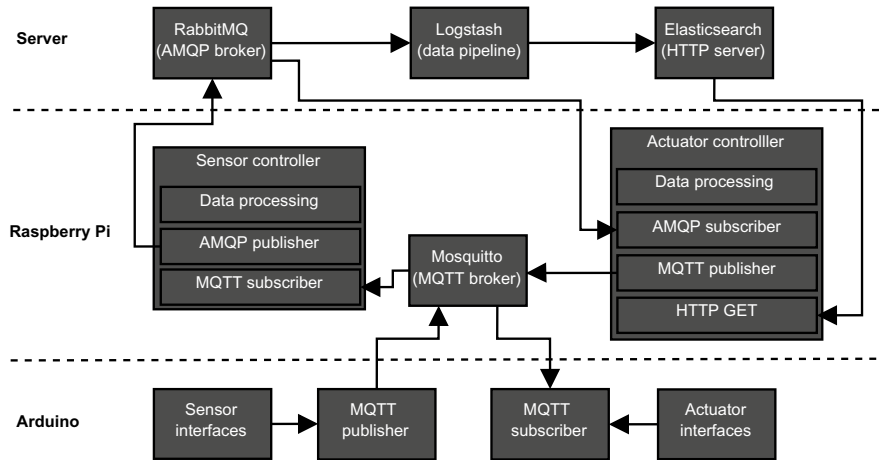


Fig. 10.6 Lab 7, part 2 components

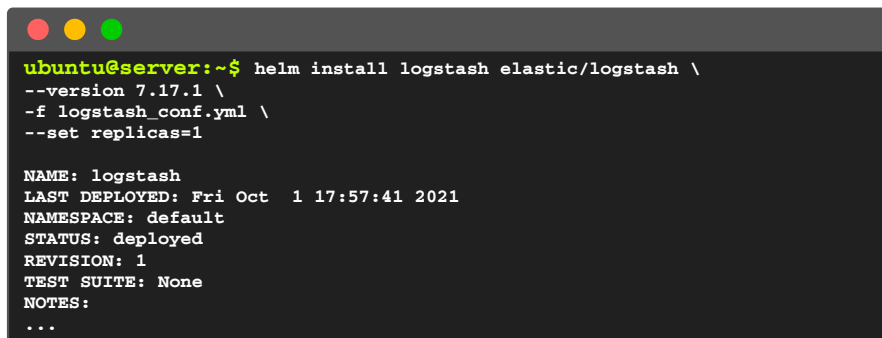
running on Raspberry Pi. This component retrieves the temperature values from Elasticsearch whenever motion is detected and forwards all values to MQTT broker accordingly. Arduino interfacing the actuators will then receive those values and trigger the actuators accordingly.

10.4.1 Task 3: Testing and Benchmarking

Before starting our final task, let us make sure that Mosquitto, RabbitMQ, and Elasticsearch are up and running. RabbitMQ and Elasticsearch are to be restarted from the beginning, making sure that the old containers have been deleted.

Step 1 - Modify Logstash Configuration File

Before installing Logstash, following the same steps as we learned in Part 4 of *Lab 5*, we need to first create a new folder (e.g., `logstash_config_integration`). In this folder, we create the same configuration file in YAML format, `logstash_conf.yml`, as provided in *Lab 5* [35], here however by making a few small modifications to specify the input and output of the logstash pipeline. In the input part, we change the queue name to `temperature_logstash`, and in the output, we change the index where we store the data to the same name—`temperature_logstash`. After saving the file, install Logstash as previously explained in Part 4 (Sect. 8.6.1) of Lab 5.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The terminal shows a command to install Logstash using Helm, followed by the output of the installation. The output includes the chart name, last deployment time, namespace, status, revision, test suite, and notes.

```
ubuntu@server:~$ helm install logstash elastic/logstash \
--version 7.17.1 \
-f logstash_conf.yml \
--set replicas=1

NAME: logstash
LAST DEPLOYED: Fri Oct  1 17:57:41 2021
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
...
```

Check in RabbitMQ dashboard, under chart *Queues* that the queue defined in the configuration file has been created, as shown here in Fig. 10.7.

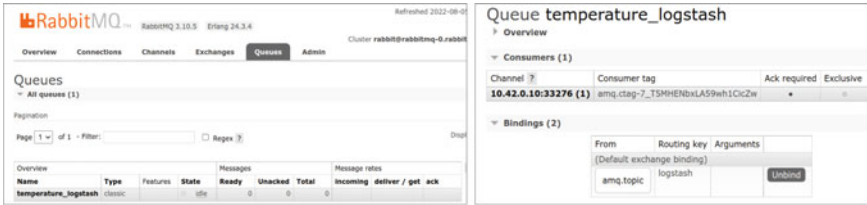


Fig. 10.7 RabbitMQ queue corresponding to the Logstash configuration

Step 2—Sending Data from Sensors Using MQTT Publisher

Next, we use the script for publishing all sensor data `publisher_all.ino`, which runs on the Arduino interfacing the sensors, as in Part 1 Sect. 10.3.1 of Lab 6. Upload the code to the Arduino and use *Serial Monitor* to check that all sensor data are being published. Since there are no changes to the code provided in Task 10.3.1 of this lab, the output is going to be the same:

```
Connecting to WPA SSID [netw1]... done, IP: 192.168.1.5
Connecting to MQTT broker... done!
Message published [ temp_topic ]: {"id":"publisher_sensors"
  ,"temperature":28.5}
Message published [ dist_topic ]: {"id":"publisher_sensors"
  ,"distance":4.686}
Message published [ motion_topic ]: {"id":"
  publisher_sensors","motion":true}
....
```

Step 3—Processing Sensor Data

Now that the sensor data is being published to MQTT broker, we run the `sensor_controller.py` script on the Raspberry Pi to start processing the received data and forwarding the data to the AMQP broker, as in Part 2 Sect. 9.3.1 of Lab 6. A minor modification to the original script is necessary in order to store temperature data arriving at the RabbitMQ in Elasticsearch, using Logstash pipeline. It includes modifying the variables `queue_rabbit_temp` and `rkey_rabbit_temp` to match the Logstash configuration file values: `temperature_logstash` and `logstash`, respectively. Also, make sure that the logging level in `sensor_controller` is set to `DEBUG` value and run the script:

```

ubuntu@raspberrypi: /~/sensor_data_processingr:~$
python3 sensor_controller.py

...
.....| INFO | AMQP - Connected to broker: SERVER_IP:30672
...
.....| DEBUG | AMQP - Queue declare and bind: temperature_logstash_queue
...
.....| DEBUG | AMQP - Queue declare and bind: dist_rabbit
...
.....| DEBUG | AMQP - Queue declare and bind: motion_rabbit
.....| INFO | MQTT - connected to broker, code 0
.....| INFO | MQTT - subscribed: 1 (1, 1, 1)
.....| DEBUG | MQTT - msg received [dist_topic]: {"id":"publisher_sensors",
"distance":2.992}
.....| DEBUG | MQTT - msg received [motion_topic]: {"id":"publisher_sensors",
"motion":false}
.....| DEBUG | MQTT - msg received [temp_topic]: {"id":"publisher_sensors",
"temperature":28.5}
.....| DEBUG | MQTT - msg received [dist_topic]: {"id":"publisher_sensors",
"distance":2.992}
.....| DEBUG | MQTT - msg received [motion_topic]: {"id":"publisher_sensors",
"motion":true}
.....| DEBUG | STARTING collection of distance...
...
.....| DEBUG | MQTT - msg received [motion_topic]: {"id":"publisher_sensors",
"motion":true}
.....| DEBUG | MQTT - msg received [temp_topic]: {"id":"publisher_sensors",
"temperature":28.5}
.....| DEBUG | MQTT - msg received [dist_topic]: {"id":"publisher_sensors",
"distance":2.992}
.....| DEBUG | SENDING distance...
...
.....| DEBUG | MQTT - msg received [motion_topic]: {"id":"publisher_sensors",
"motion":false}
.....| DEBUG | MQTT - msg received [temp_topic]: {"id":"publisher_sensors",
"temperature":28.5}
.....| DEBUG | MQTT - msg received [dist_topic]: {"id":"publisher_sensors",
"distance":3.92}
.....| DEBUG | Activation interval is over
.....| DEBUG | STOPPING collection of distance...
...

```

Check in RabbitMQ dashboard that all the queues were declared, i.e., (*temperature_logstash*, *motion_rabbit*, and *dist_rabbit*). After this, we need to wait for some time, while sensor data is being published and making sure that the motion sensor detects motion changes. Check again the activity in the dashboard, as shown here in Figs. 10.8, 10.9, and 10.10.

In the *motion_rabbit* queue, you can see based on the activity when the changes in motion have been detected, and correspondingly when motion is detected, the *sensor controller* sends distance data to the *dist_rabbit* queue. For the temperature data arriving from the sensor, the *sensor controller* only sends the average value to RabbitMQ based on last 24 received values. Temperature data is also stored in the Elasticsearch, through Logstash. Now, check that the data was actually stored by making use of HTTP GET request:

```

student@workstation:~$ curl -XGET http://{SERVER_IP}:32200/
temperature_logstash/_search?pretty

....
"max_score" : 1.0,
"hits" : [
  {
    "_index" : "temperature_logstash_index",
    "_type" : "_doc",
    "_id" : "0QEP2YEBfein-R7ZPmdk",
    "_score" : 1.0,
    "_source" : {
      "temperature" : 28.629166666666663,
      "@version" : "1",
      "@timestamp" : "2022-07-07T14:27:43.305Z",
      "id" : "publisher_sensors"
    }
  }
]
....

```

Queue motion_rabbit

Overview

Queued messages last ten minutes ?



Message rates last ten minutes ?



Fig. 10.8 Motion data arriving at RabbitMQ



Fig. 10.9 Distance data arriving at RabbitMQ

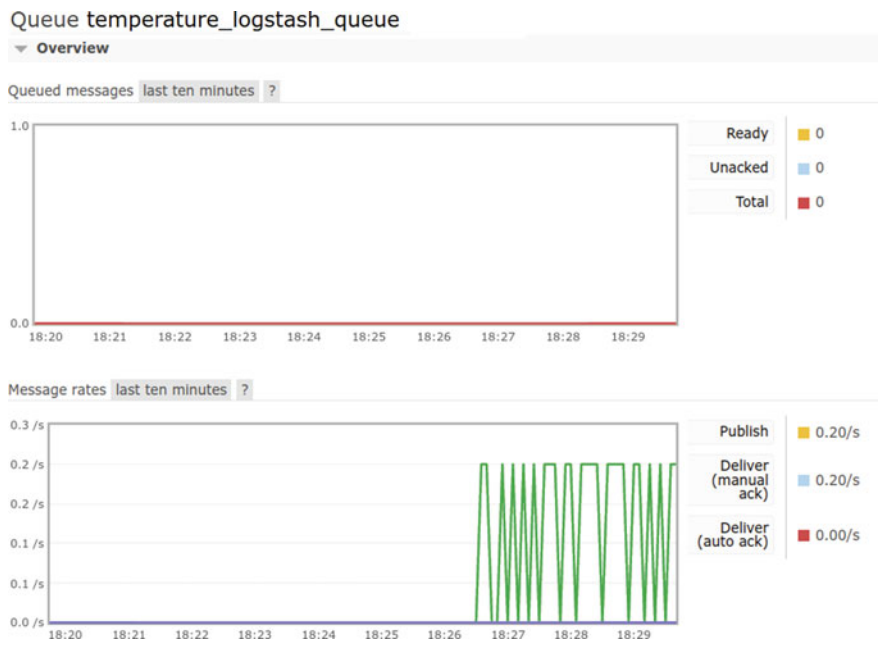


Fig. 10.10 Temperature data arriving at RabbitMQ

Step 4—Retrieving Data from the Server and Triggering Actuators

The next step is to retrieve the data from RabbitMQ and Elasticsearch in order to activate actuators accordingly. So, now we run the `actuator_controller.py` script on another terminal on the Raspberry Pi, as done in Part 3 Sect. 9.4.1 of Lab 6. A small modification to the original script is necessary in order to retrieve temperature data that is being stored in the Elasticsearch. This includes modifying the variable in logstash output pipeline `index_elastic` to the value specified in `temperature_logstash`. Also, make sure that the logging level in `actuator_controller` is set to `DEBUG` value before running the script:

```
ubuntu@raspberrypi: /~/actuator_controller:~$
python3 actuator_controller.py

.....| INFO | AMQP - Subscribed to: dist_rabbit
...
.....| INFO | AMQP - Subscribed to: motion_rabbit
```

Finally, we run the script for subscribing to different MQTT topics, based on the Arduino which interfaces LED actuators and LCD display, as done in Part 1 (Sect. 10.3.2). Before uploading the code, we modify the array for three topics (`subscribe_topics`) to match the ones defined in processing connector script `actuator_controller.py`. We use new values that correspond to `motion_led_topic`, `dist_led_topic`, and `temp_led_topic`. Use *Serial Monitor* to check that data is arriving at the Arduino:

```
Connecting to WPA SSID [netw1]... done, IP: 192.168.1.14
Connecting to MQTT broker... done!
Subscribed to topic: motion_led_topic
Subscribed to topic: dist_led_topic
Subscribed to topic: temp_led_topic
...
Message arrived [motion_led_topic]: {"id": "
  publisher_sensors", "motion": true}
Message arrived [dist_led_topic]: {"id": "publisher_sensors
  ", "distance": 2.686}
Message arrived [temp_led_topic]: {"@timestamp": "
  2022-07-09T14:17:36.118Z", "id": "publisher_sensors", "
  @version": "1", "temperature": 28.5}
Message arrived [motion_led_topic]: {"motion": false}
Message arrived [motion_led_topic]: {"id": "
  publisher_sensors", "motion": true}
Message arrived [dist_led_topic]: {"id": "publisher_sensors
  ", "distance": 2.822}
Message arrived [temp_led_topic]: {"@timestamp": "
  2022-07-09T14:17:36.118Z", "id": "publisher_sensors", "
  @version": "1", "temperature": 28.5}
```



```
Message arrived [dist_led_topic]: {"id": "publisher_sensors",
  "distance": 2.686}
...
```

The logs from *actuator_controller* should show an output similar to the following:

```
ubuntu@raspberrypi: ~/actuator_controller:~$
python3 actuator_controller.py

...
.....| DEBUG | SENDING distance...
.....| DEBUG | MQTT - published: 165
.....| DEBUG | Starting new HTTP connection (1): SERVER_IP:32200
.....| DEBUG | \url{http://SERVER_IP:32200} "GET /temperature/_search HTTP
/1.1" 200 226
.....| DEBUG | HTTP - [GET]\cite()[SERVER_IP:32200/temperature/_search]
.....| DEBUG | SENDING last temperature value...
.....| DEBUG | MQTT - published: 166
...
```

Let us now verify that the actuators are running as expected. Green LED should be turned on only if the motion is detected, and turned off otherwise after 5 s. Yellow LED should blink faster at short distances and slower at long distances, but only if before motion is detected, so that the green LED is ON too. Red LED should only blink if a new temperature average value is received. In LCD display, we can easily read and verify the measured values and see how the values shown match with the operation of the LEDs.

Assignment 3

Answer the following questions:

- (a) Since in this part both *sensor_controller* and *actuator_controller* are working together, why do we need to use in both components a boolean to determine that motion is detected? Wouldn't it be enough to consider only one of them? Explain in your own words either the reasons why we need the boolean in both components, or propose a new approach to make the current implementation better.
- (b) Why do we need to notify both motion and distance topics in *sensor_controller* when the motion is false? Would it be possible to just stop sending data and expect the same functionality? If yes, what would be the main drawback and how would we engineer the solution?

Add the answers to the lab report.

References

Libraries

1. Source code LiquidCrystal_I2C library https://github.com/johnrickman/LiquidCrystal_I2C
2. Source code DHT11 library: <https://github.com/adafruit/DHT-sensor-library>
3. Arduino Wi-FiNINA library source: <https://github.com/arduino-libraries/WiFiNINA>
4. Arduino MQTT library: <https://github.com/256dpi/arduino-mqtt>
5. Arduino JSON library: <https://github.com/bblanchon/ArduinoJson>
6. Paho Python Client: <https://www.eclipse.org/paho/index.php?page=clients/python/index.php>
7. Pika library: <https://pika.readthedocs.io/en/stable/>
8. Requests library: <https://requests.readthedocs.io/en/latest/user/install/>

Tools

9. Docker Engine installation on Ubuntu: <https://docs.docker.com/engine/install/ubuntu/>
10. Python: <https://www.python.org/downloads/>
11. OpenSSH Server: <https://ubuntu.com/server/docs/service-openssh>
12. CURL: <https://curl.se/download.html>
13. How to install Arduino IDE: <https://www.arduino.cc/en/Guide/Linux>
14. mosquito-clients (v1.6.9-1): <https://packages.ubuntu.com/focal/mosquitto-clients>
15. Package: amqp-tools (0.10.0-1): <https://packages.ubuntu.com/focal/amqp-tools>
16. Eclipse Mosquitto (Docker): https://hub.docker.com/_/eclipse-mosquitto
17. K3s container orchestrator: <https://rancher.com/docs/k3s/latest/en/installation/install-options/>
18. RabbitMQ helm chart: <https://github.com/bitnami/charts/tree/master/bitnami/rabbitmq>
19. Logstash helm chart: <https://github.com/elastic/helm-charts/tree/7.17/logstash>
20. Elasticsearch helm chart: <https://github.com/elastic/helm-charts/tree/7.17/elasticsearch>

Other Resources

21. HCSR04 Arduino header file: <https://github.com/tubskns/notelab-code/blob/master/arduino/sensors/distance/HCSR04/HCSR04.h>
22. HCSR04 Arduino source file: <https://github.com/tubskns/notelab-code/blob/master/arduino/sensors/distance/HCSR04/HCSR04.cpp>
23. PIR Arduino header file: <https://github.com/tubskns/notelab-code/blob/master/arduino/sensors/motion/PIR/PIR.h>
24. PIR Arduino source file: <https://github.com/tubskns/notelab-code/blob/master/arduino/sensors/motion/PIR/PIR.cpp>
25. Wi-Fi client Arduino header file: <https://github.com/tubskns/notelab-code/blob/master/arduino/WifiClient/WifiClient.h>
26. Wi-Fi client source file: <https://github.com/tubskns/notelab-code/blob/master/arduino/WifiClient/WifiClient.cpp>
27. MQTT client Arduino header file: <https://github.com/tubskns/notelab-code/blob/master/arduino/MqttClient/MqttClient.h>
28. MQTT client Arduino source file: <https://github.com/tubskns/notelab-code/blob/master/arduino/MqttClient/MqttClient.cpp>

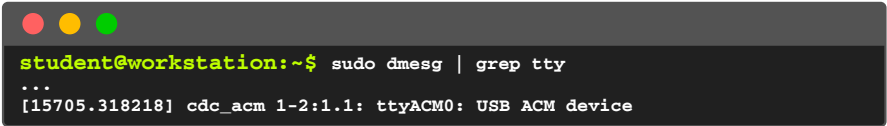
29. HTTP client in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/http_client.py
30. MQTT client in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/mqtt_client.py
31. AMQP client in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/amqp_client.py
32. Sensor controller in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/sensor_controller.py
33. Actuator controller in Python: https://github.com/tubskns/notelab-code/blob/master/rpi/actuator_controller.py
34. Logstash configuration file: https://github.com/tubskns/notelab-code/blob/master/server/logstash_conf.yml
35. MQTT publisher for all sensors Arduino sketch: https://github.com/tubskns/notelab-code/blob/master/arduino/sensors/publisher_all/publisher_all.ino
36. MQTT subscriber for all sensors Arduino sketch: https://github.com/tubskns/notelab-code/blob/master/arduino/actuators/subscriber_all/subscriber_all.ino

Appendix A

Troubleshooting: Arduino Uno

A.1 Connecting Arduinos to Workstation

When connecting Arduinos to the workstation using an USB cable, `dmesg` program can be used to check if the OS detects the device properly, and pipe with `grep` to filter the results. Open a terminal on the workstation and run `dmesg` command:

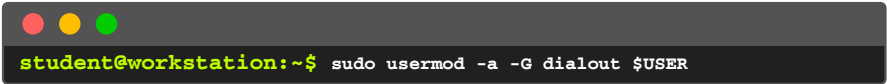
A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The prompt is 'student@workstation:~\$'. The command 'sudo dmesg | grep tty' is entered. The output shows '...' followed by '[15705.318218] cdc_acm 1-2:1.1: ttyACM0: USB ACM device'.

```
student@workstation:~$ sudo dmesg | grep tty
...
[15705.318218] cdc_acm 1-2:1.1: ttyACM0: USB ACM device
```

When a hardware device is connected or disconnected, `dmesg` command shows the logs. From the output, note the serial port name to which the Arduino is attached to; for Arduino Uno boards, this is usually either `ttyACM0` or `ttyACM1`.

A.2 Set Serial Port Permission

If when uploading the code, we face *Cannot open device: Permission denied* errors or *Error opening serial port '/dev/ttyX'*), we can run the following command to get access to the specific serial port:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The prompt is 'student@workstation:~\$'. The command 'sudo usermod -a -G dialout \$USER' is entered.

```
student@workstation:~$ sudo usermod -a -G dialout $USER
```

With this command, the user adds the current user account to `dialout` group. For the group change to take effect the first time the user needs to logout and login again.

If the error persists, use a simple Arduino setup script which is included with the official *arduino-1.8.X-linux64.tar.xz* file, named `arduino-linux-setup.sh`. Then, run it as follows:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The prompt is `student@workstation: /~/arduino-1.8.X:~$`. The command `./arduino-linux-setup.sh <username>` is entered on the next line.

```
student@workstation: /~/arduino-1.8.X:~$  
./arduino-linux-setup.sh <username>
```

For the change to take effect, reboot the workstation.

A.3 Alternative Arduino IDE Installation Method

In case the suggested version of Arduino IDE is not available anymore, you can alternatively download it from the latest release available in <https://github.com/tubskns/notelab-code/releases>.

A.4 Alternative Libraries Installation Method for Arduino IDE

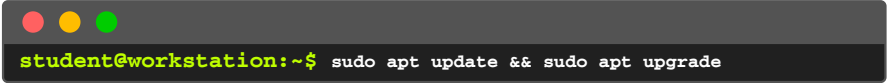
In case the specific external libraries are not available anymore, you can manually download the **.zip* files of each library from the latest release in <https://github.com/tubskns/notelab-code/releases>. Then, in the Arduino IDE, go to *Sketch*, then *Include Library*, then *Add .ZIP Library*, and select the **.zip* file of the library you want to import.

Appendix B

Troubleshooting: Raspberry Pi

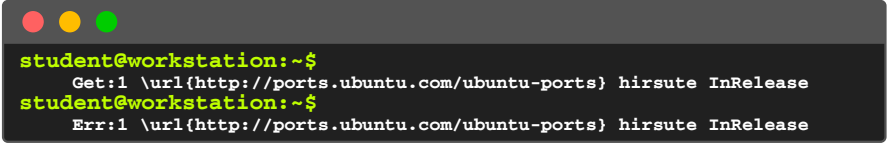
B.1 Setting Up Ubuntu and Clock Synchronization

After flashing Ubuntu on the SD card, it is highly recommended to update all packages with the following command:



```
student@workstation:~$ sudo apt update && sudo apt upgrade
```

If when updating Ubuntu we get the following errors:



```
student@workstation:~$  
Get:1 \url{http://ports.ubuntu.com/ubuntu-ports} hirsute InRelease  
student@workstation:~$  
Err:1 \url{http://ports.ubuntu.com/ubuntu-ports} hirsute InRelease
```

First check that Raspberry Pi has Internet access, for instance with `ping 1.1.1.1`. In case the connection is fine, check that timezone with the following command:

```
ubuntu@ubuntu:~$ timedatectl status

Local time: Thu 2021-04-22 00:20:01 UTC
Universal time: Thu 2021-04-22 00:20:01 UTC
RTC time: n/a
Time zone: UTC (UTC, +0000)
System clock synchronized: no
NTP service: active
RTC in local TZ: no
```

In this case, we need to update the timezone, for instance, to “Europe/Copenhagen” as follows:

```
ubuntu@ubuntu:~$ timedatectl set-timezone "Europe/Copenhagen"
```

Next set NTP value to zero, to make the service inactive while we change the date, and after that change the time and date based on “YYYY-MM-DD HH:MM:SS” format. Again set the NTP value to one to activate the NTP synchronization service.

```
ubuntu@ubuntu:~$ timedatectl set-ntp 0
ubuntu@ubuntu:~$ timedatectl set-time "YYYY-MM-DD HH:MM:SS"
ubuntu@ubuntu:~$ timedatectl set-ntp 1
```

The output of *timedatectl status* command should now be updated timezone and the update process should work as expected.

B.2 Enable SSH Service

Check the SSH service status with the following command:

```
ubuntu@ubuntu:~$ systemctl status sshd
sshd.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset
   : enabled)
   Active: active (running) since Mon 2022-05-09 16:58:58 CEST; 5min ago
     Docs: man:sshd(8)
           man:sshd_config(5)
   ...
```

In case the status is disabled, enable it with the following command:

```
ubuntu@ubuntu:~$ sudo systemctl start sshd
ubuntu@ubuntu:~$ sudo systemctl enable sshd
```

By default, SSH is configured to start at boot, and in case it is not up and running, start the SSH service with

```
ubuntu@ubuntu:~$ sudo systemctl start sshd
```

B.3 Docker Timeout Error

If when trying to deploy container in Docker, the following timeout exceeded error appears:

```
ubuntu@workstation:~$ docker run hello-world
Using default tag: latest
Error response from daemon: Get "https://registry-1.docker.io/v2/": net/http
: request canceled while waiting for connection (Client.Timeout
exceeded while awaiting headers)
```

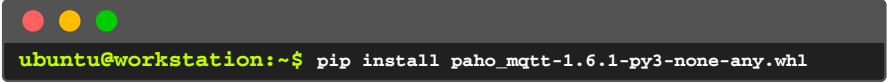
There may be a problem with DNS resolution. In the Ubuntu server, this can be solved by specifying custom DNS to, for instance, *1.1.1.1* and *8.8.8.8*. To do that, edit the network configuration and add *nameservers* as follows:

```
ubuntu@workstation:~$ sudo nano /etc/netplan/50-cloud-init.yaml

ethernets:
  eth0:
    dhcp4: true
    optional: true
    nameservers:
      addresses: [1.1.1.1, 8.8.8.8]
```


B.4 Alternative Python Packages Installation Method

When installing Python packages, in case the official suggested releases are not available anymore, you can manually download the Python packages from the latest release in <https://github.com/tubskns/notelab-code/releases> and install the *.whl files using `pip` command as usual. For instance, to install the package *paho_mqtt-1.6.1-py3-none-any.whl*, run the following command to install it:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The prompt is 'ubuntu@workstation:~\$' and the command entered is 'pip install paho_mqtt-1.6.1-py3-none-any.whl'.

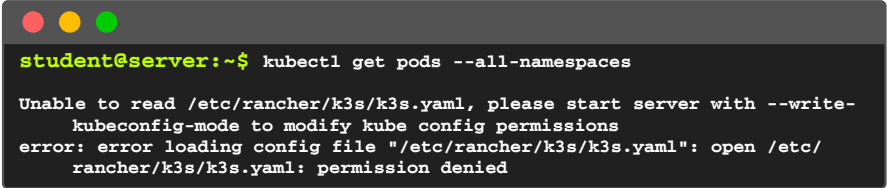
```
ubuntu@workstation:~$ pip install paho_mqtt-1.6.1-py3-none-any.whl
```

Appendix C

Troubleshooting: Server

C.1 Fixing k3s Permissions

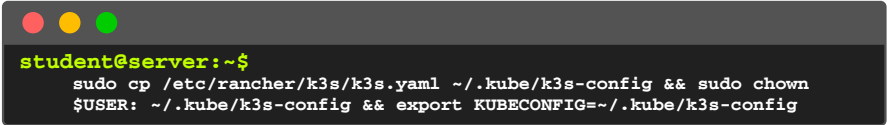
When trying to run `kubectl` commands without `sudo`, we may encounter the following error:



```
student@server:~$ kubectl get pods --all-namespaces

Unable to read /etc/rancher/k3s/k3s.yaml, please start server with --write-
  kubeconfig-mode to modify kube config permissions
error: error loading config file "/etc/rancher/k3s/k3s.yaml": open /etc/
  rancher/k3s/k3s.yaml: permission denied
```

While it is not recommended to run `kubectl` commands by a non-root user in production environments, for our testing environment, this is useful. So for the development of Note Lab, we can just fix k3s permissions in order to be able to run `kubectl` by a non-root user with the following command:



```
student@server:~$
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/k3s-config && sudo chown
$USER: ~/.kube/k3s-config && export KUBECONFIG=~/.kube/k3s-config
```

It is to be noted that we are setting an environment variable using `export` command, so you need to repeat the same command in every new terminal you open or add it to `.bashrc` or `.zshrc` files in order to be able to use `kubectl` commands without `sudo`.

C.2 Uninstalling k3s

To uninstall k3s together with all deployed services, run the following command:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The prompt is 'student@server:~\$' and the command entered is '/usr/local/bin/k3s-uninstall.sh'.

```
student@server:~$ /usr/local/bin/k3s-uninstall.sh
```

C.3 Alternative Helm Installation Method

If the official installation method for Helm fails, then first download the binary `helm-canary-linux-amd64.tar.gz` from the latest release in <https://github.com/tubskns/notelab-code/releases>. Then, unpack it and find the `helm` binary in the unpacked directory. From there `helm` can be used as usual or be moved to its recommended location in `/usr/local/bin/helm`.

C.4 Alternative K3s Installation Method

When installing K3s, in case the official installation method fails because the official release is not available anymore, then first download the k3s installation script from https://github.com/tubskns/notelab-code/blob/master/server/k3s_install.sh, and then run the script while adjusting the `INSTALL_K3S_VERSION` parameter to the latest available one from <https://github.com/tubskns/notelab-code/releases>:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The prompt is 'student@server:~\$' and the command entered is 'GITUB_URL=https://github.com/tubskns/notelab-code/releases INSTALL_K3S_VERSION=v1.0.0-alpha ./k3s_install.sh'.

```
student@server:~$ GITUB_URL=https://github.com/tubskns/notelab-code/releases INSTALL_K3S_VERSION=v1.0.0-alpha ./k3s_install.sh
```

C.5 Alternative Hello-World Installation Method

In case the suggested method for installing *hello-world* using the provided Helm chart fails, then first download *hello-world-0.2.1.tgz* from the latest release available in <https://github.com/tubskns/notelab-code/releases>, and then run the following command instead to deploy it:

```
student@server:~$ helm install my-helloworld helloworld-0.2.1.tgz \
--version 0.2.1 \
--set image.repository=ghcr.io/tubskns/http-echo
```

C.6 Alternative RabbitMQ Installation Method

In case the official suggested method for installing RabbitMQ using Helm charts fails, then first download *rabbitmq-10.1.14.tgz* from the latest release available in <https://github.com/tubskns/notelab-code/releases>, and then run the following command to deploy RabbitMQ:

```
student@server:~$ helm install rabbitmq rabbitmq-10.1.14.tgz \
--set replicaCount=1 \
--set auth.username=user,auth.password=password \
--set service.type=NodePort,service.nodePorts.amqp=30672,service.
nodePorts.manager=31672 \
--set image.registry=ghcr.io,image.repository=tubskns/rabbitmq
```

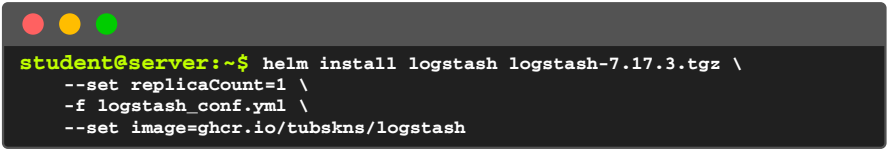
C.7 Alternative Elasticsearch Installation Method

In case the official method for installing Elasticsearch using the official Helm charts fails, then first download *elasticsearch-7.17.3.tgz* from the latest release available in <https://github.com/tubskns/notelab-code/releases>, and then run the following command to deploy Elasticsearch:

```
student@server:~$ helm install elasticsearch elasticsearch-7.17.3.tgz \
--set replicaCount=1 \
--set service.type=NodePort,service.nodePort=32200 \
--set image=ghcr.io/tubskns/elasticsearch
```

C.8 Alternative Logstash Installation Method

In case the official method for installing Logstash using the official Helm charts fails, then first download *logstash-7.17.3.tgz* from the latest release available in <https://github.com/tubskns/notelab-code/releases>, and then run the following command to deploy Logstash:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The prompt is 'student@server:~\$'. The command being entered is 'helm install logstash logstash-7.17.3.tgz \', followed by three lines of flags: '--set replicaCount=1 \', '-f logstash_conf.yml \', and '--set image=ghcr.io/tubskns/logstash'.

```
student@server:~$ helm install logstash logstash-7.17.3.tgz \  
--set replicaCount=1 \  
-f logstash_conf.yml \  
--set image=ghcr.io/tubskns/logstash
```